ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
Εθνικόν και Καποδιστριακόν
Πανεπιστήμιον Αθηνών
ΙΔΡΥΘΕΝ ΤΟ 1837

Τμήμα Φυσικής

Εφαρμογή τεχνικών Μηχανικής Μάθησης για την κατηγοριοποίηση γεγονότων στα ανοιχτά δεδομένα του πειράματος ATLAS

Παύλος Παναγιωτόπουλος

Επιβλέπων Καθηγητής:
Δημήτριος Φασσουλιώτης

Ομάδα ΕΚΠΑ-ATLAS

Αθήνα, Οκτώβριος 2026

Περίληψη

Η Μηχανική Μάθηση αποτελεί αναπόσπαστο κομμάτι των σύγχρονων πειραμάτων Φυσικής Υψηλών Ενεργειών, με τις τεχνικές της να βρίσκουν ολοένα και περισσότερες εφαρμογές σε ολόκληρο το φάσμα της πειραματικής διαδικασίας. Ιδιαίτερα σημαντική είναι η συμβολή τους στην ανάλυση των πειραματικών δεδομένων, όπου ένα από τα βασικά προβλήματα είναι η κατηγοριοποίηση των γεγονότων και ο διαχωρισμός του σήματος από το κυρίαρχο υπόβαθρο.

Στην παρούσα πτυχιακή εργασία εξετάζεται η εφαρμογή τέτοιων τεχνικών στον διαχωρισμό σήματος και υποβάθρου για την διμιονική διάσπαση του μποζονίου Higgs ($H \to \mu^+\mu^-$) χρησιμοποιώντας δεδομένα του πειράματος ATLAS στον επιταχυντή LHC στο CERN. Η μελέτη πραγματοποιείται με προσομοιωμένα δεδομένα για το Run II του LHC σε περιβάλλον C_{++} χρησιμοποιώντας τις βιβλιοθήκες ROOT και TMVA.

Μια ποικιλία μεθόδων μηχανικής μάθησης εκπαιδεύτηκαν, βελτιστοποιήθηκαν και συγκρίθηκαν χρησιμοποιώντας διαφορετικά κινηματικά χαρακτηριστικά σε διαφορετικές κατηγορίες γεγονότων του προβλήματος.

Abstract

Machine Learning has become an integral part of modern High Energy Physics experiments, with its techniques finding an increasing number of applications throughout the experimental process. Its contribution is particularly important in the analysis of experimental data, where one of the main challenges is event classification and the separation of signal from the dominant background.

In this bachelor's thesis, the application of such techniques to signal-background discrimination for the dimuon decay of the Higgs boson, $H \to \mu^+\mu^-$, is investigated using data from the ATLAS experiment at the LHC at CERN. The study is performed using simulated data corresponding to Run II of the LHC in a C_{++} environment using the ROOT and TMVA libraries.

A variety of machine learning methods were trained, optimized, and compared using different kinematic features across different event categories of the problem.

Περιεχόμενα

1 Εισαγωγή **4**

2 Μηχανική Μάθηση **4**

2.1 Εισαγωγή 4

2.1.1 Ορισμός 4

2.1.2 Σύγκριση με τις συμβατικές μεθόδους 5

2.2 Αναπαράσταση και δομή των δεδομένων 5

2.2.1 Σύνολα δεδομένων 5

2.2.2 Ετικέτες 6

2.3 Εκπαίδευση 6

2.3.1 Διαδικασία εκπαίδευσης 6

2.3.2 Υπερπαράμετροι 7

2.3.3 Διαχωρισμός δεδομένων 7

2.3.4 Προβλήματα κατά την εκπαίδευση 7

2.4 Κατηγορίες Μάθησης 8

2.4.1 Επιβλεπόμενη Μάθηση 8

2.4.2 Μη Επιβλεπόμενη Μάθηση 8

2.4.3 Ενισχυτική Μάθηση 9

2.5 Κατηγοριοποίηση 9

2.5.1 Αξιολόγηση ταξινομητών 9

2.5.2 Εφαρμογή στην Φυσική Υψηλών Ενεργειών 11

2.6 Τεχνικές Μηχανικής Μάθησης 12

2.6.1 Fisher Linear Discriminant 12

2.6.2 Projective Likelihood Estimator 14

2.6.3 k-Nearest Neighbors 15

2.6.4 Νευρωνικά Δίκτυα 16

2.6.5 Δέντρα Αποφάσεων 20

3 Μποζόνιο Higgs **22**

3.1 Καθιερωμένο Πρότυπο 22

3.2 Τρόποι παραγωγής του μποζονίου Higgs 24

3.3 Τρόποι διάσπασης του μποζονίου Higgs 26

3.4 Πειραματικές μετρήσεις του μποζονίου Higgs 27

3.5 Η Διμιονική Διάσπαση 28

3.6 Το πείραμα ATLAS 30

4 Δεδομένα και Μεταβλητές **32**

4.1 Ανοικτά Δεδομένα ATLAS 32

4.1.1 Δείγματα Σήματος και Υποβάθρου 32

4.1.2 Αξιοποίηση των Δεδομένων στο ROOT και TMVA 33

4.2 Κριτήρια Επιλογής γεγονότων 33
4.2.1 Επιλογή υποψηφίου $H \to \mu^+\mu^-$ 34
4.2.2 Επιλογή jets 35
4.3 Στάθμιση και κανονικοποίηση των γεγονότων 36
4.3.1 Στάθμιση των προσομοιωμένων γεγονότων 37
4.3.2 Κανονικοποίηση των δειγμάτων 38
4.4 Μεταβλητές της Ανάλυσης 38
4.4.1 Μεταβλητές Εισόδου 38
4.4.2 Μεταβλητές Παρακολούθησης 42
4.5 Κατασκευή των τελικών συνόλων δεδομένων 43

5 Κατασκευή και Εκπαίδευση των Μοντέλων 44
5.1 Επιλογή Μεθόδων και Υπερπαραμέτρων 45
5.2 Κοινή Στρατηγική Εκπαίδευσης 49
5.3 Διαδικασία Βελτιστοποίησης Υπερπαραμέτρων 50

6 Αποτελέσματα 51
6.1 Κατανομές και Συσχετίσεις των Μεταβλητών Εισόδου 51
6.2 Αποτελέσματα Βελτιστοποίησης Υπερπαραμέτρων 57
6.3 Τελική Απόδοση και Σύγκριση των Μοντέλων 62

7 Συμπεράσματα 70

Αναφορές 72

Κώδικες Υλοποίησης 75

1 Εισαγωγή

Στα πειράματα που διαδραματίζονται στον Μεγάλο Επιταχυντή Αδρονίων (LHC) πραγματοποιούνται συγκρούσεις πρωτονίων σε πολύ υψηλές ενέργειες με σκοπό την επέκταση των γνώσεων μας στο πεδίο της Φυσικής Στοιχειωδών Σωματιδίων. Η ανακάλυψη του σωματιδίου Higgs αποτέλεσε ένα από τα σημαντικότερα επιτεύγματα στην ιστορία του LHC και σημαντικό ορόσημο στη σύγχρονη Φυσική.

Η εξαγωγή φυσικής πληροφορίας από τις συγκρούσεις του LHC αποτελεί μία ιδιαίτερα δύσκολη διαδικασία, διότι σε κάθε πείραμα συλλέγεται ένας τεράστιος αριθμός πολύπλοκων και πολυδιάστατων δεδομένων. Για τον λόγο αυτό το περιβάλλον υψηλών ενεργειών του LHC συνιστά ένα από τα πιο απαιτητικά πεδία εφαρμογής σύγχρονων τεχνικών ανάλυσης δεδομένων.

Η Μηχανική Μάθηση αποτελεί μία από τις πιο σημαντικές μεθόδους, καθώς έχει την ικανότητα να διαχειρίζεται με ευκολία τέτοιου είδους γεγονότα και να επιλύει σύνθετα προβλήματα αναγνώρισης μοτίβων[1]. Ειδικότερα τα τελευταία χρόνια έχει επεκταθεί σε ολόκληρο το φάσμα της πειραματικής διαδικασίας[2], από την επιτάχυνση της προσομοίωσης (Event Simulation), την ανακατασκευή αντικειμένων (Object Reconstruction), μέχρι την ανίχνευση ανωμαλιών (Anomaly Detection) και την κατηγοριοποίηση γεγονότων (Event Classification).

Η παρούσα ανάλυση επικεντρώνεται στην κατηγοριοποίηση γεγονότων, συγκεκριμένα στον διαχωρισμό της σπάνιας διμυονικής διάσπασης του μποζονίου Higgs ($H \to \mu^+\mu^-$), από το κυρίαρχο υπόβαθρο της την αντίστοιχη διμυονική διάσπαση του μποζονίου Z ($Z \to \mu^+\mu^-$).

Για να επιτευχθεί η ανάλυση αυτή, εφαρμόζονται τεχνικές πολυμεταβλητής ανάλυσης μέσω του εργαλείου TMVA [3] (Toolkit for Multivariate Analysis) σε περιβάλλον C_{++} /ROOT[4], χρησιμοποιώντας τα ανοιχτά δεδομένα [5] από το Run II ($\sqrt{s} = 13$ TeV) του ανιχνευτή ATLAS.[6]

2 Μηχανική Μάθηση

2.1 Εισαγωγή

2.1.1 Ορισμός

Η Μηχανική Μάθηση (Machine Learning - ML) αποτελεί ένα πεδίο της Τεχνητής Νοημοσύνης[1] (Artificial Intelligence - AI) που χρησιμοποιεί στατιστικές μεθόδους και αλγορίθμους για να επιτρέπει στα υπολογιστικά συστήματα να αναγνωρίζουν μοτίβα στα δεδομένα, να κάνουν προβλέψεις, να κατηγοριοποιούν την πληροφορία και να αυτοματοποιούν τη λήψη αποφάσεων με βάση τα μοτίβα που έχουν μάθει.[7]

[1]Η τεχνολογία μέσω τη οποίας οι μηχανές αποκτούν την δυνατότητα να μιμούνται ανθρώπινες συμπεριφορές

2.1.2 Σύγκριση με τις συμβατικές μεθόδους

Ο παραδοσιακός τρόπος ανάλυσης μεγάλου αριθμού δεδομένων είναι μέσω της σχεδίασης αλγορίθμων που βασίζονται στα θεωρητικά μαθηματικά μοντέλα, τα οποία διαμορφώνονται έπειτα από εκτεταμένη μελέτη της φυσικής της διαδικασίας. Η απόδοσή τους εξαρτάται σε σημαντικό βαθμό από την ορθότητα και την καταλληλότητα του θεωρητικού μοντέλου. Η διαδικασία αυτή για μεγάλα και πολύπλοκα σύνολα δεδομένων μπορεί να καταστεί ιδιαίτερα επίπονη.

Η Μηχανική Μάθηση, αντίθετα, βασίζεται στην αξιοποίηση ενός ικανοποιητικά μεγάλου αριθμού παραδειγμάτων πάνω στα οποία ο αλγόριθμος εκπαίδευεται και εκτελεί την επιθυμητή εργασία (κατηγοριοποίηση, ανακατασκευή, κ.ο.κ). Συνεπώς, δεν απαιτεί την επιβολή κάποιου προκαθορισμένου συνόλου κανόνων (θεωρητικό μοντέλο), αντιθέτως το μοντέλο προκύπτει από την εκπαίδευση των αλγορίθμων στα διαθέσιμα δεδομένα, αναγνωρίζοντας τις σύνθετες συσχετίσεις και πρότυπα που κρύβονται σε αυτά.[1]

2.2 Αναπαράσταση και δομή των δεδομένων

2.2.1 Σύνολα δεδομένων

Τα δεδομένα εισόδου των αλγορίθμων Μηχανικής Μάθησης οργανώνονται με την μορφή ενός συνόλου δεδομένων (dataset), το οποίο μπορεί να αναπαρασταθεί ως ένας δισδιάστατος πίνακας $N \times D$. [8]

- Οι γραμμές N αντιπροσωπεύουν τα ανεξάρτητα δείγματα ή παρατηρήσεις (samples/ observations).
- Οι στήλες D αντιστοιχούν στα χαρακτηριστικά (features/input variables) που περιγράφουν το κάθε δείγμα.

Έτσι, ένα μεμονωμένο δείγμα i μπορεί να αναπαρασταθεί ως ένα διάνυσμα D διαστάσεων.

$$\mathbf{x}_i = (x_{i1}, x_{i2}, \ldots, x_{iD}), \tag{1}$$

όπου x_{ij} είναι η τιμή του j-οστού χαρακτηριστικού για το i-οστό δείγμα. Συνεπώς, το σύνολο των δεδομένων μπορεί να αναπαρασταθεί από τον πίνακα:

$$X = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1D} \\ x_{21} & x_{22} & \cdots & x_{2D} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \cdots & x_{ND} \end{pmatrix}. \tag{2}$$

Στην φυσική υψηλών ενεργειών οι γραμμές του πίνακα αποτελούνται από τα γεγονότα[2] και οι στήλες από τις φυσικές και κινηματικές μεταβλητές του κάθε γεγονότος[9]. Για παράδειγμα, ένα γεγονός μπορεί να χαρακτηρίζεται από την εγκάρσια ορμή p_T, την ψευδοταχύτητα η

[2]Ως γεγονός στα πειράματα του LHC χαπρακτηρίζεται κάθε καταγεγραμμένη σύγκρουση πρωτονίου - πρωτονίου στον ανιχνευτή.

και την αζιμουθιακή γωνία ϕ ενός σωματιδίου ή αντικειμένου του γεγονότος. Οι μεταβλητές που χρησιμοποιούνται ως είσοδοι στην παρούσα ανάλυση παρουσιάζονται αναλυτικά στο αντίστοιχο κεφάλαιο.

2.2.2 Ετικέτες

Σε προβλήματα Μηχανικής Μάθησης όπου είναι γνωστή η επιθυμητή έξοδος για κάθε δείγμα, σε αυτό αντιστοιχίζεται μία ετικέτα (label). Συνεπώς, ένα δείγμα περιγράφεται πλέον από το ζεύγος:

$$(\mathbf{x}_i, y_i), \tag{3}$$

όπου $\mathbf{x}_i$ αποτελεί το διάνυσμα των χαρακτηριστικών και $\mathbf{y}_i$ την αντίστοιχη ετικέτα.

Στην περίπτωση ενός προβλήματος δυαδικής κατηγοριοποίησης (binary classification), η ετικέτα μπορεί να λάβει δύο διαφορετικές τιμές,

$$y_i \in \{0, 1\}, \tag{4}$$

οι οποίες αντιστοιχούν στις δύο υπό εξέταση κατηγορίες. Στην παρούσα εργασία οι κατηγορίες αυτές αντιστοιχούν σε γεγονότα σήματος (signal) και υποβάθρου (background)

2.3 Εκπαίδευση

2.3.1 Διαδικασία εκπαίδευσης

Η διαδικασία εκπαίδευσης ενός μοντέλου Μηχανικής Μάθησης συνίσταται στην προσαρμογή των παραμέτρων του, έτσι ώστε οι προβλέψεις του να ταιριάζουν όσο το δυνατόν περισσότερο με τις ετικέτες. [8]

Έστω, ένα σύνολο δεδομένων εκπαίδευσης αποτελούμενο από N δείγματα,

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}. \tag{5}$$

Το μοντέλο μπορεί να αναπαρασταθεί γενικά από μία συνάρτηση,

$$\hat{y}_i = f(\mathbf{x}_i; \boldsymbol{\theta}), \tag{6}$$

όπου το $\hat{y}_i$ αντιστοιχεί στην πρόβλεψη του μοντέλου και θ στο σύνολο των παραμέτρων του.

Η απόκλιση μεταξύ επιθυμητής εξόδου και πρόβλεψης ποσοτικοποιείται μέσω μίας συνάρτησης σφάλματος ℓ (loss function).

Η σχέση μεταξύ της της ℓ και της συνολικής απώλειας L μπορεί να εκφραστεί από την σχέση,

$$L(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \ell\left(y_i, f(\mathbf{x}_i; \boldsymbol{\theta})\right). \tag{7}$$

Στόχος της εκπαίδευσης είναι η εύρεση των παραμέτρων που ελαχιστοποιούν την συνάρτηση

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} L(\boldsymbol{\theta}). \tag{8}$$

2.3.2 Υπερπαράμετροι

Πέραν των παραμέτρων που εκπαιδεύονται πάνω στα δεδομένα, η διαδικασία της εκπαίδευσης εξαρτάται και από ένα σύνολο παραμέτρων οι οποίες ορίζονται πριν την εκπαίδευση και ονομάζονται υπερπαραμέτροι (hyperparameters). Η ρύθμιση των υπερπαραμέτρων (hyperparameter tuning) είναι εξαιρετικά σημαντικό στάδιο της ανάπτυξης ενός μοντέλου, καθώς η κατάλληλη επιλογή τους είναι καθοριστική για την ικανότητα γενίκευσης των μοντέλων σε δεδομένα εκτός του δείγματος.[10]

Στην συγκεκριμένη ανάλυση εξετάστηκαν διαφορετικές τιμές ορισμένων υπερπαραμέτρων που επηρεάζουν την πολυπλοκότητα των χρησιμοποιούμενων μοντέλων, με σκοπό τη μελέτη της επίδρασής τους στην απόδοση, χωρίς υπερβολική προσαρμογή στα δεδομένα εκπαίδευσης.

2.3.3 Διαχωρισμός δεδομένων

Κατά την ανάπτυξη των μοντέλων, το σύνολο των δεδομένων συνήθως διαχωρίζεται σε τρία διακριτά υποσύνολα τα οποία εξυπηρετούν διαφορετικούς σκοπούς. [11]

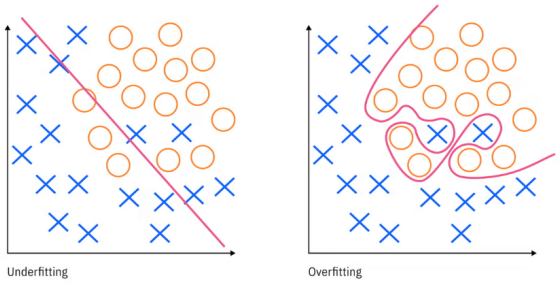
- **Σύνολο εκπαίδευσης(Training set)**: Αποτελεί το σύνολο πάνω στα οποία ο αλγόριθμος ακολουθά την διαδικασία εκπαίδευσης
- **Σύνολο επικύρωσης(Validation set)**: Το σύνολο αυτό χρησιμοποιείται κατά την εκπαίδευση για την αξιολόγηση των υπερπαραμέτρων του μοντέλου.
- **Σύνολο ελέγχου(Test set)**: Το σύνολο αυτό διατηρείται ανεξάρτητο από τη διαδικασία εκπαίδευσης και χρησιμοποιείται για τον έλεγχο της απόδοσης του μοντέλου πάνω σε άγνωστα δεδομένα.

2.3.4 Προβλήματα κατά την εκπαίδευση

Για να θεωρηθεί επιτυχημένη η εκπαίδευση ενός μοντέλου θα πρέπει η προσαρμογή των παραμέτρων του στα δεδομένα εκπαίδευσης να οδηγεί και σε ικανοποιητική απόδοση σε νέα δεδομένα. Ανάλογα με την πολυπλοκότητα του μοντέλου, η επίτευξη αυτής της ισορροπίας ανάμεσα σε προσαρμογή και γενίκευση μπορεί να οδηγήσει σε δύο βασικά προβλήματα. [11]

- **Υποεκπαίδευση (Underfitting)**: Εμφανίζεται όταν το μοντέλο είναι υπερβολικά απλό και αδυνατεί να προσαρμοστεί στα δεδομένα εκπαίδευσης, όπως απεικονίζεται στο Σχήμα 1 (αριστερά). Ως αποτέλεσμα, παρουσιάζει χαμηλή απόδοση τόσο στα δεδομένα εκπαίδευσης όσα και στα νέα δεδομένα.
- **Υπερεκπαίδευση (Overfitting)**: Εμφανίζεται όταν η πολυπλοκότητα του μοντέλου είναι υπερβολική κάτι που οδηγεί σε πλήρη προσαρμογή στα δεδομένα εκπαίδευσης και αποστήθιση των τυχαίων διακυμάνσεων τους, όπως απεικονίζεται στο Σχήμα 1

(δεξιά). Ως αποτέλεσμα, το μοντέλο έχει πολύ υψηλή απόδοση στο σύνολο εκπαίδευσης αλλά πολύ χαμηλότερη σε νέα δεδομένα.



Σχήμα 1: Σχηματική απεικόνιση των φαινομένων υποεκπαίδευσης (underfitting) και υπερεκπαίδευσης (overfitting). (Πηγή:[12])

2.4 Κατηγορίες Μάθησης

Οι κατηγορίες μάθησης διακρίνονται με κριτήριο την φύση των δεδομένων που καλούνται να διαχειριστούν κατά την εκπαίδευση.[1]

2.4.1 Επιβλεπόμενη Μάθηση

Στην επιβλεπόμενη μάθηση (supervised learning) ο αλγόριθμος εκπαιδεύεται με δεδομένα συνοδευόμενα από ετικέτα (labeled data) με σκοπό να μάθει τη σχέση μεταξύ των δεδομένων εισόδου και της αντίστοιχης επιθυμητής εξόδου. Συνεπώς, οι αλγόριθμοι γνωρίζουν το «σωστό»αποτέλεσμα για κάθε δεδομένο εισόδου και προσαρμόζουν τις παραμέτρους τους για να βελτιστοποιήσουν την πρόβλεψη τους.

Η επιβλεπόμενη μάθηση χωρίζεται σε δύο βασικές κατηγορίες:

- **Κατηγοριοποίηση (Classification):** Είναι η κατηγορία κατά την οποία ο αλγόριθμος καλείται να διαχωρίσει τα δεδομένα εισόδου του δείγματος σε διακριτές κατηγορίες (classes).
- **Παλινδρόμηση (Regression):** Στην κατηγορία αυτή, οι αλγόριθμοι καλούνται να μάθουν τη σχέση μεταξύ των μεταβλητών εισόδου και της αντίστοιχης επιθυμητής εξόδου, η οποία αποτελεί μία συνεχή αριθμητική ποσότητα.

2.4.2 Μη Επιβλεπόμενη Μάθηση

Στην μη επιβλεπόμενη μάθηση (unsupervised learning), αντίθετα, οι αλγόριθμοι εκπαιδεύονται χρησιμοποιώντας δεδομένα χωρίς ετικέτα (unlabeled data) με σκοπό την αναζήτηση συσχετίσεων και μοτίβων χωρίς να παρέχεται επιθυμητή έξοδος. Κάποιες από τις εφαρμογές της είναι η ομαδοποίηση (clustering) και η ανίχνευση ανωμαλιών (anomaly detection)

2.4.3 Ενισχυτική Μάθηση

Στην ενισχυτική μάθηση (reinforcement learning), ένας αλγόριθμος, ή agent[3], μαθαίνει να λαμβάνει διαδοχικές αποφάσεις μέσω της αλληλεπίδρασής του με ένα περιβάλλον (environment). Σε κάθε βήμα, ο agent επιλέγει μία ενέργεια (action) και λαμβάνει από το περιβάλλον μία ανταμοιβή (reward), με στόχο τη μεγιστοποίηση της συνολικής ανταμοιβής σε βάθος χρόνου. Η βέλτιστη στρατηγική μαθαίνεται μέσω επαναλαμβανόμενης αλληλεπίδρασης και δοκιμής διαφορετικών ενεργειών.

2.5 Κατηγοριοποίηση

Η κατηγοριοποίηση των δεδομένων μπορεί να γίνει σε δύο ή και περισσότερες κατηγορίες. Είδαμε στο κεφάλαιο 2.2.3 ότι στην περίπτωση της δυαδικής κατηγοριοποίησης για δείγμα $(\mathbf{x}_i, y_i)$, η έξοδος y_i δύναται να λάβει μέχρι δύο τιμές. Οι αλγόριθμοι που ακολουθούν αυτήν την διαδικασία ονομάζονται ταξινομητές (classifiers) και μπορούν γενικά να αναπαρασταθούν από μία συνάρτηση:

$$f(\mathbf{x}; \boldsymbol{\theta}) \tag{9}$$

Ανάλογα με τον αλγόριθμο, ο ταξινομητής μπορεί να παράγει μία συνεχή αριθμητική τιμή, η οποία εκφράζει τον βαθμό στον οποίο ένα δείγμα αντιστοιχεί στη μία ή την άλλη κατηγορία. Η τιμή αυτή ονομάζεται σκορ ταξινόμησης (classification score) s. Η τελική απόφαση για την κατηγορία του δείγματος λαμβάνεται με την εφαρμογή ενός κατωφλίου t (threshold) στο σκορ:

$$\hat{y} = \begin{cases} 1, & s \geq t \\ 0, & s < t \end{cases} \tag{10}$$

Οι προβλέψεις οι οποίες ξεπερνούν το ορισμένο t θεωρούνται ως θετικές (positive), ενώ διαφορετικά ως αρνητικές (negative).

2.5.1 Αξιολόγηση ταξινομητών

Έπειτα από την διαδικασία εκπαίδευσης, πρέπει ο κάθε ταξινομητής να αξιολογηθεί όχι μόνο για την ικανότητα του να κατηγοριοποιεί σωστά αλλά και για τον τρόπο με τον οποίο διαχειρίστηκε την κάθε κατηγορία. Η αξιολόγηση ποσοτικοποιείται μέσω διαφορετικών μετρικών, οι οποίες παρέχουν διαφορετικές πληροφορίες σχετικά με την απόδοση του μοντέλου. Πριν τον ορισμό των μετρικών αυτών πρέπει πρώτα να οριστεί ένας πολύ σημαντικός πίνακας στην Μηχανική Μάθηση, ο **Confusion Matrix**[13]. Τα στοιχεία του, όπως βλέπουμε στον πίνακα 1, αντιστοιχούν στις τέσσερις δυνατές περιπτώσεις που προκύπτουν από τη σύγκριση της πρόβλεψης του ταξινομητή με την πραγματική ετικέτα και μέσω αυτών ορίζονται οι μετρικές αξιολόγησης.

- **True Positive (TP):** Δείγμα της κατηγορίας 1 το οποίο ταξινομήθηκε σωστά ως 1.

[3]Ως agent χαρακτηρίζεται η οντότητα που λαμβάνει αποφάσεις και επιλέγει ενέργειες αλληλεπιδρώντας με το περιβάλλον.

- **True Negative (TN):** Δείγμα της κατηγορίας 0 το οποίο ταξινομήθηκε σωστά ως 0.
- **False Positive (FP):** Δείγμα της κατηγορίας 0 το οποίο ταξινομήθηκε λανθασμένα ως 1.
- **False Negative (FN):** Δείγμα της κατηγορίας 1 το οποίο ταξινομήθηκε λανθασμένα ως 0.

Πίνακας 1: Confusion Matrix για δυαδική κατηγοριοποίηση.

	Predicted Positive ($\hat{y} = 1$)	**Predicted Negative** ($\hat{y} = 0$)
Actual Positive ($y = 1$)	True Positive (TP)	False Negative (FN)
Actual Negative ($y = 0$)	False Positive (FP)	True Negative (TN)

Μετρικές Αξιολόγησης:

- **Accuracy**: Εκφράζει το ποσοστό σωστών προβλέψεων που απέδωσε το μοντέλο σε σχέση με τις συνολικές προβλέψεις. Είναι ένα μέτρο γενικής αξιολόγησης του μοντέλου.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \tag{11}$$

- **Precision**: Εστιάζει στην ποιότητα των positive προβλέψεων του μοντέλου, καθώς εκφράζει τον αριθμό των σωστών positive προβλέψεων προς τον συνολικό αριθμό positive προβλέψεων.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{12}$$

- **Recall**: Υπολογίζει τον αριθμό σωστών positive προβλέψεων προς τον συνολικό αριθμό positive γεγονότων. Είναι μέτρο της ικανότητας του μοντέλου να προβλέπει σωστά τα positive γεγονότα.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{13}$$

- **F1-Score**: Συνδυάζει τα precision και recall για να εξισορροπήσει την μεταξύ τους σχέση.

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}} \tag{14}$$

Οι παραπάνω μετρικές αξιολόγησης αποτυπώνουν διαφορετικές πτυχές της απόδοσης του ταξινομητή. Ωστόσο, η επιλογή των κατάλληλων μετρικών αξιολόγησης εξαρτάται άμεσα από τη φύση και τον στόχο του εκάστοτε προβλήματος. [13]

2.5.2 Εφαρμογή στην Φυσική Υψηλών Ενεργειών

Στα πειράματα του LHC σπάνιες φυσικές διεργασίες μπορούν να παράγουν πειραματικά χαρακτηριστικά παρόμοια με εκείνα των διεργασιών υποβάθρου. Χαρακτηριστικό παράδειγμα αποτελεί η αναζήτηση της διάσπασης $H \to \mu^+\mu^-$, όπου το σήμα πρέπει να διαχωριστεί από το κυρίαρχο υπόβαθρο $Z \to \mu^+\mu^-$. Στην περίπτωση αυτή είναι σημαντικό να ελέγξουμε την ικανότητα του ταξινομητή να διατηρεί όσο το δυνατόν μεγαλύτερο μέρος του σήματος, απορρίπτοντας παράλληλα αποτελεσματικά τα γεγονότα υποβάθρου. Για τον λόγο αυτό πρέπει να οριστούν και κάποια επιπλέον εργαλεία αξιολόγησης που θα χρησιμοποιηθούν εκτενώς στην παρούσα ανάλυση.

- **Signal efficiency:** Η αποδοτικότητα του σήματος εκφράζει το ποσοστό των πραγματικών γεγονότων σήματος που αναγνωρίζονται σωστά από τον ταξινομητή ως σήμα. Ορίζεται ως
$$\epsilon_S = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}. \tag{15}$$
- **Background efficiency:** Η αποδοτικότητα του υποβάθρου εκφράζει το ποσοστό των πραγματικών γεγονότων υποβάθρου που ταξινομούνται λανθασμένα ως σήμα. Ορίζεται ως
$$\epsilon_B = \frac{\mathrm{FP}}{\mathrm{FP} + \mathrm{TN}}. \tag{16}$$
- **Background rejection:** Η απόρριψη υποβάθρου εκφράζει το ποσοστό των πραγματικών γεγονότων υποβάθρου που απορρίπτονται σωστά από τον ταξινομητή. Ορίζεται ως
$$r_B = 1 - \epsilon_B. \tag{17}$$

Η αποδοτικότητα του σήματος και η αποδοτικότητα του υποβάθρου εξαρτώνται από την επιλογή του κατωφλίου t που εφαρμόζεται στο σκορ ταξινόμησης. Η αύξηση του κατωφλίου οδηγεί γενικά σε αυστηρότερα κριτήρια για την αποδοχή ενός γεγονότος ως σήματος, μεταβάλλοντας ταυτόχρονα την αποδοτικότητα του σήματος και του υποβάθρου. Η σχέση αυτή μπορεί να μελετηθεί μέσω της καμπύλης χαρακτηριστικών λειτουργίας δέκτη (Receiver Operating Characteristic, ROC).

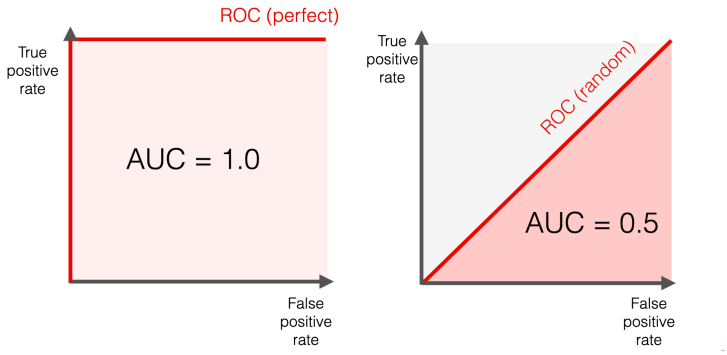
ROC curve

Η καμπύλη ROC [14] αποτελεί μία οπτική αναπαράσταση της απόδοσης του μοντέλου για όλες τις επιλογές κατωφλιού t (threshold). Είναι ένα δισδιάστατο διάγραμμα και εν γένει απεικονίζει την αποδοτικότητα του σήματος ϵ_S, ή αλλιώς True Positive Rate (TPR), συναρτήσει της αποδοτικότητας του υποβάθρου ϵ_B, η οποία αντιστοιχεί στο False Positive Rate (FPR). Στην παρούσα ανάλυση, σύμφωνα με τη σύμβαση που χρησιμοποιείται από το TMVA[3], η καμπύλη παρουσιάζεται ως η αποδοτικότητα του σήματος ϵ_S συναρτήσει της απόρριψης του υποβάθρου $r_B = 1 - \epsilon_B$.

AUC

Το εμβαδό κάτω από την καμπύλη ROC ονομάζεται AUC (Area Under the Curve) και αποτελεί ένα συνολικό μέτρο της διακριτικής ικανότητας του ταξινομητή. Όπως απεικονίζεται

στο Σχήμα 2, ένας ιδανικός ταξινομητής που διαχωρίζει τέλεια το σήμα από το υπόβαθρο έχει $AUC = 1.0$, ενώ ένας ταξινομητής που πραγματοποιεί τυχαίες προβλέψεις αντιστοιχεί σε $AUC = 0.5$.



Σχήμα 2: Σχηματική απεικόνιση των καμπυλών ROC . (Πηγή:[15])

Στη φυσική υψηλών ενεργειών και ειδικότερα στην ανάλυσή μας για το $H \to \mu^+\mu^-$, η μεγιστοποίηση του AUC αποτελεί το υπέρτατο κριτήριο βελτιστοποίησης. Μας δείχνει πόσο καλά το εργαλείο μας καταφέρνει να διατηρεί το σήμα του Higgs ενώ συνθλιβεί το τεράστιο υπόβαθρο του Z.

2.6 Τεχνικές Μηχανικής Μάθησης

Στην παρούσα εργασία χρησιμοποιήθηκαν διαφορετικές τεχνικές επιβλεπόμενης μάθησης για την κατηγοριοποίηση των γεγονότων. Οι μέθοδοι αυτές βασίζονται σε διαφορετικές προσεγγίσεις για την αναζήτηση και την εκμετάλλευση των διακριτικών χαρακτηριστικών των δεδομένων και διαφέρουν ως προς τον τρόπο με τον οποίο κατασκευάζουν τον ταξινομητή.

Οι χρησιμοποιούμενες τεχνικές μπορούν να ομαδοποιηθούν με βάση την βασική αρχή λειτουργίας τους σε γραμμικές και στατιστικές μεθόδους, μεθόδους βασισμένες σε γειτονικά δείγματα, νευρωνικά δίκτυα και μεθόδους βασισμένες σε δέντρα αποφάσεων. Στη συνέχεια παρουσιάζονται οι βασικές αρχές λειτουργίας των μεθόδων που χρησιμοποιήθηκαν, καθώς και τα σημαντικότερα χαρακτηριστικά τους.

2.6.1 Fisher Linear Discriminant

Η μέθοδος Fisher Linear Discriminant (FLD)[16], γνωστή και ως Linear Discriminant Analysis (LDA), είναι μια κλασική γραμμική μέθοδος μείωσης διαστασιμότητας.

Βασικός της στόχος είναι η εύρεση ενός βέλτιστου γραμμικού συνδυασμού των μεταβλητών εισόδου, ώστε τα δεδομένα να προβάλλονται από τον αρχικό χώρο υψηλών διαστάσεων σε έναν υποχώρο μίας διάστασης (ή χαμηλότερης διάστασης), διασφαλίζοντας παράλληλα τον μέγιστο δυνατό διαχωρισμό μεταξύ των διαφορετικών κατηγοριών (π.χ. σήματος και υποβάθρου).

Για να επιτύχει αυτόν τον βέλτιστο διαχωρισμό, η μέθοδος βασίζεται ταυτόχρονα σε δύο γεωμετρικά κριτήρια:

- Την απομάκρυνση των κέντρων (μέσων τιμών) των διαφορετικών κατηγοριών όσο το δυνατόν περισσότερο μεταξύ τους (between-class variance).
- Την ελαχιστοποίηση της εσωτερικής διασποράς των δεδομένων εντός της ίδιας κατηγορίας, ώστε να είναι συμπαγή και να μην αλληλοεπικάλυπτονται (within-class variance).

Για την επίτευξη του βέλτιστου διαχωρισμού, η μέθοδος FLD επιδιώκει να βρει ένα διάνυσμα βαρών (ή συντελεστών) $\mathbf{w}$ τέτοιο ώστε, προβάλλοντας τα πολυδιάστατα δεδομένα $\mathbf{x}$ σε αυτό, να μεγιστοποιείται ο λεγόμενος λόγος του Fisher (Fisher's Criterion).

Ας θεωρήσουμε τα διανύσματα μέσων τιμών των δύο κατηγοριών στον αρχικό χώρο ως m_1 και m_2. Ορίζουμε δύο βασικούς πίνακες που περιγράφουν τη διασπορά των δεδομένων:

- Τον **πίνακα διασποράς εντός τάξης** (within-class scatter matrix, $\mathbf{S}_W$), ο οποίος εκφράζει το πόσο απλωμένα είναι τα δεδομένα γύρω από το κέντρο της κατηγορίας τους:

$$\mathbf{S}_W = \mathbf{S}_1 + \mathbf{S}_2 \tag{18}$$

  όπου $\mathbf{S}_1, \mathbf{S}_2$ είναι οι πίνακες συνδιακύμανσης των δύο κατηγοριών.

- Τον **πίνακα διασποράς μεταξύ τάξεων** (between-class scatter matrix, $\mathbf{S}_B$), ο οποίος εκφράζει την απόσταση μεταξύ των κέντρων των κατηγοριών:

$$\mathbf{S}_B = (\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^T \tag{19}$$

Ο λόγος του Fisher $J(\mathbf{w})$ που θέλουμε να μεγιστοποιήσουμε δίνεται από τη σχέση:

$$J(\mathbf{w}) = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}} \tag{20}$$

Παραγωγίζοντας τη συνάρτηση ως προς $\mathbf{w}$ και εξισώνοντας με το μηδέν, η βέλτιστη κατεύθυνση προβολής προκύπτει από τη λύση ενός προβλήματος ιδιοτιμών, δίνοντας τη βασική σχέση:

$$\mathbf{w} \propto \mathbf{S}_W^{-1}(\mathbf{m}_1 - \mathbf{m}_2) \tag{21}$$

Με τον τρόπο αυτό, ένα γεγονός που αρχικά περιγράφεται από το πολυδιάστατο διάνυσμα μεταβλητών εισόδου $\mathbf{x}$ προβάλλεται σε μία μονοδιάστατη μεταβλητή

$$y_{\mathrm{Fi}} = \mathbf{w}^T \mathbf{x} + F_0, \tag{22}$$

όπου F_0 είναι ένας σταθερός όρος μετατόπισης.

Στην υλοποίηση της μεθόδου στο TMVA[3], ο διαχωριστής Fisher εκφράζεται αντίστοιχα ως γραμμικός συνδυασμός των μεταβλητών εισόδου,

$$y_{\mathrm{Fi}}(i) = F_0 + \sum_{k=1}^{n_{\mathrm{var}}} F_k x_k(i), \tag{23}$$

όπου $x_k(i)$ είναι η τιμή της k-οστής μεταβλητής εισόδου για το γεγονός i, ενώ οι συντελεστές F_k προσδιορίζονται κατά την εκπαίδευση από τις μέσες τιμές και τη διασπορά των

μεταβλητών των δύο κατηγοριών. Η τιμή y_{Fi} αποτελεί την έξοδο του ταξινομητή, με διαφορετικές περιοχές της κατανομής της να αντιστοιχούν κατά προτίμηση σε γεγονότα σήματος και υποβάθρου.

2.6.2 Projective Likelihood Estimator

Η μέθοδος της Projective Likelihood Estimator[3] βασίζεται στις διαφορετικές συναρτήσεις πυκνότητας πιθανότητας (Probability Density Functions, PDFs) που παρουσιάζουν οι μεταβλητές εισόδου για τις διαφορετικές κατηγορίες. Συγκεκριμένα για την περίπτωση της δυαδικής κατηγοριοποίησης, η μέθοδος εκτιμά για κάθε γεγονός πόσο συμβατές είναι οι τιμές των μεταβλητών του με τις αντίστοιχες κατανομές του σήματος και του υποβάθρου.

Για κάθε μεταβλητή εισόδου x_k κατασκευάζονται, από τα δεδομένα εκπαίδευσης, οι αντίστοιχες συναρτήσεις πυκνότητας πιθανότητας για το σήμα και το υπόβαθρο,

$$p_{S,k}(x_k) \qquad \text{και} \qquad p_{B,k}(x_k). \tag{24}$$

Οι συναρτήσεις αυτές περιγράφουν την κατανομή των τιμών της μεταβλητής x_k σε γεγονότα σήματος ή υποβάθρου αντίστοιχα. Εφόσον ένα γεγονός χαρακτηρίζεται από ένα σύνολο μεταβλητών $\mathbf{x} = (x_1, x_2, \ldots, x_{n_{\mathrm{var}}})$, μπορεί να κατασκευαστεί μία συνολική ποσότητα πιθανότητας για κάθε κατηγορία.

Στην απλούστερη μορφή της μεθόδου θεωρείται ότι οι μεταβλητές εισόδου είναι στατιστικά ανεξάρτητες, για αυτό και η μέθοδος χαρακτηρίζεται ως αφελής εκτιμητής Bayes[4] (naive Bayes estimator). Η υπόθεση αυτή απλοποιεί σημαντικά τον υπολογισμό της συνολικής πιθανότητας, καθώς αποφεύγεται η εκτίμηση μιας πολυδιάστατης συνάρτησης πυκνότητας πιθανότητας. Ωστόσο, όταν υπάρχουν ισχυρές συσχετίσεις μεταξύ των μεταβλητών εισόδου, η υπόθεση αυτή μπορεί να περιορίσει τη διακριτική ικανότητα της μεθόδου.

Η συνολική συνάρτηση πιθανότητας για το σήμα και το υπόβαθρο γράφεται ως το γινόμενο των επιμέρους πιθανοτήτων,

$$\mathcal{L}_S(\mathbf{x}) = \prod_{k=1}^{n_{\mathrm{var}}} p_{S,k}(x_k), \tag{25}$$

και

$$\mathcal{L}_B(\mathbf{x}) = \prod_{k=1}^{n_{\mathrm{var}}} p_{B,k}(x_k). \tag{26}$$

Με βάση τις δύο αυτές ποσότητες μπορεί να οριστεί η έξοδος του ταξινομητή (classifier output), μέσω ενός διαχωριστή (discriminant) της μορφής

$$y_{\mathrm{L}} = \frac{\mathcal{L}_S}{\mathcal{L}_S + \mathcal{L}_B}, \tag{27}$$

ο οποίος λαμβάνει τιμές μεταξύ 0 και 1.

[4]Το θεώρημα Bayes είναι ένας μαθηματικός τύπος ($P(A|B) = \frac{P(B|A)\,P(A)}{P(B)}$) που χρησιμοποιείται για τον προσδιορισμό της υπό όρους πιθανότητας ενός συμβάντος με βάση προηγούμενη γνώση και νέα στοιχεία.

Τιμές κοντά στο 1 υποδεικνύουν μεγαλύτερη συμβατότητα του γεγονότος με την κατανομή του σήματος, ενώ τιμές κοντά στο 0 υποδεικνύουν μεγαλύτερη συμβατότητα με το υπόβαθρο.

2.6.3 k-Nearest Neighbors

Η μέθοδος των k-πλησιέστερων γειτόνων (k-NN) [17] είναι μία μέθοδος που ταξινομεί ένα γεγονός με βάση την ομοιότητα των γειτονικών δεδομένων του συνόλου εκπαίδευσης. Σε αντίθεση με μεθόδους στις οποίες κατασκευάζεται ένα παραμετρικό μοντέλο κατά την εκπαίδευση, η k-NN διατηρεί τα δείγματα εκπαίδευσης και πραγματοποιεί την κατηγοριοποίηση με βάση τη θέση τους στον χώρο των μεταβλητών εισόδου.

Κάθε γεγονός αναπαρίσταται από ένα διάνυσμα μεταβλητών

$$\mathbf{x} = (x_1, x_2, \ldots, x_{n_{\text{var}}}), \tag{28}$$

και επομένως αντιστοιχεί σε ένα σημείο στον n_{var}-διάστατο χώρο των χαρακτηριστικών (feature space). Για την κατηγοριοποίηση ενός νέου γεγονότος $\mathbf{x}$ υπολογίζεται η απόστασή του από τα γεγονότα του δείγματος εκπαίδευσης και επιλέγονται τα k πλησιέστερα.

Στο TMVA[3] για τον υπολογισμό της απόστασης μεταξύ δύο γεγονότων $\mathbf{x}$ και $\mathbf{x}_i$ χρησιμοποείται η Ευκλείδεια απόσταση,

$$R = \sqrt{\sum_{j=1}^{n_{\text{var}}} (x_j - x_{ij})^2}. \tag{29}$$

Η κατηγορία του νέου γεγονότος προσδιορίζεται από τις κατηγορίες στις οποίες ανήκουν οι k πλησιέστεροι γείτονές του. Στην απλούστερη περίπτωση, η τελική απόφαση προκύπτει μέσω πλειοψηφίας (majority voting). Για παράδειγμα, εάν η πλειονότητα των γειτόνων ανήκει στο σήμα, το νέο γεγονός ταξινομείται ως γεγονός σήματος.

Στην περίπτωση δυαδικής κατηγοριοποίησης μπορεί επίσης να οριστεί ένα συνεχές σκορ ταξινόμησης με βάση το ποσοστό των πλησιέστερων γειτόνων που ανήκουν στο σήμα,

$$y_{\text{kNN}} = \frac{k_S}{k}, \tag{30}$$

όπου k_S είναι ο αριθμός των γεγονότων σήματος μεταξύ των k πλησιέστερων γειτόνων. Κατά συνέπεια, τιμές του y_{kNN} κοντά στο 1 υποδεικνύουν μεγαλύτερη συμβατότητα με το σήμα, ενώ τιμές κοντά στο 0 μεγαλύτερη συμβατότητα με το υπόβαθρο.

Η επιλογή του αριθμού k αποτελεί σημαντική υπερπαράμετρο της μεθόδου. Μικρές τιμές του k οδηγούν σε έναν περισσότερο ευαίσθητο ταξινομητή, ο οποίος μπορεί να επηρεάζεται έντονα από στατιστικές διακυμάνσεις και μεμονωμένα γεγονότα του δείγματος εκπαίδευσης. Αντίθετα, πολύ μεγάλες τιμές του k εξομαλύνουν περισσότερο την απόφαση και μπορεί να περιορίσουν την ικανότητα του ταξινομητή να περιγράψει τοπικές δομές των δεδομένων. Η κατάλληλη επιλογή του k επιδιώκει επομένως μία ισορροπία μεταξύ αυτών των δύο συμπεριφορών.

Επιπλέον, επειδή η μέθοδος βασίζεται άμεσα στις αποστάσεις μεταξύ των σημείων, η κλίμακα των μεταβλητών εισόδου μπορεί να επηρεάσει σημαντικά το αποτέλεσμα. Μεταβλητές με μεγαλύτερο αριθμητικό εύρος μπορούν να κυριαρχήσουν στον υπολογισμό της απόστασης, γεγονός που καθιστά συχνά χρήσιμη την κατάλληλη κανονικοποίηση ή μετατροπή των μεταβλητών πριν από την εφαρμογή της μεθόδου.

2.6.4 Νευρωνικά Δίκτυα

Τα νευρωνικά δίκτυα (Neural Networks)[18] είναι μία τεχνική ιστορικά εμπνευσμένη από τη διασύνδεση βιολογικών νευρώνων. Απαρτίζονται από διασυνδεδεμένους νευρώνες τοποθετημένους σε διαδοχικά επίπεδα τα οποία αντιστοιχούν σε διαφορετικές λειτουργίες κατά την μάθηση.

- **Επίπεδο Εισόδου**(Input Layer): Στο επίπεδο αυτό εισάγονται τα δεδομένα με την μορφή μεταβλητών και στην συνέχεια διοχετεύονται στα κρυφά επίπεδα
- **Κρυφά Επίπεδα**(Hidden Layers): Ένα δίκτυο μπορεί να αποτελείται από ένα ή και περισσότερα κρυφά επίπεδα. Οι νευρώνες τους είναι επιφορτισμένοι με το περισσότερο υπολογιστικό φορτίο καθώς καλούνται να αναπαραστήσουν τις σύνθετες και μη γραμμικές σχέσεις μεταξύ των μεταβλητών εισόδου. Το συνολικό συμπέρασμα των κρυφών επιπέδων μεταφέρεται στο επίπεδο εξόυδου.
- **Επίπεδο Εξόδου**(Output Layer): Αποτελεί το τελευταίο στρώμα από το οποίο εξαγάγονται τα αποτελέσματα ανάλογα με τον σκοπό του δικτύου. Στην περίπτωση της δυαδικής κατηγοριοποίησης η ορθότητα της πρόβλεψης έγκειται στην σύγκριση της ετικέτας του γεγονότος και της εκτίμησης του δικτύου.

Σε ένα πλήρως συνδεδεμένο επίπεδο, κάθε νευρώνας συνδέεται με όλους τους νευρώνες του προηγούμενου επιπέδου και με όλους του νευρώνες του επόμενου επιπέδου. Ο κάθε νευρώνας λαμβάνει ως είσοδο τις τιμές των νευρώνων του προηγούμενου επιπέδου $x_1, x_2, \ldots, x_n$, και επιστρέφει μια τιμή εξόδου. Μεταξύ δύο νευρώνων δημιουργείται μια μεταβλητή, το βάρος w_i. Το βάρος μας δείχνει την ισχύ της σχέσης αυτών των δύο νευρώνων. Η τιμή που θα επιστρέψει ο νευρώνας υπολογίζεται από την γραμμική σχέση:

$$z = \sum_{i=1}^{n} w_i x_i + b, \tag{31}$$

όπου b είναι ο όρος πόλωσης (bias).

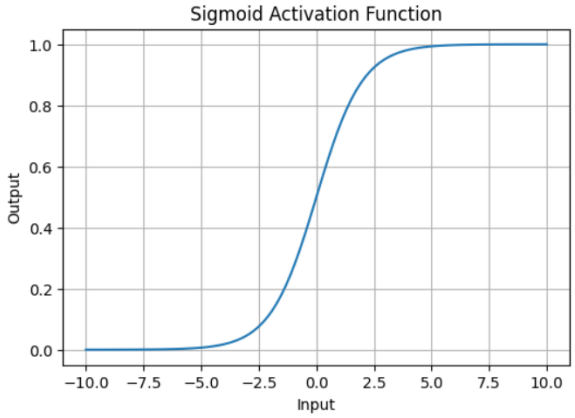
Η έξοδος του νευρώνα προκύπτει στη συνέχεια μέσω μιας συνάρτησης ενεργοποίησης (activation function) g,

$$a = g(z). \tag{32}$$

Η συνάρτηση ενεργοποίησης [19] εφαρμόζει έναν, συνήθως μη γραμμικό, μετασχηματισμό στην έξοδο του νευρώνα. Χωρίς αυτήν, η διαδοχική σύνθεση πολλών επιπέδων θα

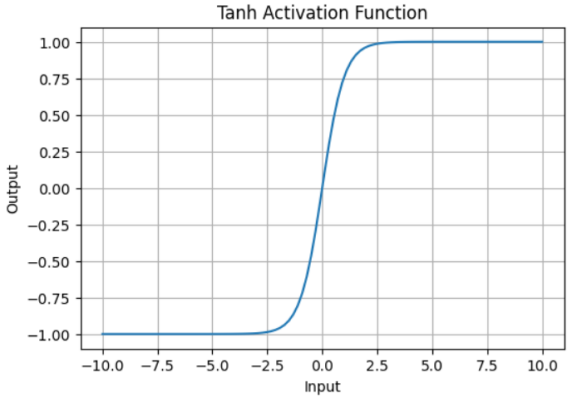
ισοδυναμούσε τελικά με έναν γραμμικό μετασχηματισμό. Κάποιες από τις πιο ευρέως χρησιμοποιούμενες είναι οι εξής:

- $\sigma(z) = \dfrac{1}{1+e^{-z}}$: Η συνάρτηση Sigmoid χαρακτηρίζεται από το σχήμα τύπου S που παρουσιάζεται στο Σχήμα 3 και λαμβάνει τιμές στο διάστημα $(0, 1)$.



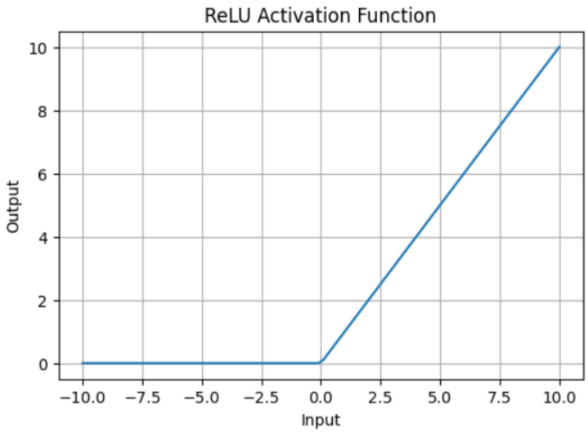
Σχήμα 3: Γραφική παράσταση της συνάρτησης ενεργοποίησης Sigmoid. (Πηγή:[20])

- $\tanh(z) = \dfrac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$: Η υπερβολική εφαπτομένη παρουσιάζει επίσης χαρακτηριστική Sigmoid μορφή, όπως φαίνεται στο Σχήμα 4, είναι όμως συμμετρική ως προς την αρχή των αξόνων και λαμβάνει τιμές στο διάστημα $(-1, 1)$.



Σχήμα 4: Γραφική παράσταση της συνάρτησης ενεργοποίησης tanh. (Πηγή:[20])

- $\mathrm{ReLU}(z) = \max(0, z)$: Η συνάρτηση ReLU, όπως απεικονίζεται στο Σχήμα 5, επιστρέφει μηδενική έξοδο για $z < 0$, ενώ για $z > 0$ η έξοδός της είναι ίση με z.



Σχήμα 5: Γραφική παράσταση της συνάρτησης ενεργοποίησης ReLU. (Πηγή:[20])

Η παραπάνω διαδικασία μπορεί να γενικευτεί για όλους τους νευρώνες ενός επιπέδου. Εάν με $\mathbf{a}^{(l-1)}$ συμβολιστεί το διάνυσμα εξόδου του προηγούμενου επιπέδου, τότε η έξοδος του επιπέδου l μπορεί να γραφεί σε διανυσματική μορφή ως

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \tag{33}$$

και

$$\mathbf{a}^{(l)} = g^{(l)}\left(\mathbf{z}^{(l)}\right), \tag{34}$$

όπου $\mathbf{W}^{(l)}$ είναι ο πίνακας των βαρών, $\mathbf{b}^{(l)}$ το διάνυσμα των όρων πόλωσης και $g^{(l)}$ η συνάρτηση ενεργοποίησης του επιπέδου l.

Η διαδικασία αυτή επαναλαμβάνεται διαδοχικά από το επίπεδο εισόδου έως το επίπεδο εξόδου και αποτελεί την προώθηση προς τα εμπρός (**forward propagation**). Με τον τρόπο αυτό, το αρχικό διάνυσμα χαρακτηριστικών $\mathbf{x}$ μετασχηματίζεται διαδοχικά μέχρι την παραγωγή της τελικής πρόβλεψης του δικτύου.

Κατά την έναρξη της εκπαίδευσης, τα βάρη και οι όροι πόλωσης του δικτύου λαμβάνουν αρχικές τιμές και στη συνέχεια προσαρμόζονται επαναληπτικά, με στόχο την ελαχιστοποίηση της συνάρτησης απώλειας (**loss function**).

Οι παράμετροι που εκπαιδεύονται στο νευρωνικό δίκτυο είναι επομένως τα βάρη και οι όροι πόλωσης,

$$\boldsymbol{\theta} = \left\{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\right\}. \tag{35}$$

Για την προσαρμογή των παραμέτρων χρησιμοποιείται η διαδικασία της οπισθοδιάδοσης του σφάλματος (**backpropagation**). Κατά τη διαδικασία αυτή υπολογίζεται η επίδραση κάθε βάρους και όρου πόλωσης στη συνάρτηση απώλειας, μεταδίδοντας την πληροφορία του σφάλματος από το επίπεδο εξόδου προς τα προηγούμενα επίπεδα του δικτύου.

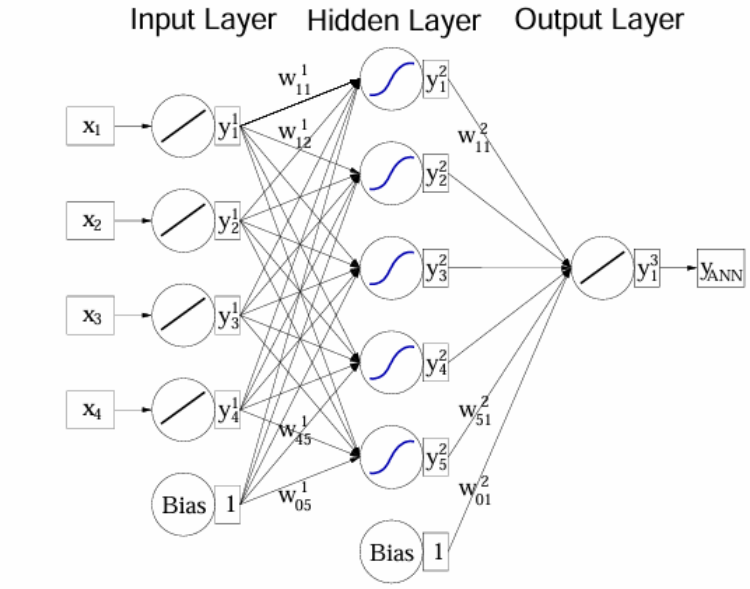
Με τη χρήση του κανόνα της αλυσίδας υπολογίζονται οι παράγωγοι της συνάρτησης απώλειας ως προς τις παραμέτρους του δικτύου. Οι παράμετροι μπορούν στη συνέχεια να

ενημερωθούν προς την κατεύθυνση που μειώνει τη συνάρτηση απώλειας. Στην περίπτωση της καθόδου κλίσης (**gradient descent**),η ενημέρωση μιας παραμέτρου θ_j μπορεί να γραφεί ως

$$\theta_j \to \theta_j - \eta \frac{\partial L}{\partial \theta_j}, \tag{36}$$

όπου η είναι ο ρυθμός μάθησης (learning rate). [11]

Η συνολική δομή και λειτουργία ενός τέτοιου νευρωνικού δικτύου με ένα κρυφό επίπεδο απεικονίζεται στο Σχήμα 6.



Σχήμα 6: Απεικόνιση νευρωνικού δικτύου. (Πηγή:[3], Fig. 18)

Τα δύο είδη νευρωνικών δικτύων που χρησιμοποιήθηκαν στην παρούσα ανάλυση είναι το Multi-layer Perceptron και το Deep Neural Network

Multi-layer Perceptron (MLP) [18]

Το MLP αποτελεί μία από τις βασικότερες αρχιτεκτονικές τεχνητών νευρωνικών δικτύων. Πρόκειται για ένα πλήρως συνδεδεμένο δίκτυο προώθησης προς τα εμπρός (fully connected feed-forward neural network), στο οποίο η πληροφορία μεταδίδεται διαδοχικά από το επίπεδο εισόδου, μέσω ενός ή περισσότερων κρυφών επιπέδων, προς το επίπεδο εξόδου, χωρίς την ύπαρξη αναδρομικών συνδέσεων.

Η πολυπλοκότητα ενός MLP καθορίζεται σε σημαντικό βαθμό από την αρχιτεκτονική του, και ιδιαίτερα από τον αριθμό των κρυφών επιπέδων και τον αριθμό των νευρώνων που περιέχονται σε αυτά. Η αύξηση του αριθμού των νευρώνων αυξάνει την ικανότητα του δικτύου να περιγράψει σύνθετες σχέσεις στα δεδομένα, αυξάνοντας όμως ταυτόχρονα τον αριθμό των εκπαιδεύσιμων παραμέτρων και, κατά συνέπεια, την πολυπλοκότητα του μοντέλου.

Deep Neural Network (DNN) [11]

Τα βαθιά νευρωνικά δίκτυα αποτελούν επέκταση της βασικής αρχιτεκτονικής των τεχνη-

τών νευρωνικών δικτύων και χαρακτηρίζονται από την παρουσία πολλαπλών κρυφών επιπέδων μεταξύ του επιπέδου εισόδου και του επιπέδου εξόδου. Ο όρος «βαθύ» (deep) αναφέρεται επομένως στο αυξημένο βάθος του δικτύου, δηλαδή στον αριθμό των διαδοχικών επιπέδων μετασχηματισμού των δεδομένων.

Σε κάθε κρυφό επίπεδο πραγματοποιείται ένας νέος μετασχηματισμός της εξόδου του προηγούμενου επιπέδου,

$$\mathbf{a}^{(l)} = g^{(l)}\left(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}\right). \tag{37}$$

Η διαδοχική εφαρμογή πολλών τέτοιων μετασχηματισμών επιτρέπει στο δίκτυο να κατασκευάζει σταδιακά περισσότερο σύνθετες αναπαραστάσεις των αρχικών μεταβλητών εισόδου. Τα πρώτα επίπεδα πραγματοποιούν μετασχηματισμούς απευθείας πάνω στις αρχικές μεταβλητές, ενώ τα επόμενα επίπεδα επεξεργάζονται τις αναπαραστάσεις που έχουν κατασκευαστεί από τα προηγούμενα.

Η αύξηση του βάθους και του αριθμού των νευρώνων παρέχει στα DNNs αυξημένη δυνατότητα αναπαράστασης σύνθετων μη γραμμικών σχέσεων. Παράλληλα, όμως, αυξάνει τον αριθμό των εκπαιδεύσιμων παραμέτρων και το υπολογιστικό κόστος της εκπαίδευσης, ενώ καθιστά ιδιαίτερα σημαντική την κατάλληλη επιλογή της αρχιτεκτονικής και των υπερπαραμέτρων του δικτύου για την επίτευξη καλής γενίκευσης.

2.6.5 Δέντρα Αποφάσεων

Τα δέντρα αποφάσεων (Decision Trees)[21] αποτελούν τεχνική της Μηχανικής Μάθησης κατά την οποία η κατηγοριοποίηση ενός γεγονότος πραγματοποιείται μέσω μιας διαδοχικής σειράς αποφάσεων πάνω στις μεταβλητές εισόδου. Η δομή τους μπορεί να αναπαρασταθεί ως ένα δέντρο αποτελούμενο από κόμβους (nodes) και κλάδους (branches), μέσω των οποίων ο χώρος των μεταβλητών εισόδου διαχωρίζεται σταδιακά σε διαφορετικές περιοχές.

Η διαδικασία ξεκινά από τον αρχικό κόμβο (root node), ο οποίος περιλαμβάνει το σύνολο των γεγονότων εκπαίδευσης. Σε κάθε εσωτερικό κόμβο εφαρμόζεται μία συνθήκη πάνω σε μία μεταβλητή εισόδου, για παράδειγμα

$$x_j < t, \tag{38}$$

όπου x_j είναι μία μεταβλητή εισόδου και t μία τιμή κατωφλίου (cut value). Ανάλογα με το αποτέλεσμα της συνθήκης, τα γεγονότα κατευθύνονται σε έναν από τους δύο θυγατρικούς κόμβους (child nodes). Η διαδικασία επαναλαμβάνεται αναδρομικά μέχρι την κατάληξη στους τελικούς κόμβους (leaf nodes), στους οποίους πραγματοποιείται η τελική ταξινόμηση.

Για την επιλογή της μεταβλητής και της τιμής κατωφλίου που θα χρησιμοποιηθούν σε κάθε διαχωρισμό, είναι απαραίτητο να εκτιμηθεί η σύνθεση των γεγονότων που περιέχονται σε έναν κόμβο. Η καθαρότητα (purity) ενός κόμβου μπορεί να οριστεί ως

$$P = \frac{N_S}{N_S + N_B}, \tag{39}$$

όπου N_S και N_B είναι ο αριθμός των γεγονότων σήματος και υποβάθρου στον συγκεκριμένο κόμβο αντίστοιχα. Ένας κόμβος με $P \simeq 1$ αποτελείται κυρίως από γεγονότα σήματος, ενώ για $P \simeq 0$ κυριαρχούν τα γεγονότα υποβάθρου. Ένα ευρέως χρησιμοποιούμενο μέτρο για την αξιολόγηση της ποιότητας ενός διαχωρισμού είναι ο δείκτης Gini (Gini Index).

Για ένα πρόβλημα δυαδικής κατηγοριοποίησης μπορεί να εκφραστεί συναρτήσει της καθαρότητας ως

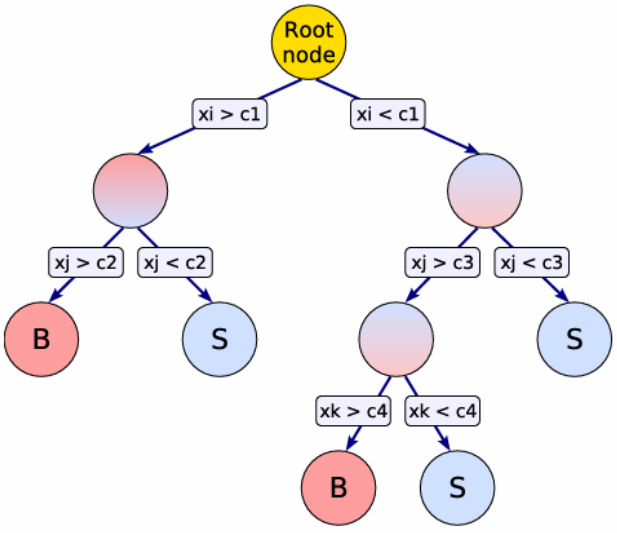
$$G(P) = P(1 - P). \tag{40}$$

Η ποσότητα αυτή λαμβάνει τη μέγιστη τιμή όταν οι δύο κλάσεις εμφανίζονται σε ίσες αναλογίες και μειώνεται καθώς ο κόμβος γίνεται περισσότερο καθαρός. Κατά την κατασκευή του δέντρου εξετάζονται διαφορετικές μεταβλητές και τιμές κατωφλίου και επιλέγεται ο διαχωρισμός που επιτυγχάνει τη μεγαλύτερη βελτίωση ως προς το επιλεγμένο κριτήριο διαχωρισμού.

Η παραπάνω διαδικασία επαναλαμβάνεται διαδοχικά, αυξάνοντας το βάθος (depth) του δέντρου. Το μέγιστο επιτρεπόμενο βάθος (MaxDepth) αποτελεί σημαντική υπερπαράμετρο, καθώς ελέγχει άμεσα την πολυπλοκότητα του μοντέλου. Ένα δέντρο μικρού βάθους πραγματοποιεί σχετικά απλούς διαχωρισμούς του χώρου των μεταβλητών, ενώ ένα βαθύτερο δέντρο μπορεί να κατασκευάσει περισσότερο σύνθετα όρια απόφασης.

Η υπερβολική αύξηση του βάθους μπορεί, ωστόσο, να οδηγήσει το δέντρο στην προσαρμογή σε στατιστικές διακυμάνσεις του δείγματος εκπαίδευσης, περιορίζοντας την ικανότητα γενίκευσής του. Αντίθετα, ένα υπερβολικά ρηχό δέντρο ενδέχεται να μην έχει επαρκή πολυπλοκότητα για την περιγραφή της δομής των δεδομένων.

Η δομή ενός δέντρου απόφασης για δυαδική κατηγοριοποίηση απεικονίζεται στο Σχήμα 7. Στο συγκεκριμένο σχήμα, η τιμή κατωφλίου κάθε διαχωρισμού συμβολίζεται με c_1.



Σχήμα 7: Απεικόνιση ενός δέντρου αποφάσεων. (Πηγή:[3], fig.22)

Παρότι ένα μεμονωμένο δέντρο απόφασης μπορεί να χρησιμοποιηθεί απευθείας ως ταξινομητής, η απόδοσή του μπορεί να είναι ευαίσθητη στις ιδιαιτερότητες του δείγματος εκπαίδευσης. Για τον λόγο αυτό, χρησιμοποιούνται συχνά σύνολα (ensembles) πολλών δέντρων απόφασης.

Στις μεθόδους ενίσχυσης (boosting), τα δέντρα κατασκευάζονται διαδοχικά, με κάθε νέο δέντρο να επικεντρώνεται σε πτυχές της ταξινόμησης που δεν περιγράφηκαν επαρκώς από τα προηγούμενα. Η τελική απόφαση προκύπτει από τον συνδυασμό των επιμέρους δέντρων.

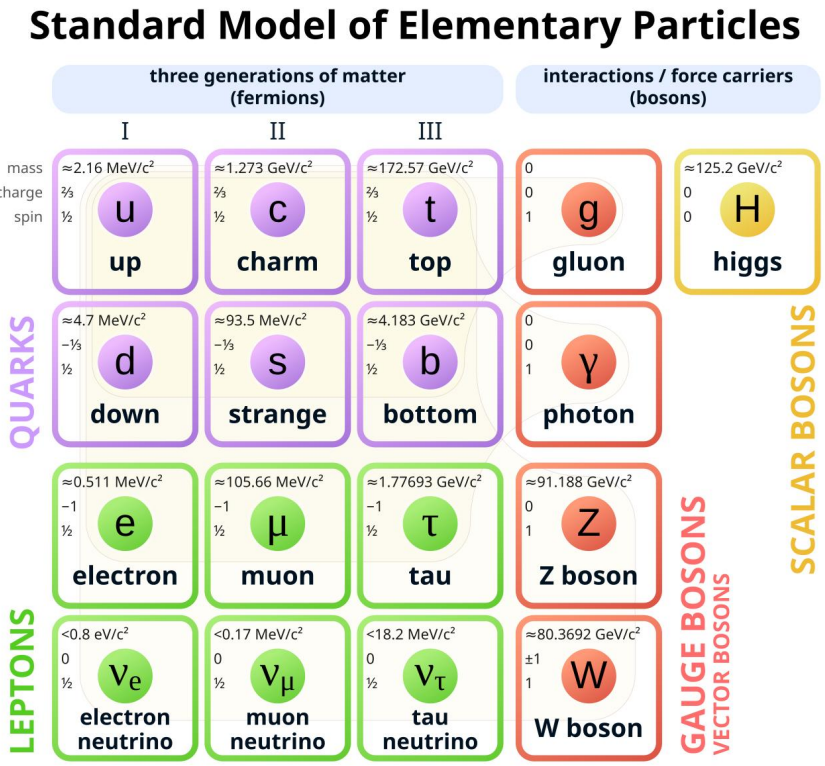
Boosted Decision Trees

Στην τεχνική Boosted Decision Trees, ένας μεγάλος αριθμός δέντρων απόφασης συνδυάζεται ώστε να σχηματίσει έναν ισχυρότερο ταξινομητή. Στην περίπτωση του αλγορίθμου Adaptive Boosting (AdaBoost), κάθε γεγονός εκπαίδευσης συνοδεύεται από ένα βάρος, το οποίο μεταβάλλεται κατά τη διαδοχική κατασκευή των δέντρων.

Μετά την εκπαίδευση ενός δέντρου, αυξάνεται η σχετική σημασία των γεγονότων που ταξινομήθηκαν λανθασμένα. Ως αποτέλεσμα, το επόμενο δέντρο δίνει μεγαλύτερη έμφαση στα γεγονότα που ήταν δυσκολότερο να ταξινομηθούν από τα προηγούμενα δέντρα. Η διαδικασία επαναλαμβάνεται για έναν προκαθορισμένο αριθμό δέντρων.

3 Μποζόνιο Higgs

3.1 Καθιερωμένο Πρότυπο



Σχήμα 8: Το Καθιερωμένο Πρότυπο. (Πηγή:[22])

Το Καθιερωμένο Πρότυπο (Standard Model, SM)[23],η σύνθεση του οποίου συνοψίζεται στο Σχήμα 8, αποτελεί το θεωρητικό μοντέλο που περιγράφει τα στοιχειώδη σωματίδια της ύλης και τις θεμελιώδεις αλληλεπιδράσεις μεταξύ τους, με εξαίρεση τη βαρυτική. Βασίζεται στην Κβαντική Θεωρία Πεδίου (Quantum Field Theory, QFT) και περιγράφει την ισχυρή, την ασθενή και την ηλεκτρομαγνητική αλληλεπίδραση μέσω της ομάδας βαθμίδας

$$SU(3)_C \times SU(2)_L \times U(1)_Y. \tag{41}$$

Τα στοιχειώδη σωματίδια του Καθιερωμένου Προτύπου διακρίνονται, ανάλογα με το σπιν τους, σε φερμιόνια και μποζόνια. Τα φερμιόνια έχουν ημιακέραιο σπιν και αποτελούν τα σωματίδια ύλης, ενώ τα μποζόνια έχουν ακέραιο σπιν και περιλαμβάνουν τους φορείς των θεμελιωδών αλληλεπιδράσεων καθώς και το μποζόνιο Higgs.

Τα φερμιόνια οργανώνονται σε δύο βασικές κατηγορίες, τα κουάρκ (quarks) και τα λεπτόνια (leptons), και κατανέμονται σε τρεις γενιές:

$$\begin{matrix} (u,d) & (c,s) & (t,b) \\ (e,\nu_e) & (\mu,\nu_\mu) & (\tau,\nu_\tau). \end{matrix} \tag{42}$$

Οι τρεις γενιές παρουσιάζουν την ίδια βασική δομή ως προς τις αλληλεπιδράσεις τους, αλλά διαφέρουν κυρίως ως προς τις μάζες των σωματιδίων που τις αποτελούν. Για κάθε φερμιόνιο υπάρχει επίσης το αντίστοιχο αντισωματίδιό του.

Τα κουάρκ συμμετέχουν στην ισχυρή, την ασθενή και, όταν είναι ηλεκτρικά φορτισμένα, στην ηλεκτρομαγνητική αλληλεπίδραση. Αντίθετα, τα φορτισμένα λεπτόνια δεν συμμετέχουν στην ισχυρή αλληλεπίδραση, ενώ τα νετρίνα είναι ηλεκτρικά ουδέτερα και αλληλεπιδρούν μόνο μέσω της ασθενούς αλληλεπίδρασης.

Οι αλληλεπιδράσεις μεταξύ των σωματιδίων περιγράφονται μέσω αντίστοιχων πεδίων βαθμίδας. Η ηλεκτρομαγνητική αλληλεπίδραση επιτυγχάνεται μέσω εκπομπής των φωτονίων γ, η ασθενής από τα μποζόνια $W^\pm$ και Z, ενώ η ισχυρή αλληλεπίδραση από τα γλουόνια g. Το φωτόνιο και τα γλουόνια είναι άμαζα, ενώ τα $W^\pm$ και Z διαθέτουν σημαντική μάζα.

Η ισχυρή αλληλεπίδραση περιγράφεται από την Κβαντική Χρωμοδυναμική (Quantum Chromodynamics, QCD) και αντιστοιχεί στη συμμετρία $SU(3)_C$. Τα κουάρκ φέρουν χρωματικό φορτίο και αλληλεπιδρούν μέσω της ανταλλαγής γλουονίων. Αντίστοιχα, η ηλεκτρομαγνητική και η ασθενής αλληλεπίδραση ενοποιούνται στο πλαίσιο της ηλεκτρασθενούς θεωρίας, η οποία βασίζεται στη συμμετρία

$$SU(2)_L \times U(1)_Y. \tag{43}$$

Η απαίτηση το θεωρητικό πλαίσιο να παραμένει αναλλοίωτο κάτω από τοπικούς μετασχηματισμούς βαθμίδας (local gauge transformations) καθορίζει σε μεγάλο βαθμό τη μορφή των αλληλεπιδράσεων. Ωστόσο, η άμεση εισαγωγή όρων μάζας για τα μποζόνια βαθμίδας $W^\pm$ και Z δεν είναι συμβατή με την αρχική ηλεκτρασθενή συμμετρία. Από την άλλη πλευρά, πειραματικά τα σωματίδια αυτά είναι γνωστό ότι έχουν μη μηδενικές μάζες.

Η αντιμετώπιση αυτού του προβλήματος επιτυγχάνεται μέσω του μηχανισμού Higgs και

του αυθόρμητου σπάσιμου της ηλεκτρασθενούς συμμετρίας (EWSB)[24]. Η εισαγωγή του πεδίου Higgs επιτρέπει στα $W^{\pm}$ και Z να αποκτήσουν μάζα, ενώ μέσω των συζεύξεων Yukawa προκύπτουν επίσης οι όροι μάζας των φερμιονίων. Η κβαντική διέγερση του πεδίου αυτού αντιστοιχεί στο μποζόνιο Higgs.

Η πειραματική ανακάλυψη ενός νέου σωματιδίου συμβατού με το μποζόνιο Higgs, δηλαδή ενός βαθμωτού σωματιδίου με σπιν μηδέν και μάζα κοντά στα 125 GeV, ανακοινώθηκε το 2012 από τα πειράματα ATLAS[25] και CMS[26] στον LHC. Εκτοτε, η μελέτη των ιδιοτήτων, των μηχανισμών παραγωγής και των διασπάσεών του αποτελεί σημαντικό μέρος του ερευνητικού προγράμματος του LHC.

Στις συγκρούσεις πρωτονίου–πρωτονίου, το μποζόνιο Higgs μπορεί να παραχθεί μέσω διαφορετικών φυσικών διεργασιών και στη συνέχεια να διασπαστεί σε διαφορετικές τελικές καταστάσεις. Οι σχετικές πιθανότητες των διαδικασιών αυτών εξαρτώνται από τις συζεύξεις του Higgs με τα υπόλοιπα σωματίδια του Καθιερωμένου Προτύπου. Στη συνέχεια παρουσιάζονται οι κυριότεροι μηχανισμοί παραγωγής και διάσπασης του μποζονίου Higgs, με ιδιαίτερη έμφαση στη διάσπαση $H \to \mu^+\mu^-$, η οποία αποτελεί το αντικείμενο της παρούσας ανάλυσης.

3.2 Τρόποι παραγωγής του μποζονίου Higgs

Στον LHC τα μποζόνια Higgs παράγονται στις συγκρούσεις πρωτονίων υψηλής ενέργειας. Καθώς τα πρωτόνια δεν είναι στοιχειώδη σωματίδια αλλά αποτελούνται από κουάρκ και γκλουόνια, οι θεμελιώδεις διεργασίες λαμβάνουν χώρα μεταξύ των συστατικών τους, τα οποία αναφέρονται συλλογικά ως παρτόνια (partons).

Το μποζόνιο Higgs μπορεί να παραχθεί μέσω διαφορετικών μηχανισμών, οι οποίοι χαρακτηρίζονται από διαφορετικές ενεργές διατομές (cross sections) και διαφορετικές κινηματικές ιδιότητες των τελικών καταστάσεων. Οι σημαντικότεροι μηχανισμοί παραγωγής στον LHC είναι η σύντηξη γκλουονίων (gluon-gluon fusion, ggF), η σύντηξη διανυσματικών μποζονίων (vector boson fusion, VBF), η συσχετισμένη παραγωγή με ένα διανυσματικό μποζόνιο (VH) και η συσχετισμένη παραγωγή με ένα ζεύγος κουάρκ κορυφής ($t\bar{t}H$).[27]

- **ggF**: Η σύντηξη γλουονίων αποτελεί τον κυρίαρχο μηχανισμό παραγωγής του μποζονίου Higgs στον LHC. Στη διαδικασία αυτή δύο γκλουόνια από τα συγκρουόμενα πρωτόνια αλληλεπιδρούν μέσω ενός ενδιάμεσου βρόχου κουάρκ και οδηγούν στην παραγωγή ενός μποζονίου Higgs. Η διαδικασία απεικονίζεται στο Σχήμα 9(a).

$$gg \to H. \tag{44}$$

- **VBF**: Η σύντηξη διανυσματικών μποζονίων αποτελεί έναν ακόμη σημαντικό μηχανισμό παραγωγής. Στη διαδικασία αυτή, δύο κουάρκ από τα συγκρουόμενα πρωτόνια εκπέμπουν ασθενή διανυσματικά μποζόνια W ή Z, τα οποία στη συνέχεια συντήκονται και παράγουν ένα μποζόνιο Higgs, σχήμα 9(b).

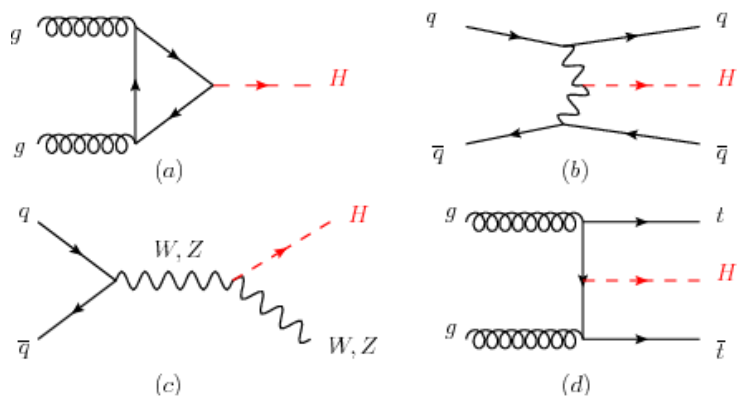
$$qq \to qqH. \tag{45}$$

- **VH**: Το μποζόνιο Higgs μπορεί επίσης να παραχθεί σε συσχετισμό με ένα ασθενές διανυσματικό μποζόνιο W ή Z. Η διαδικασία αυτή, Σχήμα 9(c), γνωστή και ως Higgs-strahlung, περιγράφεται σχηματικά από τις αντιδράσεις

$$q\bar{q} \to WH, \qquad q\bar{q} \to ZH. \tag{46}$$

- $t\bar{t}$**H**: Ένας ακόμη μηχανισμός είναι η συσχετισμένη παραγωγή του μποζονίου Higgs με ένα ζεύγος κουάρκ κορυφής, Σχήμα 9(d).

$$pp \to t\bar{t}H. \tag{47}$$



Σχήμα 9: Διαγράμματα Feynman χαμηλότερης τάξης για τους βασικούς μηχανισμούς παραγωγής του μποζονίου Higgs σε συγκρούσεις πρωτονίου–πρωτονίου. (Πηγή:[28], Fig. 11.1)

Η σχετική συνεισφορά των διαφορετικών μηχανισμών παραγωγής αποτυπώνεται στον πίνακα 2, όπου παρουσιάζονται οι αντίστοιχες ενεργές διατομές συναρτήσει της ενέργειας κέντρου μάζας $\sqrt{s}$. Παρατηρείται ότι η σύντηξη γλουονίων (ggF) αποτελεί τον κυρίαρχο μηχανισμό παραγωγής σε όλο το εξεταζόμενο ενεργειακό εύρος, ενώ η σύντηξη διανυσματικών μποζονίων (VBF) παρουσιάζει τη δεύτερη μεγαλύτερη ενεργό διατομή. Οι υπόλοιποι μηχανισμοί συνεισφέρουν σε μικρότερο βαθμό στη συνολική παραγωγή του μποζονίου Higgs.

$\sqrt{s}$ (TeV)	Production cross section (in pb) for $m_H = 125$ GeV					
	ggF	VBF	WH	ZH	$t\bar{t}H$	total
1.96	$0.95^{+17\%}_{-17\%}$	$0.065^{+8\%}_{-7\%}$	$0.13^{+8\%}_{-8\%}$	$0.079^{+8\%}_{-8\%}$	$0.004^{+10\%}_{-10\%}$	$1.23^{+15\%}_{-15\%}$
7	$16.9^{+5.5\%}_{-7.6\%}$	$1.24^{+2.2\%}_{-2.2\%}$	$0.58^{+2.2\%}_{-2.3\%}$	$0.34^{+3.1\%}_{-3.0\%}$	$0.09^{+5.6\%}_{-10.2\%}$	$19.1^{+5\%}_{-7\%}$
8	$21.4^{+5.4\%}_{-7.6\%}$	$1.60^{+2.1\%}_{-2.1\%}$	$0.70^{+2.1\%}_{-2.2\%}$	$0.42^{+3.4\%}_{-2.9\%}$	$0.13^{+5.9\%}_{-10.1\%}$	$24.2^{+5\%}_{-7\%}$
13	$48.6^{+5.6\%}_{-7.4\%}$	$3.78^{+2.1\%}_{-2.1\%}$	$1.37^{+2.0\%}_{-2.0\%}$	$0.88^{+4.1\%}_{-3.5\%}$	$0.50^{+6.8\%}_{-9.9\%}$	$55.1^{+5\%}_{-7\%}$
13.6	$52.2^{+5.6\%}_{-7.4\%}$	$4.1^{+2.1\%}_{-1.5\%}$	$1.46^{+1.8\%}_{-1.9\%}$	$0.95^{+4.0\%}_{-3.6\%}$	$0.57^{+6.9\%}_{-9.9\%}$	$59.2^{+5\%}_{-7\%}$
14	$54.7^{+5.6\%}_{-7.4\%}$	$4.28^{+2.1\%}_{-2.1\%}$	$1.51^{+1.8\%}_{-1.9\%}$	$0.99^{+4.1\%}_{-3.7\%}$	$0.61^{+6.9\%}_{-9.8\%}$	$62.1^{+5\%}_{-7\%}$

Πίνακας 2: Ενεργές διατομές των κυριότερων μηχανισμών παραγωγής του μποζονίου Higgs για $m_H = 125$ GeV. (Πηγή:[28], Table 11.2)

3.3 Τρόποι διάσπασης του μποζονίου Higgs

Το μποζόνιο Higgs είναι ένα ασταθές σωματίδιο και, μετά την παραγωγή του, διασπάται σε ελαφρύτερα σωματίδια του Καθιερωμένου Προτύπου. Οι δυνατές τελικές καταστάσεις και οι αντίστοιχες πιθανότητες διάσπασης εξαρτώνται από τη μάζα του μποζονίου Higgs και από την ισχύ της σύζευξής του με τα σωματίδια της τελικής κατάστασης. [27]

Για κάθε επιμέρους κανάλι διάσπασης $H \to X$ ορίζεται το μερικό πλάτος διάσπασης (partial decay width) $\Gamma(H \to X)$. Το συνολικό πλάτος διάσπασης του μποζονίου Higgs προκύπτει από το άθροισμα των επιμέρους καναλιών,

$$\Gamma_H = \sum_X \Gamma(H \to X). \tag{48}$$

Η σχετική πιθανότητα πραγματοποίησης ενός συγκεκριμένου καναλιού περιγράφεται από τον λόγο διακλάδωσης (branching ratio, BR), ο οποίος ορίζεται ως

$$BR(H \to X) = \frac{\Gamma(H \to X)}{\Gamma_H}. \tag{49}$$

Για μάζα του μποζονίου Higgs κοντά στα 125 GeV είναι κινηματικά δυνατές αρκετές διαφορετικές τελικές καταστάσεις, όπως φαίνεται στο Σχήμα 3. Μεταξύ των σημαντικότερων καναλιών περιλαμβάνονται οι διασπάσεις σε ζεύγη φερμιονίων, με κυρίαρχη τη διάσπαση $H \to b\bar{b}$, καθώς και η $H \to \tau^+\tau^-$. Σημαντική συνεισφορά παρουσιάζουν επίσης οι διασπάσεις σε διανυσματικά μποζόνια $H \to WW^*$ και $H \to ZZ^*$.

Επιπλέον, στον πίνακα 3 παρουσιάζονται οι διασπάσεις $H \to gg$, $H \to \gamma\gamma$ και $H \to Z\gamma$, οι οποίες πραγματοποιούνται μέσω κβαντικών βρόχων. Παρότι ορισμένα από αυτά τα κανάλια διαθέτουν μικρότερους λόγους διακλάδωσης, μπορούν να παρουσιάζουν σημαντικό πειραματικό ενδιαφέρον λόγω των χαρακτηριστικών τελικών καταστάσεών τους.

Decay channel	Branching ratio	Rel. uncertainty
$H \to \gamma\gamma$	2.28×10^{-3}	$^{+5.0\%}_{-4.9\%}$
$H \to ZZ$	2.64×10^{-2}	$^{+4.3\%}_{-4.1\%}$
$H \to W^+W^-$	2.15×10^{-1}	$^{+4.3\%}_{-4.2\%}$
$H \to \tau^+\tau^-$	6.32×10^{-2}	$^{+5.7\%}_{-5.7\%}$
$H \to b\bar{b}$	5.77×10^{-1}	$^{+3.2\%}_{-3.3\%}$
$H \to Z\gamma$	1.54×10^{-3}	$^{+9.0\%}_{-8.9\%}$
$H \to \mu^+\mu^-$	2.19×10^{-4}	$^{+6.0\%}_{-5.9\%}$

Πίνακας 3: Ο λόγος διακλάδωσης για τα σημαντικότερα κανάλια διάσπασης του Higgs. (Πηγή:[28], Table 11.3)

Στο καθιερωμένο πρότυπο η ισχύς της σύζευξης του μποζονίου Higgs με τα φερμιόνια συνδέεται με τη μάζα τους. Κατά συνέπεια, οι διασπάσεις προς βαρύτερα φερμιόνια είναι γενικά ευνοημένες σε σχέση με τις αντίστοιχες διασπάσεις προς ελαφρύτερα φερμιόνια, εφόσον αυτές είναι κινηματικά επιτρεπτές.

3.4 Πειραματικές μετρήσεις του μποζονίου Higgs

Μετά την ανακάλυψη του μποζονίου Higgs, το ενδιαφέρον των πειραμάτων του LHC στράφηκε στην ακριβή μέτρηση των ιδιοτήτων και των αλληλεπιδράσεών του, με σκοπό τον έλεγχο της συμβατότητάς τους με τις προβλέψεις του Καθιερωμένου Προτύπου.

Ο συνδυασμός διαφορετικών μηχανισμών παραγωγής και καναλιών διάσπασης επιτρέπει τη μέτρηση των αντίστοιχων ρυθμών, καθώς και τη μελέτη των συζεύξεων του μποζονίου Higgs με τα υπόλοιπα σωματίδια.

Ένα μέγεθος που χρησιμοποιείται για τη σύγκριση των πειραματικών μετρήσεων με τις θεωρητικές προβλέψεις είναι η ισχύς σήματος (signal strength) μ, η οποία ορίζεται ως

$$\mu = \frac{(\sigma \times BR)_{\text{meas}}}{(\sigma \times BR)_{\text{SM}}}, \tag{50}$$

όπου σ είναι η ενεργός διατομή παραγωγής και BR ο λόγος διακλάδωσης προς την εξεταζόμενη τελική κατάσταση.

Επομένως, τιμή $\mu = 1$ αντιστοιχεί στην πρόβλεψη του Καθιερωμένου Προτύπου.

Πρόσφατος συνδυασμός των μετρήσεων του πειράματος ATLAS, χρησιμοποιώντας έως 140 fb^{-1} δεδομένων από συγκρούσεις πρωτονίου–πρωτονίου του Run 2 του LHC σε ενέργεια κέντρου μάζας $\sqrt{s} = 13$ TeV, προσδιορίζει τη συνολική ισχύ σήματος ως

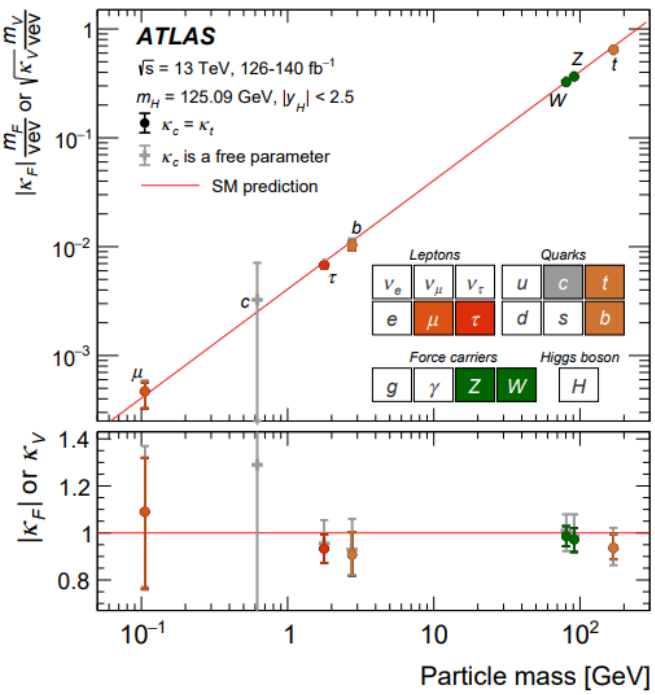
$$\mu = 0.990^{+0.053}_{-0.051}, \tag{51}$$

τιμή που βρίσκεται σε πολύ καλή συμφωνία με την πρόβλεψη του Καθιερωμένου Προτύπου. [29]

Παράλληλα, οι επιμέρους μετρήσεις των διαφορετικών μηχανισμών παραγωγής και καναλιών διάσπασης επιτρέπουν τον έλεγχο των συζεύξεων του μποζονίου Higgs με τα φερμιόνια και τα ασθενή διανυσματικά μποζόνια. Πιθανές αποκλίσεις των συζεύξεων από τις προβλέψεις του Καθιερωμένου Προτύπου μπορούν να παραμετροποιηθούν μέσω των συντελεστών σύζευξης κ_i (coupling modifiers), όπου ο δείκτης i αντιστοιχεί στο εκάστοτε σωματίδιο. Τιμή $\kappa_i = 1$ αντιστοιχεί στην πρόβλεψη του Καθιερωμένου Προτύπου για τη συγκεκριμένη σύζευξη.

Οι πειραματικές μετρήσεις των συζεύξεων παρουσιάζονται στο Σχήμα 10. Στο άνω τμήμα του σχήματος απεικονίζεται η ισχύς των συζεύξεων του μποζονίου Higgs με τα διάφορα φερμιόνια και ασθενή διανυσματικά μποζόνια ως συνάρτηση της μάζας τους. Τα πειραματικά σημεία ακολουθούν συνολικά τη συμπεριφορά που προβλέπεται από το Καθιερωμένο Πρότυπο, σύμφωνα με την οποία η ισχύς της σύζευξης του Higgs με τα σωματίδια συνδέεται με τη μάζα τους.

Στο κάτω τμήμα του σχήματος παρουσιάζονται οι αντίστοιχοι συντελεστές σύζευξης κ_i. Οι μετρούμενες τιμές είναι συνολικά συμβατές με τη μοναδιαία τιμή που προβλέπει το Καθιερωμένο Πρότυπο, εντός των αντίστοιχων πειραματικών αβεβαιοτήτων. Παρατηρείται, ωστόσο, ότι η ακρίβεια των μετρήσεων διαφέρει σημαντικά μεταξύ των διαφορετικών συζεύξεων [29].



Σχήμα 10: Μετρήσεις των συζεύξεων του μποζονίου Higgs με φερμιόνια και ασθενή διανυσματικά μποζόνια συναρτήσει της μάζας των αντίστοιχων σωματιδίων. (Πηγή: [29], fig.7)

3.5 Η Διμιονική Διάσπαση

Ιδιαίτερο ενδιαφέρον για την παρούσα εργασία παρουσιάζει η διάσπαση του μποζονίου Higgs σε ένα ζεύγος μιονίου–αντιμιονίου ($H \to \mu^+\mu^-$).

Στο Καθιερωμένο Πρότυπο οι αλληλεπιδράσεις Higgs είναι οι μόνες που εμφανίζουν διάκριση ανάμεσα στις γενιές των φερμιονίων. Λόγω της μικρής μάζας του μιονίου και, κατά συνέπεια, της ασθενούς σύζευξής του με το μποζόνιο Higgs, το συγκεκριμένο κανάλι χαρακτηρίζεται από πολύ μικρό λόγο διακλάδωσης. Παρόλα αυτά παρέχει παρέχει μια εξαιρετική ευκαιρία για την παρατήρηση της Yukawa σύζευξης του Higgs με τα λεπτόνια δεύτερης γενιάς. [30]

Παράλληλα, τα μιόνια μπορούν να ανακατασκευαστούν με υψηλή ακρίβεια στους ανιχνευτές του LHC, επιτρέποντας την αναζήτηση ενός σήματος γύρω από τη μάζα του μποζονίου Higgs στην κατανομή της αναλλοίωτης μάζας του ζεύγους μιονίων.

Ωστόσο, η παρουσία δύο μιονίων στην τελική κατάσταση δεν αποτελεί μοναδική υπογραφή της διάσπασης του μποζονίου Higgs. Η σημαντικότερη πηγή υποβάθρου προέρχεται από τη διαδικασία Drell–Yan, η οποία οδηγεί στην ίδια τελική κατάσταση με το υπό μελέτη σήμα.

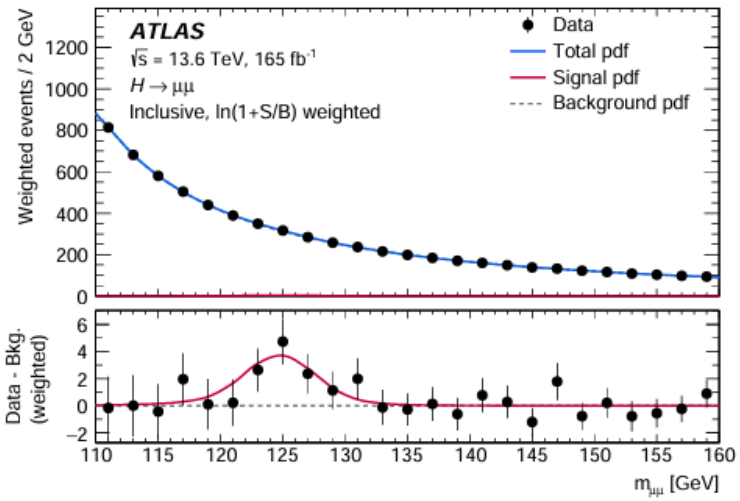
$$q\bar{q} \to Z/\gamma^* \to \mu^+\mu^-, \tag{52}$$

Η ενεργός διατομή της συγκεκριμένης διαδικασίας είναι πολύ μεγαλύτερη από εκείνη της παραγωγής και επακόλουθης διάσπασης $H \to \mu^+\mu^-$, με αποτέλεσμα το αναμενόμενο σήμα να βρίσκεται πάνω σε ένα σημαντικά μεγαλύτερο συνεχές υπόβαθρο γεγονότων διμιονίου.

Παρότι η κορυφή της παραγωγής $Z \to \mu^+\mu^-$ εντοπίζεται στην περιοχή $m_{\mu\mu} \simeq m_Z$, η

κατανομή Drell–Yan εκτείνεται και σε μεγαλύτερες τιμές της αναλλοίωτης μάζας. Κατά συνέπεια, σημαντικός αριθμός γεγονότων υποβάθρου παραμένει στην περιοχή $m_{\mu\mu} \simeq 125$ GeV, όπου αναζητείται το σήμα του μποζονίου Higgs.

Η παραπάνω συμπεριφορά αποτυπώνεται στο Σχήμα 11, όπου παρουσιάζεται η κατανομή της αναλλοίωτης μάζας του διμιονίου σε δεδομένα του πειράματος ATLAS. Η κατανομή κυριαρχείται από το ομαλά μεταβαλλόμενο υπόβαθρο, ενώ η συνεισφορά της διάσπασης $H \to \mu^+\mu^-$ εμφανίζεται ως μια μικρή τοπική περίσσεια γεγονότων γύρω από $m_{\mu\mu} \simeq 125$ GeV. Στο κάτω τμήμα του σχήματος, μετά την αφαίρεση του εκτιμώμενου υποβάθρου, η συνεισφορά αυτή γίνεται περισσότερο εμφανής.



Σχήμα 11: Κατανομή της αναλλοίωτης μάζας του διμιονίου στην αναζήτηση της διάσπασης $H \to \mu^+\mu^-$ από το πείραμα ATLAS, με δεδομένα του Run 3 σε $\sqrt{s} = 13.6$ TeV. (Πηγή:[30], fig.1)

Η αναζήτηση της διάσπασης $H \to \mu^+\mu^-$ αποτελεί επομένως ένα απαιτητικό πρόβλημα διάκρισης ενός μικρού σήματος από ένα πολύ μεγαλύτερο υπόβαθρο με την ίδια παρατηρούμενη τελική κατάσταση.

Ειδοποιός διαφορά των δύο διασπάσεων αποτελεί το σπιν των σωματιδίων. Το Higgs είναι βαθμωτό σωματίδιο με σπιν 0, ενώ το Z είναι διανυσματικό σωματίδιο με σπιν 1.

Η διαφορά αυτή επηρεάζει τη γωνιακή κατανομή των παραγόμενων μιονίων. Στην περίπτωση της διάσπασης ενός βαθμωτού μποζονίου Higgs, δεν υπάρχει προτιμώμενη διεύθυνση διάσπασης στο σύστημα ηρεμίας του, με αποτέλεσμα η κατανομή να είναι ισότροπη. Αντίθετα, στη διαδικασία $Z/\gamma^* \to \mu^+\mu^-$, η διανυσματική φύση της αλληλεπίδρασης και η πόλωση του ενδιάμεσου μποζονίου οδηγούν σε χαρακτηριστική γωνιακή εξάρτηση.

Κατά συνέπεια, μεταβλητές που περιγράφουν τη γωνιακή κατανομή των δύο μιονίων μπορούν να παρέχουν πρόσθετη πληροφορία για τη διάκριση μεταξύ γεγονότων σήματος και υποβάθρου.

Τέλος, σημαντικό ρόλο στην μελέτη των δύο διασπάεων διαδραματίζει και ο αριθμός των πιδάκων (jets) που παρατηρούνται σε κάθε γεγονός. Ως jets χαρακτηρίζονται οι στενές δέσμες από πολλά αδρόνια (σωματίδια αποτελούμενα από κουάρκ και γλουόνια) τα οποία κινούνται προς παρόμοιες κατευθύνσεις μέσα στον ανιχνευτή. Οι πίδακες ανακατασκευάζο-

νται πειραματικά και παρέχουν πληροφορία για τα αρχικά παρτόνια που συμμετείχαν στη διεργασία.

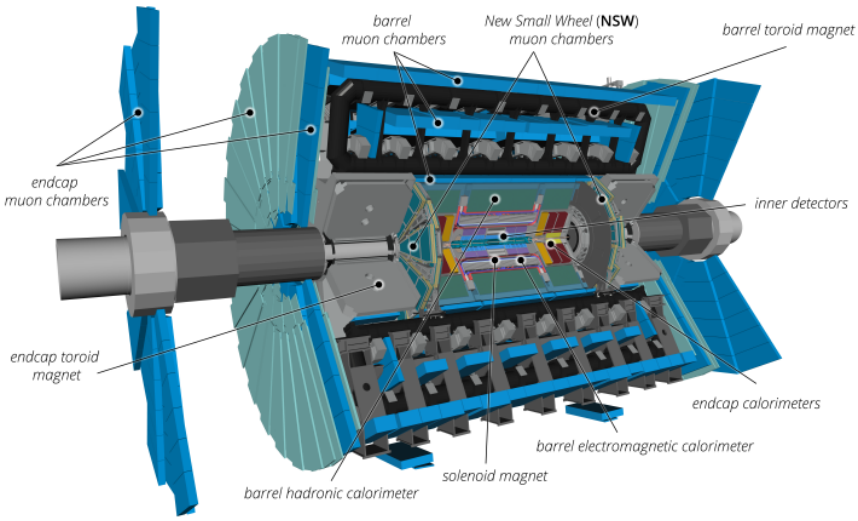
Ο αριθμός των ανακατασκευασμένων πιδάκων σε ένα γεγονός, n_j, αποτελεί σημαντικό χαρακτηριστικό της τελικής του κατάστασης και σχετίζεται με τον μηχανισμό παραγωγής και την επιπλέον αδρονική ακτινοβολία. Για παράδειγμα, τα γεγονότα VBF χαρακτηρίζονται συνήθως από την παρουσία δύο πιδάκων που προέρχονται από τα εξερχόμενα κουάρκ, ενώ πίδακες μπορούν επίσης να εμφανιστούν σε γεγονότα ggF λόγω επιπλέον ακτινοβολίας QCD.

Για τον λόγο αυτό, η κατηγοριοποίηση των γεγονότων σύμφωνα με τον αριθμό των ανακατασκευασμένων πιδάκων μπορεί να χρησιμοποιηθεί για τον διαχωρισμό γεγονότων με διαφορετικές κινηματικές και παραγωγικές ιδιότητες.

3.6 Το πείραμα ATLAS

Το ATLAS (A Toroidal LHC ApparatuS)[6] είναι ένα από τα πειράματα γενικού σκοπού του LHC στο CERN. Έχει σχεδιαστεί για τη μελέτη ενός ευρέος φάσματος φυσικών διεργασιών που προκύπτουν από συγκρούσεις πρωτονίων υψηλής ενέργειας. Μεταξύ των βασικών στόχων του περιλαμβάνονται οι ακριβείς μετρήσεις του Καθιερωμένου Προτύπου και η μελέτη των ιδιοτήτων του μποζονίου Higgs.

Ο ανιχνευτής έχει περίπου κυλινδρική γεωμετρία και αποτελείται από διαδοχικά υποσυστήματα γύρω από το σημείο αλληλεπίδρασης, όπως φαίνεται στο Σχήμα 12. Κάθε υποσύστημα καταγράφει διαφορετικές ιδιότητες των σωματιδίων που παράγονται στις συγκρούσεις, επιτρέποντας την ανακατασκευή και ταυτοποίησή τους.



Σχήμα 12: Σχηματική απεικόνιση τομής του ανιχνευτή ATLAS για το RUN 3, που υποδεικνύει τις θέσεις των μεγαλύτερων υποσυστημάτων ανιχνευτή. (Πηγή:[6], fig.1.2)

- **Εσωτερικός ανιχνευτής(Inner Detector)**: Βρίσκεται πλησιέστερα στο σημείο αλληλεπίδρασης και χρησιμοποιείται για την ανακατασκευή των τροχιών των φορτισμένων σωματιδίων και τη μέτρηση της ορμής τους μέσα σε μαγνητικό πεδίο. Εξωτερικά αυτού βρίσκονται τα ηλεκτρομαγνητικά και αδρονικά θερμιδόμετρα (calorimeters).

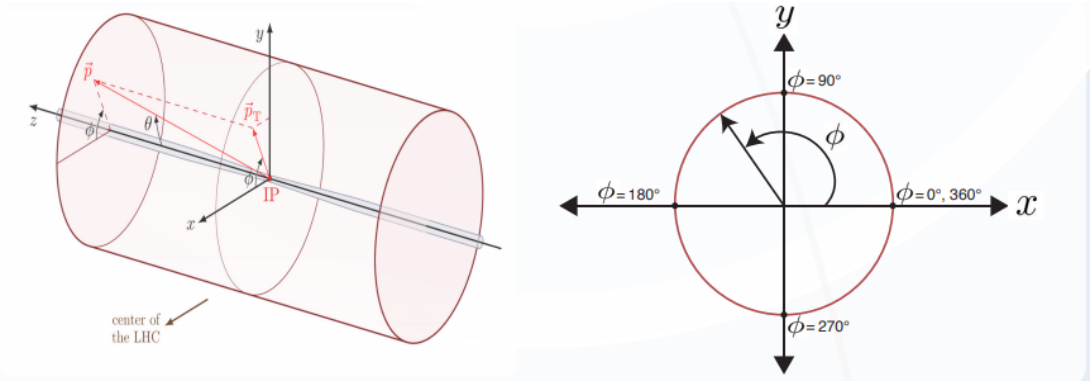
- **Θερμιδόμετρα (Calorimeters)** : Βρίσκονται εξωτερικά του εσωτερικού συστήματος ανίχνευσης και χρησιμοποιούνται για τη μέτρηση της ενέργειας των σωματιδίων μέσω της απορρόφησής τους και της ανάπτυξης καταιγισμών σωματιδίων (particle showers). Το ηλεκτρομαγνητικό θερμιδόμετρο (Electromagnetic Calorimeter) χρησιμοποιείται κυρίως για τη μέτρηση της ενέργειας ηλεκτρονίων και φωτονίων μέσω των ηλεκτρομαγνητικών καταιγισμών που αναπτύσσουν στο υλικό του ανιχνευτή.

  Το αδρονικό θερμιδόμετρο (Hadronic Calorimeter) βρίσκεται εξωτερικά του ηλεκτρομαγνητικού και χρησιμοποιείται κυρίως για τη μέτρηση της ενέργειας των αδρονίων.

- **Φασματόμετρο μιονίων (Muon Spectrometer)**: Είναι το εξωτερικότερο τμήμα του ανιχνευτή και έχει σχεδιαστεί για την ανίχνευση και την ακριβή μέτρηση της ορμής των μιονίων. Σε αντίθεση με τα περισσότερα φορτισμένα σωματίδια, τα μιόνια διαπερνούν τα θερμιδόμετρα χάνοντας σχετικά μικρό μέρος της ενέργειάς τους και φθάνουν στο εξωτερικό σύστημα ανίχνευσης. Ο συνδυασμός των μετρήσεων του εσωτερικού ανιχνευτή και του φασματόμετρου μιονίων επιτρέπει την ανακατασκευή της τροχιάς και της ορμής τους με υψηλή ακρίβεια.

- **Σύστημα Μαγνητών**: Αποτελείται από έναν εσωτερικό υπεραγώγιμο σωληνοειδή μαγνήτη και τρεις τεράστιους υπεραγώγιμους τοροειδείς μαγνήτες. Σκοπός τους είναι να δημιουργούν τα τεράστια μαγνητικά πεδία που είναι απαραίτητα για την καμπύλωση των τροχιών των φορτισμένων σωματιδίων.

Για την περιγραφή των σωματιδίων και των κινηματικών μεγεθών στο πείραμα ATLAS[6] χρησιμοποιείται ένα δεξιόστροφο σύστημα συντεταγμένων, το οποίο απεικονίζεται στο σχήμα 13. Η αρχή του συστήματος συντεταγμένων βρίσκεται στο ονομαστικό σημείο αλληλεπίδρασης (Interaction Point, IP), ενώ ο άξονας z ορίζεται κατά μήκος της διεύθυνσης των δεσμών πρωτονίων. Οι άξονες x και y ορίζουν το εγκάρσιο προς τη δέσμη επίπεδο.



Σχήμα 13: Σχηματική απεικόνιση του συστήματος συντεταγμένων που χρησιμοποιείται στο πείραμα ATLAS, της πολικής γωνίας θ, της αζιμουθιακής γωνίας ϕ και της εγκάρσιας ορμής p_T. (Πηγή: [31])

Η διεύθυνση της ορμής ενός σωματιδίου μπορεί να περιγραφεί μέσω της πολικής γωνίας θ και της αζιμουθιακής γωνίας ϕ. Η γωνία θ μετράται ως προς τον άξονα z, ενώ η ϕ ορίζεται στο εγκάρσιο επίπεδο $x - y$, όπως φαίνεται στο Σχήμα 13.

Ιδιαίτερα σημαντική ποσότητα είναι η εγκάρσια ορμή (transverse momentum) p_T, η οποία αντιστοιχεί στο μέτρο της προβολής της ορμής $\vec{p}$ στο εγκάρσιο επίπεδο και ορίζεται ως

$$p_T = \sqrt{p_x^2 + p_y^2}. \tag{53}$$

Η χρήση της εγκάρσιας ορμής είναι ιδιαίτερα χρήσιμη στις συγκρούσεις πρωτονίων, καθώς οι διαμήκεις ορμές των παρτονίων που συμμετέχουν στην επιμέρους σκληρή σκέδαση δεν είναι γνωστές εκ των προτέρων.

Αντί της πολικής γωνίας θ, χρησιμοποιείται συχνά η ψευδοωκύτητα (pseudorapidity) η, η οποία ορίζεται ως

$$\eta = -\ln\left[\tan\left(\frac{\theta}{2}\right)\right]. \tag{54}$$

Για $\theta = 90°$, δηλαδή για κίνηση κάθετα προς τη διεύθυνση των δεσμών, ισχύει $\eta = 0$, ενώ μεγάλες απόλυτες τιμές $|\eta|$ αντιστοιχούν σε διευθύνσεις που πλησιάζουν τον άξονα των δεσμών.

4 Δεδομένα και Μεταβλητές

4.1 Ανοικτά Δεδομένα ATLAS

Τα ανοικτά δεδομένα [5] του πειράματος ATLAS παρέχουν δημόσια πρόβαση σε καταγεγραμμένες συγκρούσεις σωματιδιακής φυσικής του LHC, καθώς και σε Monte Carlo προσομειωμένα γεγονότα. Σκοπός της πρωτοβουλίας είναι η εύκολη πρόσβαση και επεξεργασία πραγματικών δεδομένων Φυσικής Υψηλών Ενεργειών είτε για εκπαιδευτικούς είτε για ερευνητικούς σκοπούς.

Για την παρούσα ανάλυση που στοχεύει στην μελέτη της διάσπασης $H \to \mu^+\mu^-$ χρησιμοποιούνται Monte Carlo προσομειωμένα δεδομένα από την έκδοση που αντιστοιχεί σε συγκρούσεις πρωτονίου - πρωτονίου των ετών 2015 και 2016 με ενέργεια κέντρου μάζας $\sqrt{s} = 13$ TeV. Συγκεκριμένα, χρησιμοποιούνται δείγματα της κατηγορίας `2muons`, τα οποία αποτελούν προεπιλεγμένα υποσύνολα (skims) των αρχικών δεδομένων, κατάλληλα για αναλύσεις με τελικές καταστάσεις που περιλαμβάνουν μιόνια [32]

4.1.1 Δείγματα Σήματος και Υποβάθρου

Όπως είδαμε στην ενότητα 2.2.2, στα προβλήματα δυαδικής κατηγοριοποίησης είναι απαραίτητο τα δεδομένα να συνοδεύονται από μία ετικέτα. Συνεπώς, από τα διαθέσιμα δείγματα πρέπει να προσδιοριστούν αυτά που αντιστοιχούν στο σήμα της υπό μελέτη διαδικασίας ($H \to \mu^+\mu^-$) καθώς και το αντίστοιχο κυρίαρχο υπόβαθρο ($Z/\gamma^* \to \mu^+\mu^-$).

Τα επιμέρους δείγματα Monte Carlo[32] που χρησιμοποιήθηκαν παρουσιάζονται στον Πίνακα 4. Για την προσομοίωση του σήματος χρησιμοποιήθηκαν τρία δείγματα, που αντιστοιχούν σε τρεις διαφορετικούς μηχανισμούς παραγωγής του Higgs και ως υπόβαθρο τρία δείγματα της διαδικασίας Drell–Yan. Ο διαχωρισμός των τριών δειγμάτων του υποβάθρου

αφορά διαφορετικές απαιτήσεις φιλτραρίσματος της προσομοίωσης με βάση το περιεχόμενο βαρέων γεύσεων.

Πίνακας 4: Προσομοιωμένα δείγματα Monte Carlo που χρησιμοποιήθηκαν στην ανάλυση.

Κατηγορία	Δείγμα	DSID	Διεργασία
Σήμα	ggH	345097	$gg \to H \to \mu^+\mu^-$
Σήμα	$ggZH$	345098	$gg \to ZH,\ H \to \mu^+\mu^-$
Σήμα	VBF	345106	$qq \to qqH,\ H \to \mu^+\mu^-$
Υπόβαθρο	BFilter	700323	$Z/\gamma^* \to \mu^+\mu^- + \text{jets}$
Υπόβαθρο	CFilterBVeto	700324	$Z/\gamma^* \to \mu^+\mu^- + \text{jets}$
Υπόβαθρο	CVetoBVeto	700325	$Z/\gamma^* \to \mu^+\mu^- + \text{jets}$

4.1.2 Αξιοποίηση των Δεδομένων στο ROOT και TMVA

Τα προσομοιωμένα δείγματα που παρουσιάστηκαν προηγουμένως διατίθενται σε μορφή αρχείων ROOT, τα οποία επιτρέπουν την αποθήκευση και επεξεργασία μεγάλου όγκου δεδομένων. Οι πληροφορίες των γεγονότων είναι οργανωμένες σε αντικείμενα TTree με όνομα `analysis`, όπου κάθε εγγραφή του TTree αντιστοιχεί σε ένα προσομειωμένο γεγονός. Οι επιμέρους πληροφορίες κάθε γεγονότος αποθηκεύονται σε κλάδους (branches) οι οποίοι περιέχουν τις αντίστοιχες φυσικές και κινηματικές ποσότητες.

Οι κλάδοι των δειγμάτων που χρησιμοποιήθηκαν περιέχουν πληροφορίες για τα ανακατασκευασμένα λεπτόνια (π.χ. τύπος, ηλεκτρικό φορτίο, εγκάρσια ορμή, απομόνωση κ.ο.κ) και για τους ανακατασκευασμένους πίδακες αντίστοιχα. Οι πληροφορίες αυτές χρησιμοποιήθηκαν τόσο για τον εφαρμογή των κριτηρίων επιλογής, όσο και για την κατασκευή των μεταβλητών που δόθηκαν στις τεχνικές ταξινόμησης.

Τέλος, τα προσομοιωμένα γεγονότα συνοδεύονται από πληροφορίες σχετικές με τα βάρη προσομοίωσης και τους αντίστοιχους διορθωτικούς συντελεστές. Οι ποσότητες αυτές χρησιμοποιήθηκαν για τον υπολογισμό του τελικού βάρους κάθε γεγονότος, όπως περιγράφεται στη συνέχεια.

Η επεξεργασία των παραπάνω πληροφοριών πραγματοποιήθηκε στο περιβάλλον ROOT με χρήση της γλώσσας C_{++}. Από τα αρχικά TTree κατασκευάστηκαν, μετά την εφαρμογή των κριτηρίων επιλογής, νέα σύνολα δεδομένων που περιείχαν τις μεταβλητές εισόδου και τις απαραίτητες πληροφορίες παρακολούθησης για κάθε γεγονός. Τα σύνολα αυτά χρησιμοποιήθηκαν στη συνέχεια στο Toolkit for Multivariate Data Analysis (TMVA) για την εκπαίδευση και την αξιολόγηση των τεχνικών κατηγοριοποίησης.

4.2 Κριτήρια Επιλογής γεγονότων

Η επιλογή των γεγονότων της ανάλυσης βασίστηκε στα κριτήρια επιλογής της αντίστοιχης ανάλυσης για το Run 3 του πειράματος ATLAS [30]. Ωστόσο, τα κριτήρια αυτά δεν εφαρμόστηκαν αυτούσια, καθώς η παρούσα εργασία διαφέρει ως προς τον σκοπό της ανάλυσης και βασίζεται σε διαφορετικό σύνολο δεδομένων και διαθέσιμων πληροφοριών.

Στον Πίνακα 5 παρουσιάζονται τα κριτήρια επιλογής γεγονότων που χρησιμοποιήθηκαν στην ανάλυση του Run 3. Μία πρώτη διαφορά σε σχέση με την παρούσα εργασία αφορά τα χρησιμοποιούμενα δείγματα σήματος, καθώς στην ανάλυση αναφοράς περιλαμβάνεται ευρύτερο σύνολο μηχανισμών παραγωγής του μποζονίου Higgs, συμπεριλαμβανομένων των $t\bar{t}H$ και VH.

	Selection
Common preselection	Primary vertex Two opposite-charge muons Muons: $\|\eta\| < 2.5$, $p_{\mathrm{T}}^{\mathrm{lead}} > 27$ GeV, $p_{\mathrm{T}}^{\mathrm{sublead}} > 15$ GeV
Fit region	$m_{\mu\mu} = 110 - 160$ GeV
Jets	$p_{\mathrm{T}} > 25$ GeV and $\|\eta\| < 2.4$ or with $p_{\mathrm{T}} > 30$ GeV and $2.4 < \|\eta\| < 4.5$
b-tagged jets	$p_{\mathrm{T}} > 25$ GeV and $\|\eta\| < 2.4$ or with $p_{\mathrm{T}} > 30$ GeV and $2.4 < \|\eta\| < 2.5$ Tagging efficiency working point of 85%
$t\bar{t}H$ categories	At least one b-jet
VH 4-lepton category	Exactly two additional e or μ with $p_{\mathrm{T}} > 8$ GeV, 5 GeV (μ) / 7 GeV (e), no b-jets
VH 3-lepton categories	Exactly one additional e or μ with $p_{\mathrm{T}} > 15$ GeV, no b-jets
VH 2-lepton categories	No additional lepton, no b-jets, $E_{\mathrm{T}}^{\mathrm{miss}} > 120$ GeV
VBF and ggF categories	No additional lepton, no b-jets, $E_{\mathrm{T}}^{\mathrm{miss}} < 120$ GeV

Πίνακας 5: Κριτήρια επιλογής γεγονότων της ανάλυσης Run 3 του πειράματος ATLAS (Πηγή: [30], table 1)

Με βάση τα παραπάνω κριτήρια, η επιλογή των γεγονότων της παρούσας εργασίας πραγματοποιήθηκε σε διαδοχικά στάδια. Αρχικά κατασκευάστηκε ο υποψήφιος (candidate) $H \to \mu^+\mu^-$, μέσω της επιλογής του κατάλληλου ζεύγους μιονίων, και στη συνέχεια τα επιλεγμένα γεγονότα διαχωρίστηκαν σε κατηγορίες ανάλογα με τον αριθμό των jets.

4.2.1 Επιλογή υποψηφίου $H \to \mu^+\mu^-$

Η αρχική επιλογή των μιονίων πραγματοποιήθηκε από τα ανακατασκευασμένα λεπτόνια κάθε γεγονότος. Τα αρχεία των οποίων τα δεδομένα χρησιμοποιήθηκαν είχαν προηγουμένως υποστεί προεπιλογή ως προς το περιεχόμενο σε μιόνια (`2muons skim`). Συγκεκριμένα, διατηρήθηκαν γεγονότα που περιείχαν τουλάχιστον 2 μιόνια με εγκάρσια ορμή $p_T > 10$ GeV, και ικανοποιούσαν το χαλαρό κριτήριο ταυτοποίησης (`Loose ID`).

Στη συνέχεια, μεταξύ των κατάλληλων μιονίων αναζητήθηκαν όλα τα δυνατά ζεύγη αντίθετου ηλεκτρικού φορτίου, όπως απαιτείται για τη διάσπαση $H \to \mu^+\mu^-$.

Σε γεγονότα στα οποία υπήρχαν περισσότερα από ένα δυνατά ζεύγη, ως υποψήφιος $H \to \mu^+\mu^-$ επιλέχθηκε το ζεύγος με τη μεγαλύτερη εγκάρσια ορμή του πρωτεύοντος μιονίου (leading muon) και, σε περίπτωση ισοδυναμίας, με τη μεγαλύτερη εγκάρσια ορμή του δευτερεύοντος μιονίου (subleading muon).

Αφού προσδιορίστηκε το ζεύγος μιονίων, εφαρμόστηκαν τα κινηματικά κριτήρια της επιλογής. Το μιόνιο με τη μεγαλύτερη εγκάρσια ορμή απαιτήθηκε να ικανοποιεί $p_T^{\mu_1} > 27$ GeV, ενώ για το δεύτερο μιόνιο απαιτήθηκε $p_T^{\mu_2} > 15$ GeV. Παράλληλα, και τα δύο μιόνια απαιτήθηκε να βρίσκονται εντός της περιοχής $|\eta_\mu| < 2.5$.

Οι απαιτήσεις αυτές περιορίζουν την επιλογή σε μιόνια με κατάλληλη κινηματική και εντός της περιοχής αποδοχής που χρησιμοποιείται στην ανάλυση.

Για την περαιτέρω επιλογή των μιονίων χρησιμοποιήθηκαν κριτήρια που σχετίζονται με την παράμετρο κρούσης ως προς την πρωτεύουσα κορυφή (primary vertex) του γεγονότος. Συγκεκριμένα, απαιτήθηκε $\left|\frac{z_0}{\cosh\eta}\right| < 0.5$ mm, $|d_0^{\text{sig}}| < 3$. Τα κριτήρια αυτά περιορίζουν τη συνεισφορά μιονίων που δεν είναι συμβατά με την πρωτεύουσα αλληλεπίδραση του γεγονότος.

Επιπλέον, και τα δύο μιόνια του υποψηφίου απαιτήθηκε να ικανοποιούν το κριτήριο απομόνωσης που παρέχεται στα Open Data, μέσω της μεταβλητής `lep_isLooseIso`. Η απαίτηση απομόνωσης χρησιμοποιείται για τον περιορισμό μιονίων που βρίσκονται εντός περιοχών με σημαντική δραστηριότητα άλλων σωματιδίων, όπως μπορεί να συμβαίνει στην περίπτωση μιονίων που προέρχονται από διασπάσεις αδρονίων.

Στη συνέχεια εφαρμόστηκε η απαίτηση σκανδαλισμού (trigger) του γεγονότος. Για τον σκοπό αυτό χρησιμοποιήθηκε η μεταβλητή `lep_isTrigMatched` και απαιτήθηκε τουλάχιστον ένα από τα δύο μιόνια του επιλεγμένου ζεύγους να είναι αντιστοιχισμένο με τον αντίστοιχο σκανδαλισμό.

Ακολούθως, απορρίφθηκαν γεγονότα στα οποία, πέρα από τα δύο μιόνια που σχηματίζουν τον υποψήφιο $H \to \mu^+\mu^-$, υπήρχε επιπλέον αποθηκευμένο ηλεκτρόνιο ή μιόνιο με $p_T > 7$ GeV που ικανοποιούσε το κριτήριο ταυτοποίησης `Loose ID`. Με τον τρόπο αυτό εφαρμόστηκε η απαίτηση απουσίας επιπλέον λεπτονίων που χρησιμοποιείται στην επιλογή των γεγονότων.

Τέλος, η αναλλοίωτη μάζα του επιλεγμένου ζεύγους μιονίων υπολογίστηκε από τα τετραδιανύσματα των δύο μιονίων και απαιτήθηκε να βρίσκεται στην περιοχή $120 < m_{\mu\mu} < 130$ GeV. Η επιλογή αυτή αποτελεί προσαρμογή σε σχέση με την ανάλυση αναφοράς, στην οποία χρησιμοποιείται ευρύτερη περιοχή αναλλοίωτης μάζας. Στην παρούσα εργασία επιλέχθηκε ένα στενότερο παράθυρο γύρω από τη μάζα του μποζονίου Higgs, καθώς το ενδιαφέρον επικεντρώνεται στην εφαρμογή των τεχνικών κατηγοριοποίησης στην περιοχή του σήματος.

4.2.2 Επιλογή jets

Μετά την επιλογή του υποψηφίου $H \to \mu^+\mu^-$, τα γεγονότα διαχωρίστηκαν σε κατηγορίες ανάλογα με τον αριθμό των ανακατασκευασμένων πιδάκων. Για τον σκοπό αυτό χρησιμοποιήθηκαν οι πίδακες που είναι αποθηκευμένοι στα χρησιμοποιούμενα Open Data.

Κατά την παραγωγή των Open Data έχει ήδη πραγματοποιηθεί μια αρχική επιλογή των πιδάκων. Συγκεκριμένα, οι αποθηκευμένοι πίδακες απαιτείται να ικανοποιούν $p_T^j > 20$ GeV, $|\eta_j| < 2.5$.

Επιπλέον, πριν από την αποθήκευσή τους εφαρμόζεται διαδικασία overlap removal, με σκοπό την αποφυγή της διπλής καταμέτρησης ανακατασκευασμένων αντικειμένων που επικαλύπτονται.

Κατά συνέπεια, στην παρούσα ανάλυση δεν επαναλαμβάνονται τα παραπάνω κριτήρια επιλογής. Ως αριθμός πιδάκων ενός γεγονότος χρησιμοποιείται απευθείας η μεταβλητή `jet_n`, η οποία αντιστοιχεί στον αριθμό των αποθηκευμένων πιδάκων που έχουν περάσει την παρα-

πάνω επιλογή. Επομένως, ο αριθμός πιδάκων της ανάλυσης ορίζεται ως

$$n_j \equiv \texttt{jet_n}.$$

Με βάση την πολλαπλότητα των πιδάκων, τα γεγονότα διαχωρίστηκαν σε τρεις αμοιβαία αποκλειόμενες κατηγορίες:

$$\begin{aligned} n_j = 0: &\quad \texttt{jet_n} = 0, \\ n_j = 1: &\quad \texttt{jet_n} = 1, \\ n_j \geq 2: &\quad \texttt{jet_n} \geq 2. \end{aligned}$$

Ο διαχωρισμός σε κατηγορίες πολλαπλότητας πιδάκων είναι ιδιαίτερα χρήσιμος στην αναζήτηση του $H \to \mu^+\mu^-$, καθώς οι διαφορετικοί μηχανισμοί παραγωγής του μποζονίου Higgs χαρακτηρίζονται από διαφορετική δραστηριότητα πιδάκων. Ωστόσο, οι κατηγορίες της παρούσας εργασίας δεν ταυτίζονται με συγκεκριμένους μηχανισμούς παραγωγής. Γεγονότα από τα δείγματα ggH, VBF και $ggZH$ μπορούν να συνεισφέρουν στις διαφορετικές κατηγορίες ανάλογα με τον αριθμό των ανακατασκευασμένων πιδάκων του κάθε γεγονότος.

Τα κριτήρια επιλογής των γεγονότων συνοψίζονται στον Πίνακα 6 .

Πίνακας 6: Κριτήρια επιλογής γεγονότων της παρούσας ανάλυσης.

Κοινή επιλογή γεγονότων	
Υποψήφιος $H \to \mu^+\mu^-$	Ζεύγος μιονίων αντίθετου ηλεκτρικού φορτίου
Μιόνια	`Loose ID`, $p_T^{\mu_1} > 27$ GeV, $p_T^{\mu_2} > 15$ GeV, $\lvert\eta_\mu\rvert < 2.5$
Παράμετροι κρούσης	$\lvert z_0/\cosh\eta\rvert < 0.5$ mm, $\lvert d_0^{\text{sig}}\rvert < 3$
Απομόνωση	`lep_isLooseIso` $= 1$ και για τα δύο μιόνια
Σκανδαλισμός	Τουλάχιστον ένα μιόνιο με `lep_isTrigMatched` $= 1$
Επιπλέον λεπτόνια	Απουσία επιπλέον ηλεκτρονίου ή μιονίου με $p_T > 7$ GeV και `Loose ID`
Αναλλοίωτη μάζα	$120 < m_{\mu\mu} < 130$ GeV
Επιλογή πιδάκων	
Πίδακες	$p_T^j > 20$ GeV, $\lvert\eta_j\rvert < 2.5$, overlap removal
Κατηγορίες πολλαπλότητας πιδάκων	
$n_j = 0$	Κανένας πίδακας
$n_j = 1$	Ακριβώς ένας πίδακας
$n_j \geq 2$	Τουλάχιστον δύο πίδακες

4.3 Στάθμιση και κανονικοποίηση των γεγονότων

Τα δείγματα Monte Carlo που χρησιμοποιούνται στην ανάλυση δεν αποτελούν απλώς σύνολα ισοδύναμων γεγονότων. Η απευθείας χρήση του αριθμού των προσομοιωμένων γεγονότων θα αγνοούσε τόσο τις διαφορετικές πιθανότητες παραγωγής των επιμέρους φυσικών διεργασιών όσο και τις αποκλίσεις μεταξύ της προσομοίωσης και των πραγματικών δεδομένων ως προς την απόκριση του ανιχνευτή και τις αποδόσεις των κριτηρίων επιλογής. Για τον λόγο αυτό, σε κάθε προσομοιωμένο γεγονός αποδίδεται ένα βάρος, το οποίο λαμβάνει υπόψη τις

παραπάνω επιδράσεις και επιτρέπει τη συνεπή χρήση και τον συνδυασμό των διαφορετικών δειγμάτων.

Η διαδικασία που ακολουθήθηκε αποτελείται από δύο στάδια. Αρχικά εφαρμόζονται οι διορθώσεις που αφορούν το ίδιο το γεγονός και την ανακατασκευή των φυσικών αντικειμένων και στη συνέχεια πραγματοποιείται η κανονικοποίηση του κάθε δείγματος με βάση τα χαρακτηριστικά της αντίστοιχης προσομοιωμένης διεργασίας.

4.3.1 Στάθμιση των προσομοιωμένων γεγονότων

Για κάθε γεγονός χρησιμοποιείται αρχικά το βάρος `mcWeight`, το οποίο παρέχεται στα Open Data και προέρχεται από τη διαδικασία παραγωγής του αντίστοιχου δείγματος Monte Carlo. Το βάρος αυτό αποτελεί τη βασική συνεισφορά της γεννήτριας στο συνολικό βάρος του γεγονότος και μπορεί, ανάλογα με το χρησιμοποιούμενο δείγμα, να λαμβάνει και αρνητικές τιμές.

Πέρα από το βάρος της γεννήτριας, απαιτούνται διορθωτικοί παράγοντες ώστε η προσομοίωση να περιγράφει όσο το δυνατόν πιστότερα τις αντίστοιχες αποδόσεις που παρατηρούνται στα πραγματικά δεδομένα. Οι παράγοντες αυτοί παρέχονται απευθείας στα χρησιμοποιούμενα Open Data ως μεταβλητές ανά γεγονός.

Στην παρούσα ανάλυση χρησιμοποιείται ο παράγοντας `ScaleFactor_MUON`, ο οποίος διορθώνει τις διαφορές μεταξύ δεδομένων και προσομοίωσης που σχετίζονται με την ανακατασκευή και την επιλογή των μιονίων. Η εφαρμογή της διόρθωσης αυτής είναι απαραίτητη, καθώς η επιλογή των γεγονότων βασίζεται στην ανακατασκευή του ζεύγους μιονίων που σχηματίζει τον υποψήφιο $H \to \mu^+\mu^-$.

Αντίστοιχα, χρησιμοποιείται ο παράγοντας `ScaleFactor_LepTRIGGER`, προκειμένου να διορθωθούν οι διαφορές στην απόδοση του συστήματος trigger μεταξύ πραγματικών και προσομοιωμένων γεγονότων. Η διόρθωση αυτή συνδέεται άμεσα με την επιλογή γεγονότων της ανάλυσης, στην οποία απαιτείται τουλάχιστον ένα από τα δύο μιόνια του υποψηφίου να είναι συμβατό με το αντίστοιχο trigger. Δεν εφαρμόζεται επιπλέον ο παράγοντας `ScaleFactor_MuTRIGGER`, ώστε να μην πραγματοποιείται διπλή διόρθωση της ίδιας απόδοσης.

Επιπλέον εφαρμόζεται ο παράγοντας `ScaleFactor_PILEUP`. Στα πραγματικά δεδομένα περισσότερες από μία αλληλεπιδράσεις πρωτονίου–πρωτονίου μπορούν να πραγματοποιούνται στην ίδια ή σε γειτονικές διασταυρώσεις δεσμών, φαινόμενο γνωστό ως pile-up. Η κατανομή του pile-up στην προσομοίωση δεν ταυτίζεται απαραίτητα με εκείνη των πραγματικών δεδομένων και επομένως τα προσομοιωμένα γεγονότα επαναστα θμίζονται ώστε να διορθωθεί η σχετική διαφορά.

Το διορθωμένο βάρος ενός γεγονότος γράφεται επομένως ως

$$w_i^{\mathrm{corr}} = w_i^{\mathrm{MC}}\, SF_i^{\mu}\, SF_i^{\mathrm{trig}}\, SF_i^{\mathrm{PU}}, \tag{55}$$

όπου οι παραπάνω ποσότητες αντιστοιχούν στις μεταβλητές `mcWeight`, `ScaleFactor_MUON`, `ScaleFactor_LepTRIGGER`, `ScaleFactor_PILEUP`.

4.3.2 Κανονικοποίηση των δειγμάτων

Η προηγούμενη διαδικασία διορθώνει το σχετικό βάρος των γεγονότων εντός ενός προσομοιωμένου δείγματος, δεν λαμβάνει όμως από μόνη της υπόψη τη διαφορετική πιθανότητα παραγωγής των φυσικών διεργασιών. Για παράδειγμα, τα δείγματα σήματος που αντιστοιχούν στους διαφορετικούς μηχανισμούς παραγωγής του H δεν έχουν την ίδια ενεργό διατομή, ενώ το ίδιο ισχύει για τα επιμέρους δείγματα του υποβάθρου. Συνεπώς, ο αριθμός των προσομοιωμένων γεγονότων που περιέχεται σε ένα αρχείο δεν μπορεί να χρησιμοποιηθεί ως μέτρο της φυσικής συχνότητας εμφάνισης της αντίστοιχης διεργασίας.

Για κάθε δείγμα s ορίζεται για τον λόγο αυτό ένας παράγοντας κανονικοποίησης

$$C_s = \frac{\sigma_s \, k_s \, \epsilon_s}{\sum_{i \in s} w_i^{\mathrm{MC}}}, \tag{56}$$

όπου σ_s είναι η ενεργός διατομή παραγωγής της διεργασίας, k_s ο αντίστοιχος διορθωτικός παράγοντας ανώτερης τάξης, ϵ_s η απόδοση του φίλτρου που χρησιμοποιήθηκε κατά την παραγωγή του δείγματος και $\sum_{i \in s} w_i^{\mathrm{MC}}$ το άθροισμα των βαρών της γεννήτριας για ολόκληρο το αντίστοιχο δείγμα.

Οι ποσότητες αυτές λαμβάνονται από τις πληροφορίες μεταδεδομένων (metadata) των δειγμάτων Monte Carlo των ATLAS Open Data. Ιδιαίτερα σημαντικό είναι ότι το άθροισμα $\sum w_i^{\mathrm{MC}}$ αφορά το συνολικό παραγόμενο δείγμα και όχι μόνο τα γεγονότα που επιβιώνουν μετά την επιλογή ή ανήκουν σε μία συγκεκριμένη κατηγορία πιδάκων. Κατά συνέπεια, ο παράγοντας κανονικοποίησης ενός συγκεκριμένου δείγματος παραμένει ο ίδιος στις κατηγορίες $n_j = 0$, $n_j = 1$ και $n_j \geq 2$.

Το τελικό βάρος που χρησιμοποιείται για κάθε γεγονός της ανάλυσης δίνεται επομένως από

$$w_i = w_i^{\mathrm{MC}} \, SF_i^{\mu} \, SF_i^{\mathrm{trig}} \, SF_i^{\mathrm{PU}} \, \frac{\sigma_s \, k_s \, \epsilon_s}{\sum_{j \in s} w_j^{\mathrm{MC}}}. \tag{57}$$

Με αυτόν τον τρόπο διατηρείται τόσο η σχετική στάθμιση των γεγονότων στο εσωτερικό κάθε δείγματος όσο και η σωστή σχετική κανονικοποίηση μεταξύ των διαφορετικών διεργασιών σήματος και υποβάθρου. Τα βάρη αυτά χρησιμοποιούνται στη συνέχεια κατά την κατασκευή των συνόλων δεδομένων και την εκπαίδευση των τεχνικών πολυμεταβλητής ανάλυσης.

4.4 Μεταβλητές της Ανάλυσης

4.4.1 Μεταβλητές Εισόδου

Για την εφαρμογή των τεχνικών κατηγοριοποίησης επιλέχθηκε ένα σύνολο κινηματικών μεταβλητών, οι οποίες χρησιμοποιούνται ως μεταβλητές εισόδου των αντίστοιχων μοντέλων. Στόχος της επιλογής τους είναι η αξιοποίηση διαφορών στα κινηματικά χαρακτηριστικά μεταξύ των γεγονότων σήματος και υποβάθρου.

Οι μεταβλητές εισόδου διαφοροποιούνται ανάλογα με την πολλαπλότητα των πιδάκων. Στην κατηγορία $n_j = 0$ χρησιμοποιούνται αποκλειστικά μεταβλητές που περιγράφουν το διμιονικό σύστημα, ενώ στις κατηγορίες $n_j = 1$ και $n_j \geq 2$ προστίθενται μεταβλητές που περιγράφουν την κινηματική των πιδάκων και τη σχέση τους με το διμιονικό σύστημα.

Κοινές και στις τρεις κατηγορίες είναι οι μεταβλητές $p_T^{\mu\mu}$, $Y_{\mu\mu}$ και $\cos\theta^*$, οι οποίες περιγράφουν την κινηματική και τη γωνιακή δομή του συστήματος των δύο μιονίων.

Κατηγορία $n_j = 0$

$\mathbf{p_T^{\mu\mu}}$: Εγκάρσια ορμή του διμιονικού συστήματος

Εκφράζει το μέτρο της συνολικής εγκάρσιας ορμής του συστήματος των δύο επιλεγμένων μιονίων. Ορίζεται από το διανυσματικό άθροισμα των εγκάρσιων ορμών τους,

$$\vec{p}_T^{\mu\mu} = \vec{p}_T^{\mu_1} + \vec{p}_T^{\mu_2}, \tag{58}$$

και επομένως

$$p_T^{\mu\mu} = |\vec{p}_T^{\mu_1} + \vec{p}_T^{\mu_2}|\,. \tag{59}$$

Η μεταβλητή αυτή χαρακτηρίζει την εγκάρσια κινηματική του διμιονικού συστήματος και μπορεί να παρουσιάζει διαφορετική κατανομή για τα γεγονότα σήματος και υποβάθρου.

$\mathbf{Y_{\mu\mu}}$: Ωκύτητα του διμιονικού συστήματος

Περιγράφει την κίνηση του διμιονικού συστήματος κατά μήκος του άξονα της δέσμης και ορίζεται ως

$$Y_{\mu\mu} = \frac{1}{2}\ln\left(\frac{E_{\mu\mu} + p_{z,\mu\mu}}{E_{\mu\mu} - p_{z,\mu\mu}}\right), \tag{60}$$

όπου $E_{\mu\mu}$ και $p_{z,\mu\mu}$ είναι αντίστοιχα η ενέργεια και η διαμήκης ορμή του συστήματος των δύο μιονίων. Η μεταβλητή παρέχει πληροφορία για τη διαμήκη κινηματική του γεγονότος.

$\cos\theta^*$: Γωνία Collins-Soper

Περιγράφει τη γωνιακή κατανομή των δύο μιονίων στο σύστημα Collins–Soper[5] και παρέχει πληροφορία για τη γωνιακή δομή της διαδικασίας.. Για κάθε μιόνιο ορίζονται οι ποσότητες

$$P_i^{\pm} = E_i \pm p_{z,i}, \tag{61}$$

ενώ η μεταβλητή υπολογίζεται ως

$$\cos\theta^* = \frac{2\left(P_1^+ P_2^- - P_1^- P_2^+\right)}{\sqrt{m_{\mu\mu}^2\left[m_{\mu\mu}^2 + (p_T^{\mu\mu})^2\right]}}\,\mathrm{sign}(p_z^{\mu\mu}). \tag{62}$$

[5]Το σύστημα Collins–Soper είναι ένα σύστημα αναφοράς που χρησιμοποιείται για τη μελέτη της γωνιακής κατανομής των προϊόντων διάσπασης ενός συστήματος δύο λεπτονίων σε συγκρούσεις αδρονίων. Ο άξονάς του ορίζεται στο σύστημα ηρεμίας του διλεπτονικού συστήματος με βάση τις διευθύνσεις των δύο αρχικών δεσμών.

Η χρήση της γωνιακής αυτής πληροφορίας προσθέτει πληροφορία διαφορετική από εκείνη που περιέχουν αποκλειστικά οι μεταβλητές ορμής του διμιονικού συστήματος.

Κατηγορία $n_j = 1$

Στην κατηγορία $n_j = 1$, η παρουσία ενός πίδακα επιτρέπει την αξιοποίηση επιπλέον κινηματικής πληροφορίας. Για τον λόγο αυτό, στις τρεις κοινές μεταβλητές του διμιονικού συστήματος προστέθηκαν η εγκάρσια ορμή $p_T^{j_1}$ και η ψευδοωκύτητα η_{j_1} του πίδακα, καθώς και η αζιμουθιακή γωνιακή διαφορά $\Delta\phi_{\mu\mu,j_1}$ μεταξύ του πίδακα και του διμιονικού συστήματος.

$p_T^{j_1}$: Εγκάρσια ορμή του πίδακα

Η μεταβλητή $p_T^{j_1}$ αντιστοιχεί στην εγκάρσια ορμή του πίδακα του γεγονότος και χαρακτηρίζει την εγκάρσια κινηματική του. Η πληροφορία αυτή μπορεί να αξιοποιηθεί για τη διάκριση μεταξύ σήματος και υποβάθρου, καθώς η κινηματική των πιδάκων σχετίζεται με τον μηχανισμό παραγωγής και τη συνολική τοπολογία του γεγονότος.

η_{j_1}: Ψευδοωκύτητα του πίδακα

Η μεταβλητή η_{j_1} περιγράφει τη διεύθυνση του πίδακα ως προς τον άξονα της δέσμης. Η ψευδοωκύτητα ορίζεται μέσω της πολικής γωνίας θ όπως είδαμε στην ενότητα 3.6. Η μεταβλητή παρέχει πληροφορία για τη χωρική κατανομή του πίδακα και επομένως για την τοπολογία του γεγονότος.

$\Delta\phi_{\mu\mu,j_1}$: Αζιμουθιακή γωνιακή διαφορά

Η μεταβλητή $\Delta\phi_{\mu\mu,j_1}$ εκφράζει την αζιμουθιακή γωνιακή διαφορά μεταξύ του διμιονικού συστήματος και του πίδακα. Στην παρούσα ανάλυση χρησιμοποιείται η απόλυτη γωνιακή διαφορά, περιορισμένη στην περιοχή

$$0 \leq \Delta\phi_{\mu\mu,j_1} \leq \pi. \tag{63}$$

Η μεταβλητή περιγράφει τη σχετική γεωμετρική διάταξη του πίδακα και του διμιονικού συστήματος στο εγκάρσιο επίπεδο.

Κατηγορία $n_j \geq 2$

Στην κατηγορία $n_j \geq 2$, η παρουσία τουλάχιστον δύο πιδάκων επιτρέπει την αξιοποίηση περισσότερων πληροφοριών σχετικά με την κινηματική και την τοπολογία του γεγονότος. Εκτός από τις τρεις κοινές μεταβλητές του διμιονικού συστήματος, χρησιμοποιήθηκαν μεταβλητές που περιγράφουν τους δύο πίδακες με τη μεγαλύτερη εγκάρσια ορμή, το σύστημα που αυτοί σχηματίζουν, καθώς και τη συνολική δραστηριότητα του γεγονότος.

Οι δύο πίδακες με τη μεγαλύτερη εγκάρσια ορμή συμβολίζονται ως j_1 και j_2, με

$$p_T^{j_1} > p_T^{j_2}. \tag{64}$$

Μεταβλητές των δύο πρώτων πιδάκων

Για καθέναν από τους δύο πίδακες χρησιμοποιήθηκαν η εγκάρσια ορμή η ψευδοωκύτητά του, καθώς και η αζιμουθιακή γωνιακή διαφορά του από το διμιονικό σύστημα,

$$p_T^{j_1}, \quad \eta_{j_1}, \qquad p_T^{j_2}, \quad \eta_{j_2}, \qquad \Delta\phi_{j_1,\mu\mu}, \qquad \Delta\phi_{j_2,\mu\mu}. \tag{65}$$

Οι μεταβλητές αυτές περιγράφουν την κινηματική των δύο πιδάκων και τη σχετική γεωμετρική τους διάταξη ως προς το σύστημα των δύο μιονίων.

Μεταβλητές του συστήματος δύο πιδάκων

Οι δύο πίδακες j_1 και j_2 χρησιμοποιήθηκαν επίσης για την κατασκευή του συστήματος δύο πιδάκων. Το τετραδιάνυσμά του προκύπτει από το άθροισμα των τετραδιανυσμάτων των δύο πιδάκων,

$$p_{jj}^{\mu} = p_{j_1}^{\mu} + p_{j_2}^{\mu}. \tag{66}$$

Από το σύστημα δύο πιδάκων χρησιμοποιήθηκαν η εγκάρσια ορμή p_T^{jj}, η ωκύτητα Y_{jj} και η αναλλοίωτη μάζα m_{jj}, η οποία δίνεται από

$$m_{jj}^2 = \left(p_{j_1}^{\mu} + p_{j_2}^{\mu}\right)^2. \tag{67}$$

Επιπλέον, χρησιμοποιήθηκε η αζιμουθιακή γωνιακή διαφορά $\Delta\phi_{jj,\mu\mu}$ μεταξύ του συστήματος δύο πιδάκων και του διμιονικού συστήματος, η οποία περιγράφει τη σχετική τους διάταξη στο εγκάρσιο επίπεδο.

p_T^{miss}: Ελλείπουσα εγκάρσια ορμή

Η ελλείπουσα εγκάρσια ορμή p_T^{miss} αποτελεί μέτρο της ανισορροπίας της ορμής στο εγκάρσιο επίπεδο του γεγονότος. Η μεταβλητή αυτή παρέχει πρόσθετη πληροφορία για τη συνολική κινηματική και τη δραστηριότητα του γεγονότος.

H_T: Βαθμωτό άθροισμα των εγκάρσιων ορμών των πιδάκων

Η μεταβλητή H_T χαρακτηρίζει τη συνολική δραστηριότητα των πιδάκων του γεγονότος και ορίζεται ως το βαθμωτό άθροισμα των εγκάρσιων ορμών όλων των αποθηκευμένων πιδάκων,

$$H_T = \sum_{i=1}^{n_j} p_T^{j_i}. \tag{68}$$

Σε αντίθεση με τις μεταβλητές του συστήματος δύο πιδάκων, οι οποίες κατασκευάζονται από τους δύο πίδακες με τη μεγαλύτερη εγκάρσια ορμή, στον υπολογισμό του H_T συμμετέχουν όλοι οι πίδακες του γεγονότος.

Οι μεταβλητές εισόδου που χρησιμοποιήθηκαν συνοψίζονται στον Πίνακα 7.

Πίνακας 7: Οι μεταβλητές εισόδου που χρησιμοποιήθηκαν σε κάθε κατηγορία πολλαπλότητας πιδάκων.

	$n_j = 0$	$n_j = 1$	$n_j \geq 2$
Κοινές	$p_T^{\mu\mu}$, $Y_{\mu\mu}$, $\cos\theta^*$		
$n_j \geq 1$	–	$p_T^{j_1}$, η_{j_1}, $\Delta\phi_{j_1,\mu\mu}$	
$n_j \geq 2$	–	–	$p_T^{j_2}$, η_{j_2}, $\Delta\phi_{j_2,\mu\mu}$ p_T^{jj}, Y_{jj}, $\Delta\phi_{jj,\mu\mu}$, m_{jj} p_T^{miss}, H_T
Σύνολο	**3**	**6**	**15**

4.4.2 Μεταβλητές Παρακολούθησης

Εκτός από τις μεταβλητές εισόδου (input variables), στα τελικά σύνολα δεδομένων διατηρήθηκαν επιπλέον πληροφορίες για κάθε γεγονός, οι οποίες δεν χρησιμοποιήθηκαν ως είσοδοι στις τεχνικές κατηγοριοποίησης. Οι μεταβλητές αυτές (spectator variables) χρησιμοποιούνται για την παρακολούθηση, την ταυτοποίηση και τον έλεγχο των γεγονότων κατά τη διάρκεια της εκπαίδευσης.

$m_{\mu\mu}$: Αναλλοίωτη μάζα του διμιονικού συστήματος

Η μεταβλητή $m_{\mu\mu}$ αντιστοιχεί στην αναλλοίωτη μάζα του επιλεγμένου ζεύγους μιονίων και υπολογίζεται από το τετραδιάνυσμα του διμιονικού συστήματος,

$$m_{\mu\mu}^2 = \left(p_{\mu_1}^\alpha + p_{\mu_2}^\alpha\right)^2. \tag{69}$$

Η ποσότητα αυτή χρησιμοποιήθηκε κατά την επιλογή των γεγονότων, μέσω της απαίτησης

$$120 < m_{\mu\mu} < 130 \text{ GeV}, \tag{70}$$

και διατηρήθηκε στα τελικά σύνολα δεδομένων για σκοπούς παρακολούθησης και ελέγχου. Η εξαίρεσή της από τις μεταβλητές εισόδου επιτρέπει η κατηγοριοποίηση να βασίζεται στις υπόλοιπες κινηματικές και τοπολογικές ιδιότητες των γεγονότων και όχι άμεσα στη θέση τους ως προς την κορυφή της αναλλοίωτης μάζας του σήματος.

`eventNumber`: Αριθμός γεγονότος

Η μεταβλητή `eventNumber` αποτελεί τον αναγνωριστικό αριθμό του εκάστοτε γεγονότος. Διατηρήθηκε στα τελικά σύνολα δεδομένων ώστε να είναι δυνατή η αναγνώριση και η παρακολούθηση μεμονωμένων γεγονότων κατά την επεξεργασία και τον έλεγχο των δεδομένων.

`jet_n`: Πολλαπλότητα πιδάκων

Η μεταβλητή `jet_n` περιέχει τον αριθμό των αποθηκευμένων πιδάκων του γεγονότος

και χρησιμοποιήθηκε, όπως περιγράφηκε προηγουμένως, για τον διαχωρισμό των γεγονότων στις κατηγορίες $n_j = 0$, $n_j = 1$ και $n_j \geq 2$. Διατηρήθηκε στα τελικά σύνολα δεδομένων για την παρακολούθηση της κατηγορίας στην οποία ανήκει κάθε γεγονός.

`channelNumber`: Αναγνωριστικό δείγματος

Η μεταβλητή `channelNumber` προσδιορίζει το προσομοιωμένο δείγμα από το οποίο προέρχεται κάθε γεγονός μέσω του αντίστοιχου DSID. Η πληροφορία αυτή διατηρήθηκε ώστε να είναι δυνατή η διάκριση της προέλευσης των γεγονότων από τα διαφορετικά δείγματα Monte Carlo, καθώς και η εφαρμογή των αντίστοιχων παραμέτρων κανονικοποίησης.

Πίνακας 8: Μεταβλητές παρακολούθησης που διατηρήθηκαν στα τελικά σύνολα δεδομένων.

Μεταβλητή	**Χρήση**
$m_{\mu\mu}$	Αναλλοίωτη μάζα του επιλεγμένου διμιονικού συστήματος
`eventNumber`	Αναγνώριση και παρακολούθηση του γεγονότος
`jet_n`	Προσδιορισμός της κατηγορίας πολλαπλότητας πιδάκων
`channelNumber`	Προσδιορισμός του δείγματος Monte Carlo

4.5 Κατασκευή των τελικών συνόλων δεδομένων

Μετά την εφαρμογή των κριτηρίων επιλογής και τον διαχωρισμό των γεγονότων με βάση την πολλαπλότητα των πιδάκων, κατασκευάστηκαν τα τελικά σύνολα δεδομένων που χρησιμοποιήθηκαν για την εκπαίδευση και την αξιολόγηση των τεχνικών κατηγοριοποίησης.

Η κατασκευή νέων συνόλων δεδομένων κρίθηκε απαραίτητη ώστε από τα αρχικά αρχεία προσομοίωσης να διατηρηθούν μόνο τα γεγονότα που ικανοποιούν τα κριτήρια της ανάλυσης και μόνο οι πληροφορίες που απαιτούνται στα επόμενα στάδια. Με τον τρόπο αυτό, τα αρχικά δείγματα μετατράπηκαν σε μικρότερα και ομοιόμορφα σύνολα δεδομένων, κατάλληλα για την εφαρμογή των τεχνικών μηχανικής μάθησης.

Για κάθε επιλεγμένο γεγονός αποθηκεύτηκαν οι μεταβλητές εισόδου που αντιστοιχούν στην εκάστοτε κατηγορία πολλαπλότητας πιδάκων, οι μεταβλητές παρακολούθησης, καθώς και το αντίστοιχο βάρος του γεγονότος. Τα γεγονότα σήματος από τους διαφορετικούς μηχανισμούς παραγωγής του μποζονίου Higgs συνενώθηκαν σε ένα κοινό σύνολο σήματος, ενώ αντίστοιχα τα τρία δείγματα της διαδικασίας Drell–Yan συνενώθηκαν σε ένα κοινό σύνολο υποβάθρου.

Η διαδικασία πραγματοποιήθηκε ανεξάρτητα για τις τρεις κατηγορίες $n_j = 0$, $n_j = 1$ και $n_j \geq 2$. Έτσι, για κάθε κατηγορία κατασκευάστηκε ένα σύνολο γεγονότων σήματος και ένα σύνολο γεγονότων υποβάθρου, τα οποία χρησιμοποιήθηκαν στη συνέχεια ως είσοδος στο TMVA.

Στον Πίνακα 9 αποτυπώνονται ο αριθμός των γεγονότων σήματος που διατηρήθηκαν για τους τρεις μηχανισμούς παραγωγής σε κάθε κατηγορία πολλαπλότητας πιδάκων.

Πίνακας 9: Αριθμός επιλεγμένων γεγονότων σήματος ανά μηχανισμό παραγωγής και κατηγορία πολλαπλότητας πιδάκων.

Κατηγορία	ggH	$ggZH$	VBF	Σύνολο
$n_j = 0$	34 619	2 559	20 664	57 842
$n_j = 1$	24 592	4 491	23 890	52 973
$n_j \geq 2$	15 985	11 461	15 742	43 188

Πίνακας 10: Αριθμός γεγονότων στα τελικά σύνολα δεδομένων για κάθε κατηγορία πολλαπλότητας πιδάκων.

Κατηγορία	Γεγονότα σήματος	Γεγονότα υποβάθρου	Σύνολο
$n_j = 0$	57 842	39 713	97 555
$n_j = 1$	52 973	72 040	125 013
$n_j \geq 2$	43 188	125 652	168 840
Σύνολο	**154 003**	**237 405**	**391 408**

Όπως φαίνεται στον Πίνακα 10, μετά την εφαρμογή των κριτηρίων επιλογής διατηρείται σημαντικός αριθμός γεγονότων τόσο για το σήμα όσο και για το υπόβαθρο και στις τρεις κατηγορίες πολλαπλότητας πιδάκων. Συνολικά, τα τελικά σύνολα περιλαμβάνουν 154 003 γεγονότα σήματος και 237 405 γεγονότα υποβάθρου. Το διαθέσιμο πλήθος γεγονότων επιτρέπει τον περαιτέρω διαχωρισμό των δεδομένων σε σύνολα εκπαίδευσης και ελέγχου και παρέχει επαρκή στατιστική βάση για την εκπαίδευση και τη συγκριτική αξιολόγηση των τεχνικών κατηγοριοποίησης που εξετάζονται στη συνέχεια.

5 Κατασκευή και Εκπαίδευση των Μοντέλων

Μετά την ολοκλήρωση της επιλογής των γεγονότων και την κατασκευή των τελικών συνόλων δεδομένων, ακολούθησε η εφαρμογή των τεχνικών κατηγοριοποίησης με στόχο τη διάκριση μεταξύ των γεγονότων σήματος $H \to \mu^+\mu^-$ και του υποβάθρου $Z/\gamma^* \to \mu^+\mu^-$.

Στην κατηγορία $n_j = 0$ εξετάστηκαν οι μέθοδοι Fisher, Likelihood, k-Nearest Neighbors (kNN), Multilayer Perceptron (MLP), Deep Neural Network (DNN) και Boosted Decision Trees (BDT).

Για τις κατηγορίες $n_j = 1$ και $n_j \geq 2$, όπου ο χώρος εισόδου περιλαμβάνει περισσότερες μεταβλητές και πολυπλοκότερες κινηματικές συσχετίσεις, η μελέτη επικεντρώθηκε στις μεθόδους DNN και BDT. Η επιλογή αυτή επιτρέπει τη μελέτη δύο ευέλικτων μη γραμμικών τεχνικών, διατηρώντας παράλληλα το εύρος της σύγκρισης διαχειρίσιμο.

Η διαδικασία κατασκευής των μοντέλων περιλάμβανε τον καθορισμό των υπερπαραμέτρων κάθε μεθόδου, τη βελτιστοποίηση επιλεγμένων υπερπαραμέτρων όπου αυτή κρίθηκε απαραίτητη και την εφαρμογή κοινής στρατηγικής διαχωρισμού, εκπαίδευσης και αξιολόγησης των δεδομένων.

5.1 Επιλογή Μεθόδων και Υπερπαραμέτρων

Οι μέθοδοι κατηγοριοποίησης που χρησιμοποιήθηκαν στην ανάλυση διαφέρουν ως προς τον αριθμό και το είδος των υπερπαραμέτρων που καθορίζουν τη δομή και τη διαδικασία εκπαίδευσής τους. Για τον λόγο αυτό, για κάθε μέθοδο καθορίστηκαν οι κατάλληλες ρυθμίσεις, ενώ η διαδικασία βελτιστοποίησης επικεντρώθηκε στις υπερπαραμέτρους που επηρεάζουν ουσιαστικά την πολυπλοκότητα και την απόδοση του αντίστοιχου μοντέλου.

Fisher

Για τη μέθοδο Fisher Linear Discriminant δεν πραγματοποιήθηκε βελτιστοποίηση υπερπαραμέτρων. Όπως εξηγήθηκε στην ενότητα 2.6 η μέθοδος κατασκευάζει έναν γραμμικό συνδυασμό των μεταβλητών εισόδου. Επομένως, δεν απαιτείται ο καθορισμός παραμέτρων που ελέγχουν τη δομή ή την πολυπλοκότητα του μοντέλου. Στην παρούσα ανάλυση χρησιμοποιήθηκε η βασική υλοποίηση της μεθόδου που παρέχεται από το TMVA, χωρίς εφαρμογή πρόσθετου μετασχηματισμού στις μεταβλητές εισόδου (`VarTransform=None`). Επομένως, η μέθοδος χρησιμοποιήθηκε ως ένα απλό γραμμικό μοντέλο αναφοράς με το οποίο θα συγκριθούν οι πιο σύνθετες τεχνικές.

Likelihood

Στη μέθοδο Likelihood, η εκτίμηση των συναρτήσεων πυκνότητας πιθανότητας των μεταβλητών εισόδου εξαρτάται από ορισμένες υπερπαραμέτρους που καθορίζουν τον τρόπο κατασκευής και εξομάλυνσης των αντίστοιχων κατανομών και συνοψίζονται στον Πίνακα 11. Για την παρεμβολή των συναρτήσεων πυκνότητας πιθανότητας χρησιμοποιήθηκε η επιλογή `Spline2`, ενώ η παράμετρος `NSmooth`, η οποία ελέγχει την εξομάλυνση των κατανομών, διατηρήθηκε σταθερή στην τιμή 1. Επιπλέον, δεν εφαρμόστηκε πρόσθετος μετασχηματισμός στις μεταβλητές εισόδου (`VarTransform=None`). Μία από τις σημαντικότερες είναι η `Nbins`, η οποία καθορίζει τον αριθμό των διαστημάτων που χρησιμοποιούνται για την κατασκευή των κατανομών. Στην υπερπαράμετρο αυτή πραγματοποιήθηκε βελτιστοποίηση, καθώς η επιλογή της τιμής της επηρεάζει την ανάλυση με την οποία περιγράφεται η μορφή των κατανομών: ένας μικρός αριθμός διαστημάτων μπορεί να οδηγήσει σε απώλεια πληροφορίας, ενώ ένας υπερβολικά μεγάλος αριθμός μπορεί να αυξήσει την ευαισθησία σε στατιστικές διακυμάνσεις. Εξετάστηκαν οι τιμές 50, 100, 150, 200, 250 και 300, ενώ οι υπόλοιπες ρυθμίσεις της μεθόδου διατηρήθηκαν σταθερές.

Τέλος, η `NAvEvtPerBin` τέθηκε 0, καθώς καθορίζει τον μέσο όρο γεγονότων ανά bin όταν το TMVA επιλέγει αυτόματο binning για την κατασκευή των συναρτήσεων πυκνότητας πιθανότητας. Στην προκειμένη περίπτωση, όμως το binning έγινε μέσω της επιλογής των τιμών της υπερπαραμέτρου `Nbins`.

Πίνακας 11: Υπερπαράμετροι της μεθόδου Likelihood για την κατηγορία $n_j = 0$.

Υπερπαράμετρος	**Τιμές**
Nbins	50, 100, 150, 200, 250, 300
PDFInterpol	Spline2
NSmooth	1
NAvEvtPerBin	0
VarTransform	None

kNN

Στη μέθοδο k-Nearest Neighbors (kNN), η βασικότερη υπερπαράμετρος είναι ο αριθμός k των πλησιέστερων γειτόνων, η οποία καθορίζεται στο TMVA μέσω της παραμέτρου `nkNN`. Για αυτό επιλέχθηκε για την βελτιστοποίηση και εξετάστηκαν οι τιμές 50, 100, 150, 200, 250 και 300, με τις υπόλοιπες ρυθμίσεις της μεθόδου να διατηρούνται σταθερές.

Στην παρούσα ανάλυση η χρήση συνάρτησης πυρήνα απενεργοποιήθηκε μέσω της επιλογής `UseKernel=False`, κάτι που σημαίνει ότι δεν εφαρμόστηκε στάθμιση των γειτονικών γεγονότων μέσω πυρήνα. Η `ScaleFrac=0.8` καθορίζει το τμήμα του εύρους των κατανομών που χρησιμοποιείται για τον υπολογισμό της κλιμάκωσης των μεταβλητών. Επιπλέον, δεν εφαρμόστηκε περικοπή του συνόλου των γειτόνων (`Trim=False`) και ενεργοποιήθηκε η χρήση βαρών μέσω της επιλογής `UseWeight=True`.

Στον Πίνακα 12 παρουσιάζεται συνοπτικά το σύνολο των υπερπαραμέτρων .

Πίνακας 12: Υπερπαράμετροι της μεθόδου k-Nearest Neighbors (kNN) για την κατηγορία $n_j = 0$.

Υπερπαράμετρος	**Τιμές**
nkNN	50, 100, 150, 200, 250, 300
ScaleFrac	0.8
UseKernel	False
UseWeight	True
Trim	False

MLP

Στη μέθοδο Multilayer Perceptron (MLP), η αρχιτεκτονική του νευρωνικού δικτύου αποτελεί βασικό παράγοντα που καθορίζει την πολυπλοκότητα και την ικανότητά του να περιγράψει τις συσχετίσεις μεταξύ των μεταβλητών εισόδου. Στην παρούσα ανάλυση χρησιμοποιήθηκε ένα κρυφό επίπεδο, ενώ ως συνάρτηση ενεργοποίησης των νευρώνων επιλέχθηκε η υπερβολική εφαπτομένη (`tanh`). Η εκπαίδευση πραγματοποιήθηκε με τη μέθοδο `Backpropagation` για 600 κύκλους εκπαίδευσης (`NCycles`). Επιπλέον, εφαρμόστηκε κανονικοποίηση των μεταβλητών εισόδου μέσω της επιλογής `VarTransform=N`.

Για τη μέθοδο MLP πραγματοποιήθηκε βελτιστοποίηση του αριθμού των νευρώνων του κρυφού επιπέδου. Συγκεκριμένα, εξετάστηκαν οι τιμές 4, 6, 8, 10, 12, 14, 16, 18 και 20, ενώ οι υπόλοιπες υπερπαράμετροι του δικτύου διατηρήθηκαν σταθερές. Με τον τρόπο αυτό μελετήθηκε η επίδραση της πολυπλοκότητας του δικτύου στην απόδοση του ταξινομητή.

Στον Πίνακα 13 κατάγραφονται συνοπτικά όλες οι υπερπαράμετροι.

Πίνακας 13: Υπερπαράμετροι της μεθόδου Multilayer Perceptron (MLP) για την κατηγορία $n_j = 0$.

Υπερπαράμετρος	**Τιμές**
HiddenLayers	4, 6, 8, 10, 12, 14, 16, 18, 20
NeuronType	tanh
NCycles	600
TrainingMethod	BP
VarTransform	N
UseRegulator	False

DNN

Στην περίπτωση του Deep Neural Network (DNN), η αρχιτεκτονική του δικτύου αποτέλεσε τη βασική υπερπαράμετρο που εξετάστηκε. Σε αντίθεση με τις προηγούμενες μεθόδους, η DNN εφαρμόστηκε και στις τρεις κατηγορίες γεγονότων, $n_j = 0$, $n_j = 1$ και $n_j \geq 2$, με διαφορετικές αρχιτεκτονικές να εξετάζονται σε κάθε περίπτωση.

Για την κατηγορία $n_j = 0$, η μελέτη επικεντρώθηκε στο βάθος του δικτύου. Ο αριθμός των νευρώνων διατηρήθηκε σταθερός στους 6 ανά κρυφό επίπεδο, ενώ εξετάστηκαν αρχιτεκτονικές με 1 έως 6 κρυφά επίπεδα, με σκοπό να ελεγχθεί κατά πόσο η αύξηση των κρυφών επιπέδων οδηγεί σε καλύτερο διαχωρισμό.

Για τις κατηγορίες $n_j = 1$ και $n_j \geq 2$, όπου χρησιμοποιείται μεγαλύτερος αριθμός μεταβλητών εισόδου, εξετάστηκαν διαφορετικοί συνδυασμοί τόσο του αριθμού των κρυφών επιπέδων όσο και του αριθμού των νευρώνων σε κάθε επίπεδο, έτσι ώστε να ελεγχθεί εάν η αύξηση της πολυπλοκότητας των δεδομένων απαιτεί και αύξηση της πολυπλοκότητας του μοντέλου. Στον Πίνακα 14 απεικονίζονται οι τρεις αρχιτεκτονικές.

Πίνακας 14: Αρχιτεκτονικές της μεθόδου Deep Neural Network (DNN) που εξετάστηκαν στις τρεις κατηγορίες γεγονότων.

Κατηγορία	**Αρχιτεκτονικές κρυφών επιπέδων**
$n_j = 0$	6, 6−6, 6−6−6, 6−6−6−6, 6−6−6−6−6, 6−6−6−6−6−6
$n_j = 1$	6−6, 12−12, 24−12, 6−6−6, 12−12−12, 24−24−24, 24−12−6, 12−12−12−12, 36−24−12−6
$n_j \geq 2$	15−15, 30−15, 30−30, 15−15−15, 30−30−30, 60−30−15, 15−15−15−15, 30−30−30−30, 75−60−30−15

Οι υπόλοιπες βασικές υπερπαράμετροι της εκπαίδευσης, οι οποίες απεικονίζονται συνοπτικά τον Πίνακα 15, διατηρήθηκαν κοινές στις τρεις κατηγορίες. Ως συνάρτηση ενεργοποίησης των κρυφών επιπέδων χρησιμοποιήθηκε η `tanh`, ενώ η εκπαίδευση πραγματοποιήθηκε με τον βελτιστοποιητή `ADAM` (`Optimizer`), με ρυθμό μάθησης 10^{-3} (`LearningRate`) και μέγεθος παρτίδας 100 (`BatchSize`). Ο μέγιστος αριθμός εποχών εκπαίδευσης ορίστηκε

στις 1000 (`MaxEpochs`), ενώ ως συνάρτηση σφάλματος χρησιμοποιήθηκε η `Cross-Entropy` (`ErrorStrategy`). Τα αρχικά βάρη του δικτύου προσδιορίστηκαν με τη μέθοδο `Xavier Uniform` (`WeightInitialization`) και εφαρμόστηκε κανονικοποίηση των μεταβλητών εισόδου.

Πίνακας 15: Κοινές υπερπαράμετροι της μεθόδου Deep Neural Network (DNN) για τις τρεις κατηγορίες γεγονότων.

Υπερπαράμετρος	**Τιμή**
Activation	tanh
LearningRate	10^{-3}
Momentum	0.9
BatchSize	100
MaxEpochs	1000
ConvergenceSteps	50
Optimizer	ADAM
ErrorStrategy	CROSSENTROPY
WeightInitialization	XAVIERUNIFORM
WeightDecay	0.0
Regularization	None
VarTransform	N

BDT

Στην περίπτωση των Boosted Decision Trees (BDT), η βασική υπερπαράμετρος που εξετάστηκε ήταν το μέγιστο βάθος των επιμέρους δέντρων (MaxDepth). Η παράμετρος αυτή καθορίζει τον μέγιστο αριθμό διαδοχικών διαχωρισμών που μπορούν να πραγματοποιηθούν σε κάθε δέντρο και, επομένως, επηρεάζει άμεσα την πολυπλοκότητα του μοντέλου.

Η μέθοδος εφαρμόστηκε και στις τρεις κατηγορίες γεγονότων. Για την κατηγορία $n_j = 0$ εξετάστηκαν τιμές της MaxDepth από 1 έως 6, ενώ για τις κατηγορίες $n_j = 1$ και $n_j \geq 2$ το εύρος επεκτάθηκε από 1 έως 8, όπως απεικονίζεται στον Πίνακα 16. Με τον τρόπο αυτό εξετάστηκε εάν η αύξηση της πολυπλοκότητας των δεδομένων στις κατηγορίες με πίδακες απαιτεί αντίστοιχα μεγαλύτερο βάθος των δέντρων για την αποτελεσματική διάκριση σήματος και υποβάθρου.

Πίνακας 16: Τιμές της υπερπαραμέτρου MaxDepth που εξετάστηκαν στις τρεις κατηγορίες γεγονότων.

Κατηγορία	**MaxDepth**
$n_j = 0$	1, 2, 3, 4, 5, 6
$n_j = 1$	1, 2, 3, 4, 5, 6, 7, 8
$n_j \geq 2$	1, 2, 3, 4, 5, 6, 7, 8

Οι υπόλοιπες υπερπαράμετροι της μεθόδου διατηρήθηκαν σταθερές και κοινές στις τρεις κατηγορίες γεγονότων και συνοψίζονται στον Πίνακα 17. Ο αριθμός των δέντρων απόφασης (`Ntrees`) διατηρήθηκε σταθερός στα 800 και η `MinNodeSize` η οποία καθορίζει το ελάχιστο μέγεθος ενός τελικού κόμβου σε σχέση με το δείγμα εκπαίδευσης, τέθηκε ίση με $2,5\%$.

Παράλληλα, η μέθοδος boosting που χρησιμοποιήθηκε ήταν η `AdaBoost` με `AdaBoostBeta` ίσο με 0.5 παράμετρος που ελέγχει την αναπροσαρμογή των βαρών των γεγονότων κατά τη διαδικασία του boosting. Επιπλέον, η `UseBaggedBoost` εισάγει επιπλέον bagging στην διαδικασία boosting με `BaggedSampleFraction` 0.5 που σημαίνει ότι για την εκπαίδευση κάθε δέντρου χρησιμοποιείται ένα τυχαία επιλεγμένο δείγμα που αντιστοιχεί στο 50% του διαθέσιμου δείγματος εκπαίδευσης. Τέλος, ως `SeparationType` χρησιμοποιήθηκε η `GiniIndex` και η παράμετρος `nCuts`, η οποία καθορίζει τον αριθμό των υποψήφιων σημείων διαχωρισμού που εξετάζονται κατά την αναζήτηση του κατάλληλου cut για κάθε μεταβλητή, τέθηκε ίση με 20.

Πίνακας 17: Κοινές υπερπαράμετροι της μεθόδου Boosted Decision Trees (BDT) για τις τρεις κατηγορίες γεγονότων.

Υπερπαράμετρος	**Τιμή**
NTrees	800
MinNodeSize	2.5%
BoostType	AdaBoost
AdaBoostBeta	0.5
UseBaggedBoost	True
BaggedSampleFraction	0.5
SeparationType	GiniIndex
nCuts	20

5.2 Κοινή Στρατηγική Εκπαίδευσης

Για την εκπαίδευση και την αξιολόγηση των διαφορετικών μεθόδων κατηγοριοποίησης ακολουθήθηκε κοινή στρατηγική, με στόχο τη διατήρηση μιας συνεπούς διαδικασίας μεταξύ των μοντέλων και τη δυνατότητα άμεσης σύγκρισης της απόδοσής τους. Η στρατηγική αυτή περιλαμβάνει τον διαχωρισμό των διαθέσιμων γεγονότων σε σύνολα εκπαίδευσης και ελέγχου και την εφαρμογή κοινών ρυθμίσεων κατά την προετοιμασία των δεδομένων στο TMVA.

Οι επιλογές που διαμόρφωσαν την κοινή στρατηγική εκπαίδευσης κατά την προετοιμασία και εκπαίδευση των μοντέλων συνοψίζονται στον Πίνακα 18.

Πίνακας 18: Κοινές επιλογές που χρησιμοποιήθηκαν κατά την προετοιμασία και εκπαίδευση των μοντέλων.

Επιλογή	**Τιμή**
Training set	80%
Test set	20%
`SplitMode`	Random
`SplitSeed`	12345
`NormMode`	NumEvents
Βάρος γεγονότος	`eventWeight`

Αρχικά, για κάθε κατηγορία γεγονότων, το συνολικό διαθέσιμο δείγμα διαχωρίστηκε σε δύο ανεξάρτητα υποσύνολα, με το 80% των γεγονότων να χρησιμοποιείται ως σύνολο εκπαίδευσης (training set) και το υπόλοιπο 20% ως σύνολο ελέγχου (test set). Η επιλογή του διαχωρισμού 80/20 επιτρέπει τη χρησιμοποίηση του μεγαλύτερου μέρους των διαθέσιμων

γεγονότων για την εκπαίδευση των μοντέλων, διατηρώντας παράλληλα ένα επαρκές ανεξάρτητο δείγμα το οποίο να χρησιμοποιείται για την τελική αξιολόγησή τους και οχι για την προσαρμογή των παραμέτρων του μοντέλου ή για την επιλογή των υπερπαραμέτρων.

Συνεπώς, η διαδικασία βελτιστοποίησης πραγματοποιείται χρησιμοποιώντας αποκλειστικά τα δεδομένα εκπαίδευσης, ενώ το σύνολο ελέγχου χρησιμοποιείται μετά την ολοκλήρωση της επιλογής του τελικού μοντέλου. Με αυτόν τον τρόπο περιορίζεται η πιθανότητα η τελική εκτίμηση της απόδοσης να επηρεαστεί από τη διαδικασία βελτιστοποίησης.

Ο διαχωρισμός πραγματοποιήθηκε τυχαία μέσω της επιλογής `SplitMode=Random`, ενώ χρησιμοποιήθηκε σταθερή τιμή `SplitSeed=12345`. Η χρήση σταθερού seed εξασφαλίζει ότι ο τυχαίος διαχωρισμός είναι αναπαραγωγίσιμος. Με αυτόν τον τρόπο, οι διαφορετικές μέθοδοι που εφαρμόζονται στην ίδια κατηγορία γεγονότων εκπαιδεύονται και αξιολογούνται πάνω στον ίδιο διαχωρισμό των δεδομένων, ώστε οι παρατηρούμενες διαφορές στην απόδοσή τους να μην οφείλονται σε διαφορετική τυχαία επιλογή γεγονότων για τα σύνολα εκπαίδευσης και ελέγχου.

Κατά την προετοιμασία των συνόλων δεδομένων χρησιμοποιήθηκε επίσης η επιλογή `NormMode=NumEvents` του TMVA. Με τη ρύθμιση αυτή πραγματοποιείται κανονικοποίηση των κλάσεων με βάση τον αριθμό των γεγονότων, ώστε η διαφορετική πληθικότητα των δειγμάτων σήματος και υποβάθρου να μην καθορίζει από μόνη της τη σχετική συνεισφορά τους κατά την εκπαίδευση.

Τέλος, ιδιαίτερη αντιμετώπιση απαιτούν τα γεγονότα με αρνητικά βάρη, τα οποία μπορούν να εμφανιστούν στα προσομοιωμένα δείγματα. Στις μεθόδους για τις οποίες απαιτείται σχετική ρύθμιση, χρησιμοποιήθηκε η επιλογή `IgnoreNegWeightsInTraining`, ώστε τα γεγονότα με αρνητικό βάρος να μη συμμετέχουν στη διαδικασία εκπαίδευσης. Η επιλογή αυτή αφορά ειδικά τη φάση εκπαίδευσης και δεν ισοδυναμεί με την αφαίρεση των συγκεκριμένων γεγονότων από τα αρχικά σύνολα δεδομένων.

5.3 Διαδικασία Βελτιστοποίησης Υπερπαραμέτρων

Για τις μεθόδους στις οποίες εξετάστηκαν διαφορετικές τιμές υπερπαραμέτρων, η επιλογή της τελικής διαμόρφωσης πραγματοποιήθηκε μέσω διασταυρούμενης επικύρωσης (cross-validation). Η διαδικασία εφαρμόστηκε αποκλειστικά στο σύνολο εκπαίδευσης, δηλαδή στο 80% των διαθέσιμων γεγονότων, διατηρώντας το ανεξάρτητο σύνολο ελέγχου εκτός της διαδικασίας βελτιστοποίησης.

Συγκεκριμένα, χρησιμοποιήθηκε διασταυρούμενη επικύρωση 5 τμημάτων (5-fold cross-validation). Το σύνολο εκπαίδευσης χωρίστηκε σε πέντε υποσύνολα (folds). Για κάθε υποψήφια τιμή της εξεταζόμενης υπερπαραμέτρου πραγματοποιήθηκαν πέντε διαδοχικές εκπαιδεύσεις. Σε κάθε επανάληψη, τέσσερα από τα πέντε τμήματα χρησιμοποιήθηκαν για την εκπαίδευση του μοντέλου, ενώ το εναπομείναν τμήμα χρησιμοποιήθηκε για την αξιολόγησή του. Η διαδικασία επαναλήφθηκε έτσι ώστε κάθε τμήμα να χρησιμοποιηθεί μία φορά για την αξιολόγηση.

Ο διαχωρισμός των γεγονότων στα επιμέρους τμήματα πραγματοποιήθηκε με τη μέθοδο `RandomStratified`, ώστε τα γεγονότα να κατανέμονται τυχαία στα διαφορετικά folds,

διατηρώντας παράλληλα την αντιπροσώπευση των κλάσεων κατά τον διαχωρισμό.

Για κάθε υποψήφια διαμόρφωση υπολογίστηκε η Area Under the ROC Curve (AUC) σε καθένα από τα πέντε folds. Στη συνέχεια υπολογίστηκαν η μέση τιμή της AUC,

$$\overline{\mathrm{AUC}} = \frac{1}{5}\sum_{k=1}^{5} \mathrm{AUC}_k,$$

καθώς και η τυπική απόκλιση των επιμέρους τιμών. Η μέση AUC χρησιμοποιήθηκε ως το βασικό κριτήριο για τη σύγκριση των διαφορετικών διαμορφώσεων. Η χρήση της AUC επιτρέπει την αξιολόγηση της ικανότητας διαχωρισμού σήματος και υποβάθρου σε ολόκληρο το εύρος της καμπύλης ROC, χωρίς να απαιτείται ο εκ των προτέρων καθορισμός ενός συγκεκριμένου κατωφλίου στην έξοδο του ταξινομητή.

Για κάθε μέθοδο επιλέχθηκε η διαμόρφωση που παρουσίασε την καλύτερη επίδοση κατά τη διαδικασία της διασταυρούμενης επικύρωσης. Μετά την ολοκλήρωση της επιλογής των υπερπαραμέτρων, το αντίστοιχο μοντέλο εκπαιδεύτηκε με την επιλεγμένη διαμόρφωση στο σύνολο των γεγονότων εκπαίδευσης (80%) και η τελική του απόδοση προσδιορίστηκε στο ανεξάρτητο σύνολο ελέγχου (20%), το οποίο δεν είχε συμμετάσχει στη διαδικασία επιλογής.

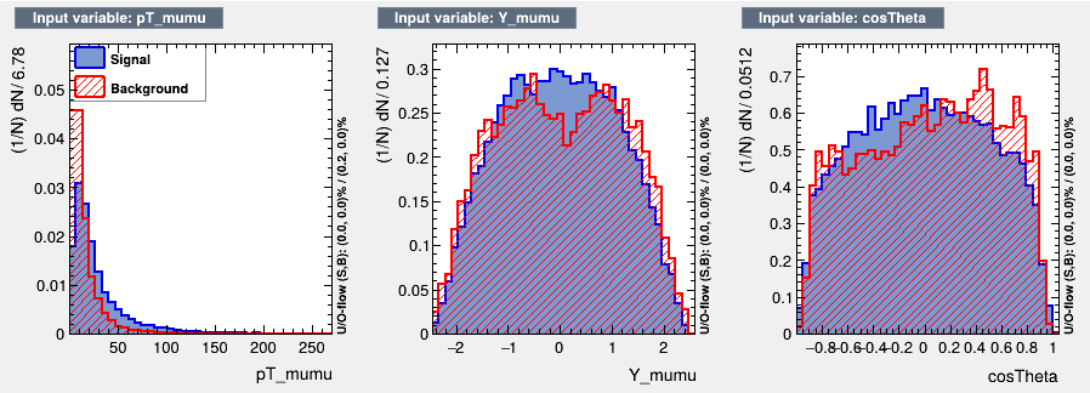
6 Αποτελέσματα

6.1 Κατανομές και Συσχετίσεις των Μεταβλητών Εισόδου

Πριν από την αξιολόγηση της απόδοσης των ταξινομητών, εξετάστηκαν οι κατανομές των μεταβλητών εισόδου για τα γεγονότα σήματος και υποβάθρου, καθώς και οι μεταξύ τους γραμμικές συσχετίσεις. Η μελέτη αυτή επιτρέπει μια πρώτη εκτίμηση της διαχωριστικής πληροφορίας που περιέχεται στις επιλεγμένες μεταβλητές και της πιθανής ύπαρξης ισχυρών συσχετίσεων μεταξύ τους.

$\mathbf{n_j = 0}$

Οι κανονικοποιημένες κατανομές τους για τα γεγονότα σήματος και υποβάθρου παρουσιάζονται στο Σχήμα 14.



Σχήμα 14: Κανονικοποιημένες κατανομές των μεταβλητών εισόδου $p_T^{\mu\mu}$, $Y_{\mu\mu}$ και $\cos\theta^*$ για τα γεγονότα σήματος και υποβάθρου στην κατηγορία $n_j = 0$.

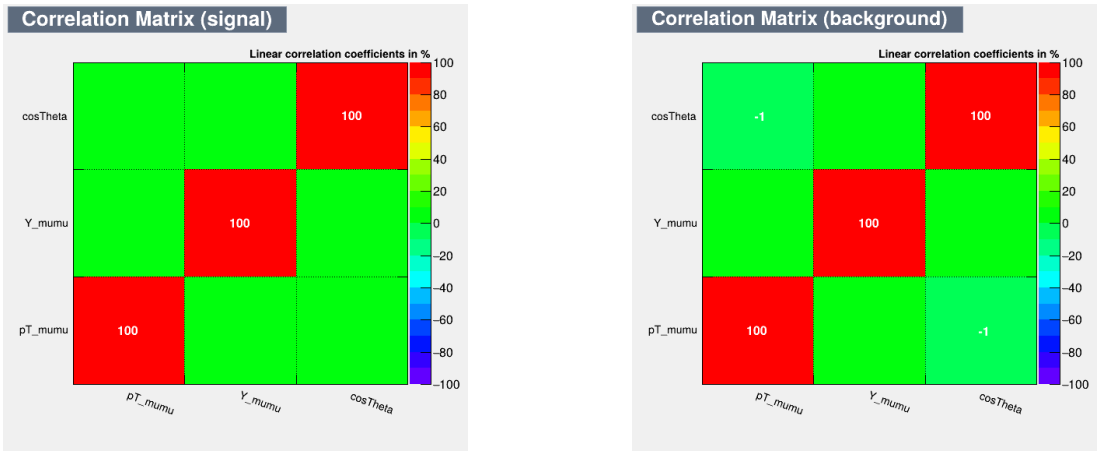
Οι κατανομές παρουσιάζουν διαφορές μεταξύ των γεγονότων σήματος και υποβάθρου, ιδιαίτερα στην κατανομή της $p_T^{\mu\mu}$ παρατηρείται διαφορετική συμπεριφορά των δύο δειγμάτων, με το σήμα να εμφανίζει σχετικά μεγαλύτερη συνεισφορά σε υψηλότερες τιμές εγκάρσιας ορμής.

Διαφορές παρατηρούνται επίσης στην κατανομή της $Y_{\mu\mu}$, όπου το σήμα εμφανίζεται περισσότερο συγκεντρωμένο στην κεντρική περιοχή, γύρω από $Y_{\mu\mu} = 0$, ενώ το υπόβαθρο παρουσιάζει σχετικά μεγαλύτερη συνεισφορά σε μεγαλύτερες απόλυτες τιμές της ωκύτητας.

Τέλος, στην κατανομή της $\cos\theta^*$ απεικονίζεται η ισότροπη φύση της διάσπασης του βαθμωτού μποζονίου Higgs, ενώ το υπόβαθρο παρουσιάζει διαφορετική εξάρτηση από τη γωνία θ^*.

Παρά τις παραπάνω διαφορές, παρατηρείται σημαντική επικάλυψη μεταξύ σήματος και υποβάθρου. Επομένως, καμία από τις μεταβλητές δεν παρέχει από μόνη της πλήρη διαχωρισμό των δύο κλάσεων, γεγονός που αιτιολογεί την ταυτόχρονη αξιοποίησή τους από τις πολυμεταβλητές μεθόδους.

Εκτός από τις επιμέρους κατανομές, εξετάστηκαν οι γραμμικές συσχετίσεις μεταξύ των μεταβλητών εισόδου. Οι αντίστοιχοι πίνακες συσχέτισης για το σήμα και το υπόβαθρο παρουσιάζονται στο Σχήμα 15



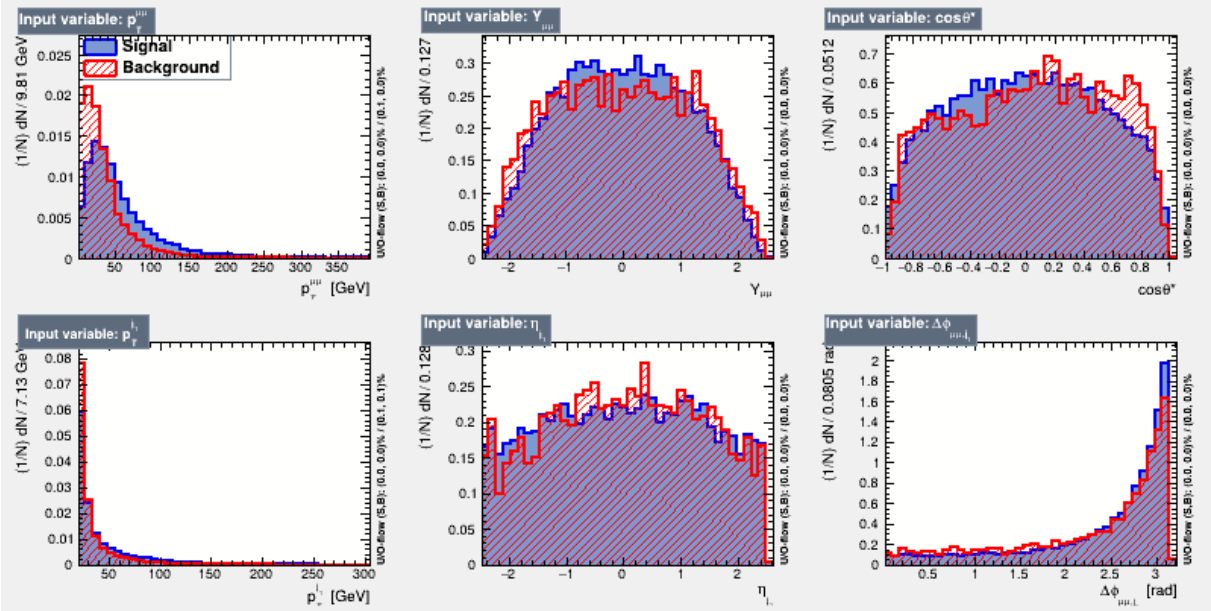
(α′) Γεγονότα σήματος. (β′) Γεγονότα υποβάθρου.

Σχήμα 15: Πίνακες γραμμικών συντελεστών συσχέτισης των μεταβλητών εισόδου για την κατηγορία $n_j = 0$.

Όπως προκύπτει από τους πίνακες συσχέτισης, οι τρεις μεταβλητές παρουσιάζουν πολύ μικρές γραμμικές συσχετίσεις μεταξύ τους τόσο για το σήμα όσο και για το υπόβαθρο. Οι εκτός διαγωνίου συντελεστές βρίσκονται κοντά στο μηδέν, με τις μεγαλύτερες απόλυτες τιμές να είναι της τάξης του 1%. Συνεπώς, δεν παρατηρείται σημαντική γραμμική εξάρτηση μεταξύ των μεταβλητών στη συγκεκριμένη κατηγορία γεγονότων. Οι μεταβλητές μπορούν επομένως να παρέχουν σε μεγάλο βαθμό συμπληρωματική πληροφορία στους ταξινομητές, παρά τη σημαντική επικάλυψη των επιμέρους κατανομών σήματος και υποβάθρου.

$\mathbf{n_j = 1}$

Για την κατηγορία $n_j = 1$, εκτός από τις μεταβλητές που χαρακτηρίζουν το διμιονικό σύστημα, χρησιμοποιήθηκαν και μεταβλητές που περιγράφουν τον ανακατασκευασμένο πίδακα και τη σχετική κινηματική του ως προς το διμιονικό σύστημα. Οι κανονικοποιημένες κατανομές των έξι μεταβλητών εισόδου για τα γεγονότα σήματος και υποβάθρου παρουσιάζονται στο Σχήμα 16.



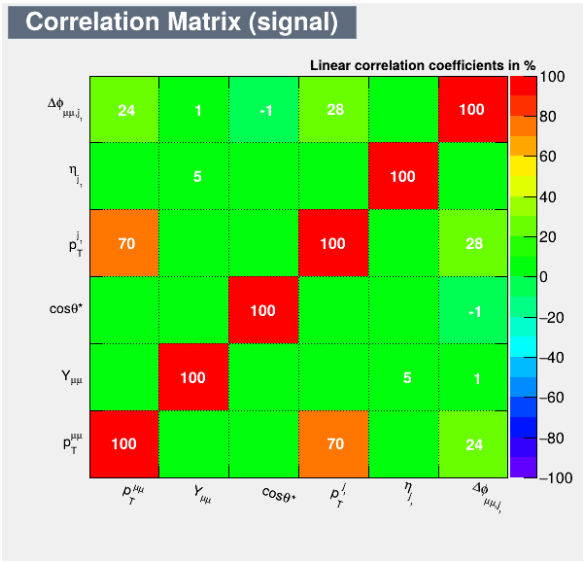
Σχήμα 16: Κανονικοποιημένες κατανομές των μεταβλητών εισόδου $p_T^{\mu\mu}$, $Y_{\mu\mu}$, $\cos\theta^*$, $p_T^{j_1}$, η_{j_1} και $\Delta\phi_{\mu\mu,j_1}$ για τα γεγονότα σήματος και υποβάθρου στην κατηγορία $n_j = 1$.

Όπως και στην κατηγορία $n_j = 0$, οι κατανομές των μεταβλητών παρουσιάζουν διαφορετικό βαθμό διαχωρισμού μεταξύ σήματος και υποβάθρου. Στην κατανομή της $p_T^{\mu\mu}$ το σήμα εμφανίζει σχετικά μεγαλύτερη συνεισφορά σε υψηλότερες τιμές εγκάρσιας ορμής. Αντίστοιχη διαφοροποίηση παρατηρείται και στην $p_T^{j_1}$ του πίδακα, με το σήμα να παρουσιάζει σχετικά ενισχυμένη συνεισφορά σε μεγαλύτερες τιμές.

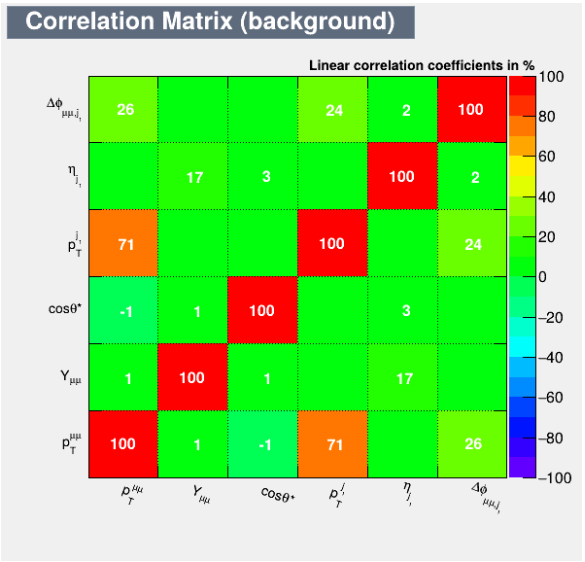
Οι κατανομές των $Y_{\mu\mu}$, η_{j_1} και $\cos\theta^*$ παρουσιάζουν επίσης διαφορές μεταξύ των δύο κλάσεων, αλλά με σημαντική επικάλυψη. Η κατανομή της $\Delta\phi_{\mu\mu,j_1}$ παρουσιάζει έντονη συγκέντρωση σε τιμές κοντά στο π τόσο για το σήμα όσο και για το υπόβαθρο, γεγονός που υποδεικνύει ότι ο πίδακας και το διμιονικό σύστημα εμφανίζονται συχνά με μεγάλη αζιμουθιακή γωνιακή απόσταση.

Η εισαγωγή των επιπλέον κινηματικών μεταβλητών στην κατηγορία $n_j = 1$ παρέχει επομένως πρόσθετη πληροφορία σχετικά με την κινηματική του γεγονότος. Παράλληλα, όμως, οι σημαντικές επικαλύψεις των επιμέρους κατανομών δείχνουν ότι ο διαχωρισμός σήματος και υποβάθρου δεν μπορεί να βασιστεί αποτελεσματικά σε μία μόνο μεταβλητή, αλλά απαιτεί τον συνδυασμό της πληροφορίας που περιέχεται στο σύνολο των μεταβλητών εισόδου.

Οι γραμμικές συσχετίσεις μεταξύ των έξι μεταβλητών παρουσιάζονται στο Σχήμα 17 για τα γεγονότα σήματος και υποβάθρου.



(α′) Γεγονότα σήματος.



(β′) Γεγονότα υποβάθρου.

Σχήμα 17: Πίνακες γραμμικών συντελεστών συσχέτισης των μεταβλητών εισόδου για τα γεγονότα σήματος και υποβάθρου στην κατηγορία $n_j = 1$.

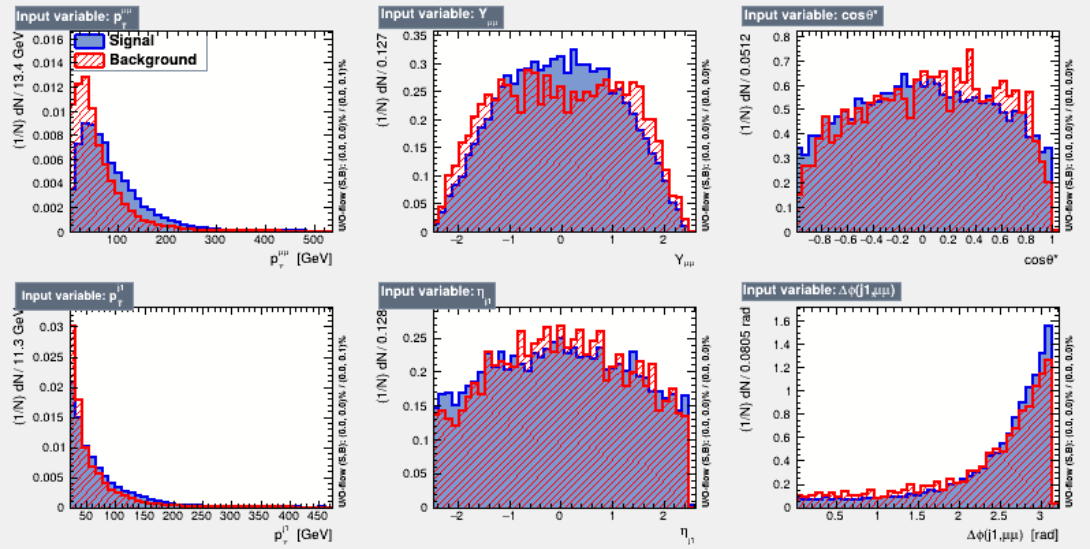
Σε αντίθεση με την κατηγορία $n_j = 0$, στην κατηγορία $n_j = 1$ εμφανίζονται εντονότερες γραμμικές συσχετίσεις μεταξύ ορισμένων μεταβλητών. Η ισχυρότερη παρατηρείται μεταξύ της εγκάρσιας ορμής του διμιονικού συστήματος $p_T^{\mu\mu}$ και της εγκάρσιας ορμής του πίδακα $p_T^{j_1}$, με συντελεστή συσχέτισης περίπου 70% για το σήμα και 71% για το υπόβαθρο. Η συμπεριφορά αυτή ανταναικλά την κινηματική σύνδεση μεταξύ του διμιονικού συστήματος και του ανακατασκευασμένου πίδακα στο γεγονός.

Ασθενέστερες συσχετίσεις παρατηρούνται μεταξύ της $\Delta\phi_{\mu\mu,j_1}$ και των εγκάρσιων ορμών $p_T^{\mu\mu}$ και $p_T^{j_1}$, με συντελεστές της τάξης του $24\% - 28\%$. Αντίθετα, η $\cos\theta^*$ παρουσιάζει πολύ μικρές γραμμικές συσχετίσεις με τις υπόλοιπες μεταβλητές. Συνολικά, η κατηγορία $n_j = 1$ εμφανίζει πλουσιότερη δομή συσχετίσεων σε σχέση με την κατηγορία $n_j = 0$, καθώς οι μεταβλητές που περιγράφουν το διμιονικό σύστημα συνδυάζονται πλέον με πληροφορία που αφορά την κινηματική του πίδακα.

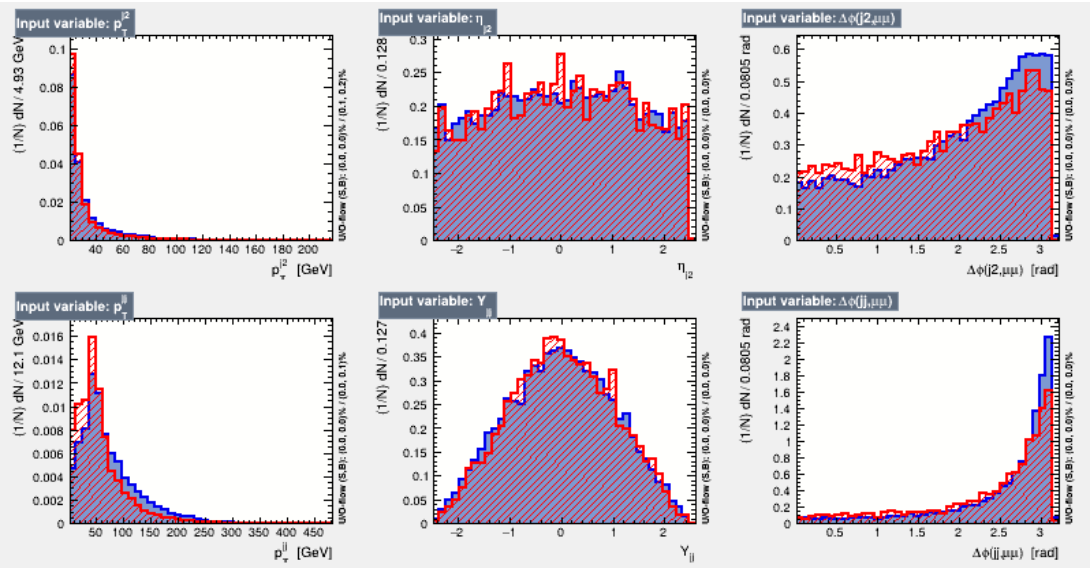
$\mathbf{n_j \geq 2}$

Στην κατηγορία $n_j \geq 2$ χρησιμοποιείται το μεγαλύτερο σύνολο μεταβλητών εισόδου, καθώς εκτός από τις μεταβλητές του διμιονικού συστήματος περιλαμβάνονται πληροφορίες για τους δύο πίδακες, το σύστημα διπίδακα και τη συνολική κινηματική του γεγονότος. Οι κανονικοποιημένες κατανομές των μεταβλητών παρουσιάζονται στα Σχήματα 18, 19 και 20.

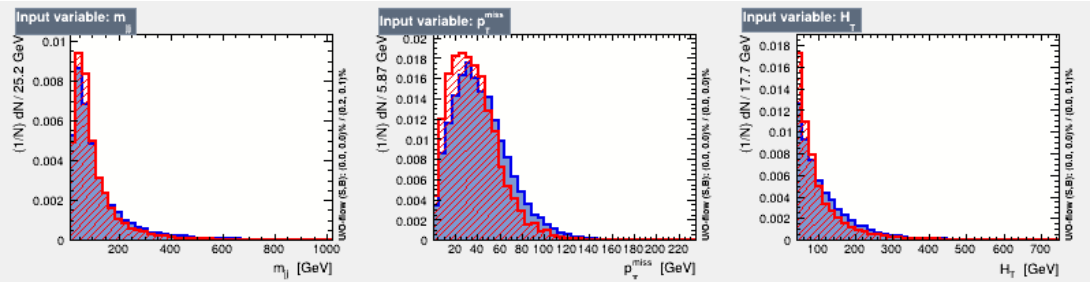
Οι κατανομές στην κατηγορία αυτή παρουσιάζουν διαφορετικούς βαθμούς διαχωρισμού μεταξύ σήματος και υποβάθρου. Οι μεταβλητές που χαρακτηρίζουν το διμιονικό σύστημα διατηρούν γενικά τη διαφορετική συμπεριφορά που παρατηρήθηκε και στις προηγούμενες κατηγορίες. Παράλληλα, η παρουσία τουλάχιστον δύο πιδάκων επιτρέπει την αξιοποίηση πρόσθετης κινηματικής πληροφορίας μέσω των χαρακτηριστικών των επιμέρους πιδάκων και του συστήματος διπίδακα.



Σχήμα 18: Κανονικοποιημένες κατανομές των μεταβλητών $p_T^{\mu\mu}$, $Y_{\mu\mu}$, $\cos\theta^*$, $p_T^{j_1}$, η_{j_1} και $\Delta\phi(j_1, \mu\mu)$ για τα γεγονότα σήματος και υποβάθρου στην κατηγορία $n_j \geq 2$.



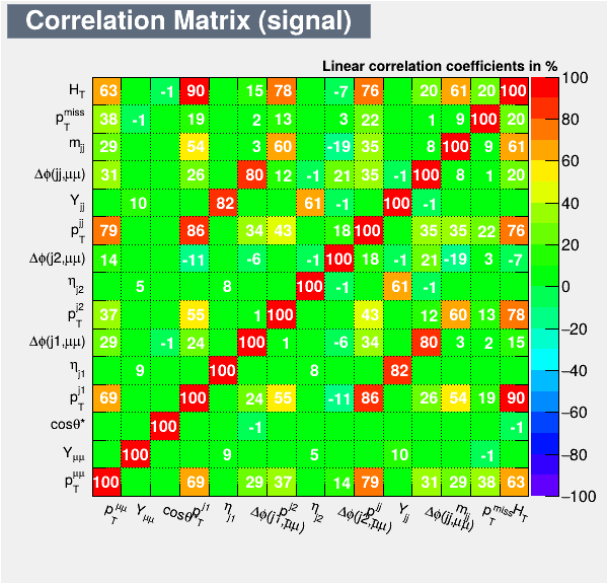
Σχήμα 19: Κανονικοποιημένες κατανομές των μεταβλητών $p_T^{j_2}$, η_{j_2}, $\Delta\phi(j_2, \mu\mu)$, p_T^{jj}, Y_{jj} και $\Delta\phi(jj, \mu\mu)$ για τα γεγονότα σήματος και υποβάθρου στην κατηγορία $n_j \geq 2$.



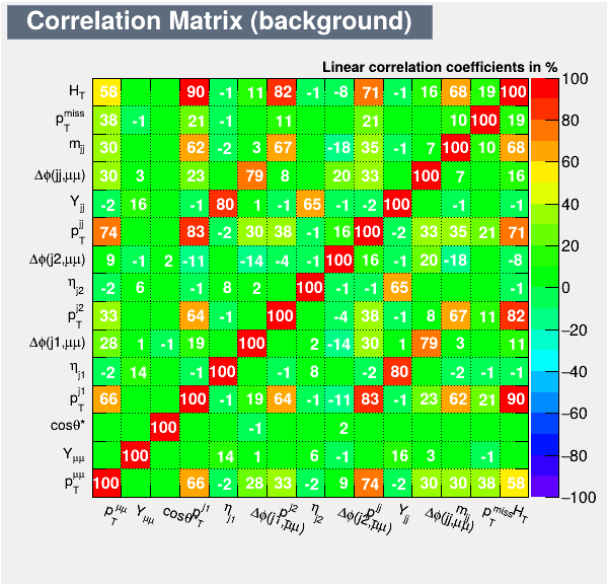
Σχήμα 20: Κανονικοποιημένες κατανομές των μεταβλητών m_{jj}, p_T^{miss} και H_T για τα γεγονότα σήματος και υποβάθρου στην κατηγορία $n_j \geq 2$.

Διαφορές μεταξύ σήματος και υποβάθρου παρατηρούνται, μεταξύ άλλων, στις κατανομές των εγκάρσιων ορμών των πιδάκων και του συστήματος διπίδακα, καθώς και στη μάζα m_{jj}. Αντίστοιχα, οι μεταβλητές p_T^{miss} και H_T παρέχουν πληροφορία για τη συνολικότερη κινηματική δραστηριότητα του γεγονότος. Παρότι σε ορισμένες μεταβλητές οι διαφορές είναι εμφανέστερες από ό,τι σε άλλες, σημαντική επικάλυψη μεταξύ των δύο κλάσεων εξακολουθεί να υπάρχει στο σύνολο των επιμέρους κατανομών.

Η μεγαλύτερη διάσταση του χώρου εισόδου και η ταυτόχρονη παρουσία μεταβλητών που περιγράφουν διαφορετικές πτυχές του ίδιου γεγονότος οδηγούν επίσης σε πλουσιότερη δομή συσχετίσεων. Οι πίνακες γραμμικών συντελεστών συσχέτισης για το σήμα και το υπόβαθρο παρουσιάζονται στο Σχήμα 21.



(α′) Γεγονότα σήματος.



(β′) Γεγονότα υποβάθρου.

Σχήμα 21: Πίνακες γραμμικών συντελεστών συσχέτισης των μεταβλητών εισόδου για τα γεγονότα σήματος και υποβάθρου στην κατηγορία $n_j \geq 2$.

Σε αντίθεση με τις προηγούμενες κατηγορίες, στην κατηγορία $n_j \geq 2$ παρατηρούνται αρκετές ισχυρές γραμμικές συσχετίσεις. Χαρακτηριστικό παράδειγμα αποτελεί η συσχέτιση μεταξύ $p_T^{j_1}$ και H_T, η οποία φτάνει περίπου στο 90% τόσο για το σήμα όσο και για το υπόβαθρο. Ισχυρή συσχέτιση παρατηρείται επίσης μεταξύ των $p_T^{j_1}$ και p_T^{jj}, με τιμές περίπου 86% για το σήμα και 83% για το υπόβαθρο, καθώς και μεταξύ των η_{j_1} και Y_{jj}, με συντελεστές περίπου 82% και 80% αντίστοιχα.

Σημαντικές συσχετίσεις εμφανίζονται ακόμη μεταξύ των μεταβλητών που περιγράφουν τις αζιμουθιακές γωνίες. Ενδεικτικά, η $\Delta\phi(j_1, \mu\mu)$ παρουσιάζει συσχέτιση περίπου 80% με την $\Delta\phi(jj, \mu\mu)$ τόσο για το σήμα όσο και για το υπόβαθρο. Παράλληλα, το m_{jj} παρουσιάζει αισθητή συσχέτιση με κινηματικές μεταβλητές των πιδάκων, ενώ το H_T, ως μέτρο της συνολικής εγκάρσιας δραστηριότητας του γεγονότος, εμφανίζει αναμενόμενα ισχυρές συσχετίσεις με τις εγκάρσιες ορμές.

Συνολικά, η κατηγορία $n_j \geq 2$ χαρακτηρίζεται από σημαντικά πλουσιότερη δομή συσχετίσεων σε σχέση με τις κατηγορίες $n_j = 0$ και $n_j = 1$. Το γεγονός αυτό είναι αναμενόμενο, καθώς αρκετές από τις μεταβλητές εισόδου κατασκευάζονται από κοινά κινηματικά αντικείμενα ή περιγράφουν διαφορετικές όψεις της ίδιας κινηματικής δομής.

Επομένως, η πληροφορία που παρέχεται στους ταξινομητές δεν προκύπτει μόνο από τις επιμέρους κατανομές των μεταβλητών, αλλά και από τις μεταξύ τους σχέσεις, τις οποίες οι μη γραμμικές πολυμεταβλητές μέθοδοι μπορούν να αξιοποιήσουν κατά τη διαδικασία κατηγοριοποίησης.

6.2 Αποτελέσματα Βελτιστοποίησης Υπερπαραμέτρων

Κατηγορία $\mathbf{n_j = 0}$

Likelihood

Η μεταβολή της μέσης AUC και της αντίστοιχης τυπικής απόκλισης για την μέθοδο Likelihood παρουσιάζεται στον Πίνακα 19.

Πίνακας 19: Αποτελέσματα της διασταυρούμενης επικύρωσης πέντε τμημάτων για διαφορετικές τιμές του N_{bins}.

Nbins	$\overline{\text{AUC}}_{\text{CV}}$	σ_{AUC}
50	0.604082	0.004325
100	0.654327	0.004934
150	**0.694384**	**0.009395**
200	0.692967	0.009416
250	0.690704	0.009085
300	0.684797	0.009092

Η μέση AUC αυξάνεται σημαντικά με την αύξηση του αριθμού των διαστημάτων μέχρι την τιμή `Nbins`$= 150$, για την οποία λαμβάνει τη μέγιστη τιμή.

Για μεγαλύτερες τιμές της υπερπαραμέτρου δεν παρατηρείται περαιτέρω βελτίωση, αλλά μία αρχική σταθεροποίηση και έπειτα σταδιακή μείωση της μέσης AUC. Για τον λόγο αυ-

τό, σύμφωνα με το προκαθορισμένο κριτήριο της μέγιστης μέσης AUC, επιλέχθηκε η τιμή `Nbins`= 150 για την τελική διαμόρφωση του ταξινομητή.

kNN

Η μεταβολή της μέσης AUC και της αντίστοιχης τυπικής απόκλισης για την μέθοδο kNN Πίνακα 20.

Πίνακας 20: Αποτελέσματα της διασταυρούμενης επικύρωσης πέντε τμημάτων για διαφορετικές τιμές του αριθμού γειτόνων k.

k	$\overline{\text{AUC}}_{\text{CV}}$	σ_{AUC}
50	0.685870	0.009132
100	0.690542	0.009101
150	0.692181	0.008999
200	0.692935	0.008913
250	0.693289	0.008861
300	**0.693417**	**0.008843**

Η μέση AUC αυξάνεται με την αύξηση του αριθμού των γειτόνων, με τη βελτίωση να γίνεται σταδιακά μικρότερη για τις μεγαλύτερες τιμές του k. Ειδικότερα, για $k = 200$, 250 και 300 η απόδοση παρουσιάζει τάση σταθεροποίησης, καθώς οι αντίστοιχες τιμές της μέσης AUC βρίσκονται πολύ κοντά μεταξύ τους. Η μέγιστη μέση τιμή προέκυψε για $k = 300$.

Για τον λόγο αυτό επιλέχθηκε η τιμή $k = 300$ για την τελική διαμόρφωση του ταξινομητή. Ωστόσο, η μικρή διαφορά της απόδοσης σε σχέση με τις γειτονικές τιμές $k = 200$ και $k = 250$, σε συνδυασμό με τη διακύμανση μεταξύ των επιμέρους folds, δείχνει ότι η επιλογή αυτή δεν αντιστοιχεί σε σαφώς διακριτό βέλτιστο, αλλά σε περιοχή όπου η απόδοση της μεθόδου έχει ουσιαστικά σταθεροποιηθεί.

MLP

Η μεταβολή της μέσης AUC και της αντίστοιχης τυπικής απόκλισης για την μέθοδο MLP παρουσιάζονται στον Πίνακα 21.

Πίνακας 21: Αποτελέσματα της διασταυρούμενης επικύρωσης πέντε τμημάτων για διαφορετικές τιμές του αριθμού νευρώνων N_{hidden} του κρυφού επιπέδου του MLP.

$\mathbf{N}_{\text{hidden}}$	$\overline{\text{AUC}}_{\text{CV}}$	σ_{AUC}
4	0.700908	0.0106680
6	0.701004	0.0107143
8	0.701010	0.0109026
10	0.701044	0.0105892
12	0.701103	0.0105198
14	**0.701169**	**0.0106249**
16	0.701052	0.0105161
18	0.700882	0.0105435
20	0.700259	0.0100870

Η απόδοση του MLP παρουσιάζει πολύ μικρές μεταβολές σε ολόκληρο το εξεταζόμενο εύρος του αριθμού των νευρώνων, σχηματίζοντας ουσιαστικά ένα πλατώ. Η μέγιστη μέση

τιμή της AUC προέκυψε για $N_{\text{hidden}} = 14$.

Η περαιτέρω αύξηση του αριθμού των νευρώνων δεν οδηγεί σε βελτίωση της μέσης απόδοσης. Παράλληλα, οι διαφορές της μέσης AUC μεταξύ των εξεταζόμενων διαμορφώσεων είναι πολύ μικρές σε σχέση με τη διακύμανση που παρατηρείται μεταξύ των επιμέρους folds. Συνεπώς, η επιλογή των 14 νευρώνων δεν υποδηλώνει σαφή υπεροχή της συγκεκριμένης διαμόρφωσης έναντι των γειτονικών τιμών, αλλά προκύπτει από το προκαθορισμένο κριτήριο της μέγιστης μέσης AUC.

DNN

Η μεταβολή της μέσης AUC και της αντίστοιχης τυπικής απόκλισης για την μέθοδο DNN παρουσιάζονται στον Πίνακα 22.

Πίνακας 22: Αποτελέσματα της διασταυρούμενης επικύρωσης πέντε τμημάτων για διαφορετικό αριθμό κρυφών επιπέδων L του DNN στην κατηγορία $n_j = 0$.

L	$\overline{\text{AUC}}_{\text{CV}}$	σ_{AUC}
1	0.689209	0.007765
2	0.692258	0.008968
3	0.689265	0.005748
4	0.696440	0.009487
5	**0.697236**	**0.010294**
6	0.696223	0.011517

Η μεταβολή του αριθμού των κρυφών επιπέδων οδηγεί σε πολύ μικρές μεταβολές της απόδοσης του DNN. Η μέγιστη τιμή προέκυψε για $L = 5$. Ωστόσο, η διαφορά της μέσης AUC μεταξύ των διαμορφώσεων με $L = 4$, 5 και 6 είναι μικρή σε σχέση με τη διακύμανση μεταξύ των επιμέρους folds. Συνεπώς, η επιλογή του $L = 5$ δεν υποδηλώνει σαφή υπεροχή της συγκεκριμένης αρχιτεκτονικής, αλλά προκύπτει από το προκαθορισμένο κριτήριο της μέγιστης μέσης AUC, ενώ τα αποτελέσματα υποδεικνύουν σταθεροποίηση της απόδοσης για τις βαθύτερες διαμορφώσεις.

BDT

Η μεταβολή της μέσης AUC και της αντίστοιχης τυπικής απόκλισης για την μέθοδο BDT παρουσιάζονται στον Πίνακα 23.

Πίνακας 23: Αποτελέσματα της διασταυρούμενης επικύρωσης πέντε τμημάτων για διαφορετικές τιμές του μέγιστου βάθους `MaxDepth` του BDT στην κατηγορία $n_j = 0$.

`MaxDepth`	$\overline{\text{AUC}}_{\text{CV}}$	σ_{AUC}
1	0.585116	0.005054
2	**0.697216**	**0.010430**
3	0.692616	0.009449
4	0.687643	0.009607
5	0.679318	0.007372
6	0.673915	0.009210

Η μετάβαση από `MaxDepth`= 1 σε `MaxDepth`= 2 οδηγεί σε σημαντική αύξηση της απόδο-

σης, γεγονός που δείχνει ότι το βάθος ενός επιπέδου δεν επαρκεί για την αποτελεσματική αξιοποίηση της πληροφορίας των μεταβλητών εισόδου. Αντίθετα, για `MaxDepth`> 2 παρατηρείται σταδιακή μείωση της μέσης AUC καθώς αυξάνεται το βάθος των δέντρων.

Συνεπώς, η αύξηση της πολυπλοκότητας των επιμέρους δέντρων πέρα από την τιμή `MaxDepth`$= 2$ δεν οδηγεί σε βελτίωση της ικανότητας γενίκευσης του ταξινομητή. Με βάση το προκαθορισμένο κριτήριο της μέγιστης μέσης AUC, επιλέχθηκε επομένως η τιμή `MaxDepth`$= 2$ για την τελική διαμόρφωση του BDT.

Κατηγορία $\mathbf{n_j = 1}$

DNN

Η μεταβολή της μέσης AUC και της αντίστοιχης τυπικής απόκλισης για την μέθοδο DNN παρουσιάζονται στον Πίνακα 24.

Πίνακας 24: Αποτελέσματα της διασταυρούμενης επικύρωσης πέντε τμημάτων για διαφορετικές αρχιτεκτονικές του DNN στην κατηγορία $n_j = 1$.

Αρχιτεκτονική	$\overline{\text{AUC}}_{\text{CV}}$	σ_{AUC}
$6-6$	0.667569	0.011663
$12-12$	0.672414	0.005212
$24-12$	0.664244	0.010514
$\mathbf{6-6-6}$	**0.673448**	**0.005735**
$12-12-12$	0.669227	0.023410
$24-24-24$	0.666918	0.017894
$24-12-6$	0.668295	0.006558
$12-12-12-12$	0.670278	0.009103
$36-24-12-6$	0.661639	0.011827

Η μεγαλύτερη μέση τιμή της AUC προέκυψε για την αρχιτεκτονική $6-6-6$, η οποία αποτελείται από τρία κρυφά επίπεδα με έξι νευρώνες ανά επίπεδο. Η αύξηση του αριθμού των νευρώνων ή του αριθμού των κρυφών επιπέδων δεν οδηγεί σε συστηματική βελτίωση της απόδοσης. Αντίθετα, αρκετές από τις πιο σύνθετες αρχιτεκτονικές παρουσιάζουν μικρότερη μέση AUC, ενώ η αρχιτεκτονική $6-6-6$ επιτυγχάνει τη μεγαλύτερη μέση τιμή με σχετικά μικρή διακύμανση μεταξύ των επιμέρους folds.

Με βάση το προκαθορισμένο κριτήριο της μέγιστης μέσης AUC, επιλέχθηκε επομένως η αρχιτεκτονική $6-6-6$ για την τελική διαμόρφωση του DNN. Το αποτέλεσμα δείχνει ότι, για το συγκεκριμένο σύνολο μεταβλητών εισόδου, η αύξηση της πολυπλοκότητας του δικτύου πέρα από τη συγκεκριμένη αρχιτεκτονική δεν συνοδεύεται από βελτίωση της ικανότητας γενίκευσης.

BDT

Η μεταβολή της μέσης AUC και της αντίστοιχης τυπικής απόκλισης για την μέθοδο BDT παρουσιάζονται στον Πίνακα 25.

Πίνακας 25: Αποτελέσματα της διασταυρούμενης επικύρωσης πέντε τμημάτων για διαφορετικές τιμές του μέγιστου βάθους `MaxDepth` του BDT στην κατηγορία $n_j = 1$.

`MaxDepth`	$\overline{\text{AUC}}_{\text{CV}}$	σ_{AUC}
1	0.604050	0.005711
2	**0.680234**	**0.007036**
3	0.672991	0.007432
4	0.662036	0.006089
5	0.642542	0.007302
6	0.629264	0.010730
7	0.619312	0.008727
8	0.609871	0.005979

Η μεγαλύτερη μέση τιμή της AUC προέκυψε ξανά για `MaxDepth`= 2. Παρόμοια με την κατηγορία $n_j = 0$, για μεγαλύτερες τιμές του μέγιστου βάθους παρατηρείται σταδιακή μείωση της μέσης AUC. Συνεπώς, η αύξηση της πολυπλοκότητας των επιμέρους δέντρων πέρα από την τιμή `MaxDepth`= 2 δεν οδηγεί σε βελτίωση της απόδοσης του ταξινομητή. Για τον λόγο αυτό, η συγκεκριμένη τιμή επιλέχθηκε για την τελική διαμόρφωση του BDT στην κατηγορία $n_j = 1$.

Κατηγορία $\mathbf{n_j \geq 2}$

DNN

Η μεταβολή της μέσης AUC και της αντίστοιχης τυπικής απόκλισης για την μέθοδο DNN παρουσιάζονται στον Πίνακα 26.

Πίνακας 26: Αποτελέσματα της διασταυρούμενης επικύρωσης πέντε τμημάτων για διαφορετικές αρχιτεκτονικές του DNN στην κατηγορία $n_j \geq 2$.

Αρχιτεκτονική	$\overline{\text{AUC}}_{\text{CV}}$	σ_{AUC}
$15 - 15$	0.600601	0.011831
$30 - 15$	0.602526	0.012576
$\mathbf{30 - 30}$	**0.603158**	**0.014761**
$15 - 15 - 15$	0.600798	0.009348
$30 - 30 - 30$	0.595748	0.010285
$60 - 30 - 15$	0.593170	0.013995
$15 - 15 - 15 - 15$	0.594049	0.011939
$30 - 30 - 30 - 30$	0.593239	0.009840
$75 - 60 - 30 - 15$	0.595305	0.017253

Η μεγαλύτερη μέση τιμή της AUC προέκυψε για την αρχιτεκτονική $30 - 30$, η οποία αποτελείται από δύο κρυφά επίπεδα με 30 νευρώνες ανά επίπεδο. Η αύξηση του αριθμού των κρυφών επιπέδων ή του αριθμού των νευρώνων δεν συνοδεύεται από συστηματική βελτίωση της απόδοσης. Αντίθετα, οι μεγαλύτερες αρχιτεκτονικές παρουσιάζουν παρόμοιες ή χαμηλότερες τιμές της μέσης AUC. Επιπλέον, οι διαφορές μεταξύ αρκετών από τις εξεταζόμενες διαμορφώσεις είναι μικρές σε σχέση με τη διακύμανση που παρατηρείται μεταξύ των επιμέρους folds.

Με βάση το προκαθορισμένο κριτήριο της μέγιστης μέσης AUC, επιλέχθηκε επομένως η αρχιτεκτονική $30-30$ για την τελική διαμόρφωση του DNN. Τα αποτελέσματα δείχνουν ότι, για το συγκεκριμένο σύνολο μεταβλητών εισόδου, η περαιτέρω αύξηση της πολυπλοκότητας του δικτύου δεν οδηγεί σε βελτίωση της ικανότητας γενίκευσης.

BDT

Η μεταβολή της μέσης AUC και της αντίστοιχης τυπικής απόκλισης για την μέθοδο BDT παρουσιάζονται στον Πίνακα 27.

Πίνακας 27: Αποτελέσματα της διασταυρούμενης επικύρωσης πέντε τμημάτων για διαφορετικές τιμές του μέγιστου βάθους `MaxDepth` του BDT στην κατηγορία $n_j \geq 2$.

`MaxDepth`	$\overline{\text{AUC}}_{\text{CV}}$	σ_{AUC}
1	0.626520	0.003956
2	**0.656909**	**0.006565**
3	0.643290	0.008651
4	0.624684	0.010854
5	0.606275	0.010320
6	0.602237	0.015681
7	0.595082	0.013110
8	0.587888	0.012602

Η μεγαλύτερη μέση τιμή της AUC προέκυψε ξανά για `MaxDepth`$= 2$, Όπως και στις κατηγορίες $n_j = 0$ και $n_j = 1$, η αύξηση του μέγιστου βάθους πέρα από την τιμή αυτή συνοδεύεται από σταδιακή μείωση της μέσης AUC. Επομένως, η περαιτέρω αύξηση της πολυπλοκότητας των επιμέρους δέντρων δεν οδηγεί σε βελτίωση της απόδοσης του ταξινομητή.

6.3 Τελική Απόδοση και Σύγκριση των Μοντέλων

Κατηγορία $\mathbf{n_j = 0}$

Στον Πίνακα 28 συγκρίνεται η μέση AUC που προέκυψε κατά τη διασταυρούμενη επικύρωση με την τελική τιμή της στο ανεξάρτητο σύνολο ελέγχου. Για τη μέθοδο Fisher δεν πραγματοποιήθηκε αντίστοιχη διαδικασία επιλογής υπερπαραμέτρων και επομένως παρουσιάζεται μόνο η τελική της απόδοση.

Πίνακας 28: Σύγκριση της μέσης AUC της διασταυρούμενης επικύρωσης με την τελική AUC στο ανεξάρτητο σύνολο ελέγχου για την κατηγορία $n_j = 0$.

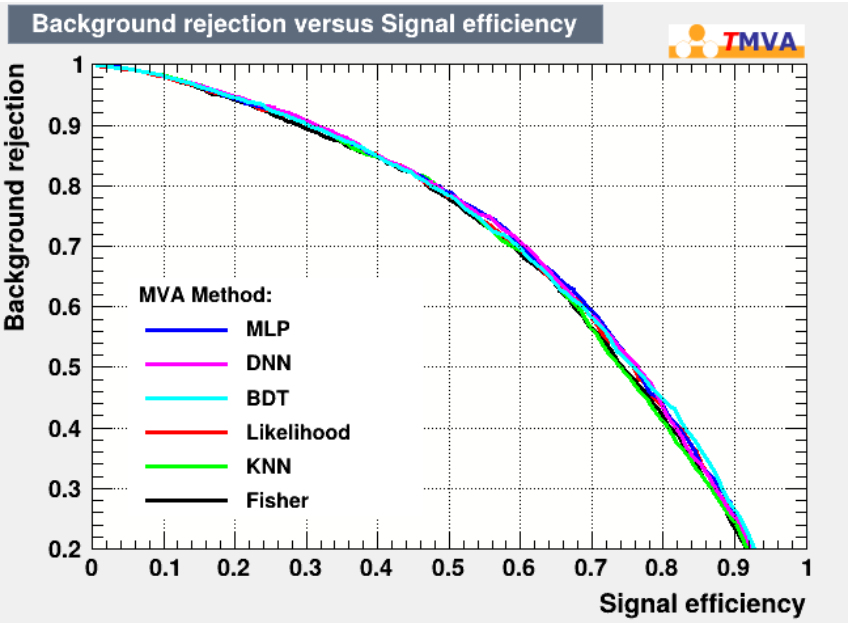
Μέθοδος	$\overline{\text{AUC}}_{\text{CV}}$	AUC_{test}
Fisher	–	0.687815
Likelihood	0.694384	0.695086
kNN	0.693417	0.689634
MLP	0.701169	**0.698529**
DNN	0.697236	0.697582
BDT	0.697216	0.697498

Οι τελικές τιμές της AUC_{test} παραμένουν κοντά στις αντίστοιχες μέσες τιμές $\overline{\text{AUC}}_{\text{CV}}$.

Η συμφωνία αυτή δείχνει ότι η απόδοση που εκτιμήθηκε κατά τη διαδικασία επιλογής των υπερπαραμέτρων διατηρείται σε γενικές γραμμές και στο ανεξάρτητο σύνολο ελέγχου, αποτελώντας ένδειξη καλής γενικευτικής ικανότητας των επιλεγμένων μοντέλων.

Τη μεγαλύτερη τιμή στο ανεξάρτητο σύνολο ελέγχου παρουσιάζει το MLP, με $\mathrm{AUC}_{\mathrm{test}} = 0.698529$, ενώ πολύ κοντινές τιμές προκύπτουν για τα DNN και BDT. Αντίθετα, τη μικρότερη τιμή παρουσιάζει η γραμμική μέθοδος Fisher, με $\mathrm{AUC}_{\mathrm{test}} = 0.687815$.

Η συνολική διαχωριστική ικανότητα των τελικών ταξινομητών παρουσιάζεται επιπλέον μέσω των καμπυλών ROC στο Σχήμα 22.

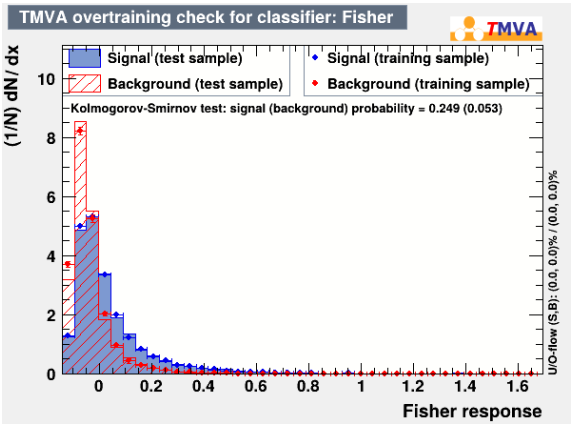


Σχήμα 22: Σύγκριση των καμπυλών ROC των τελικών ταξινομητών στην κατηγορία $n_j = 0$.

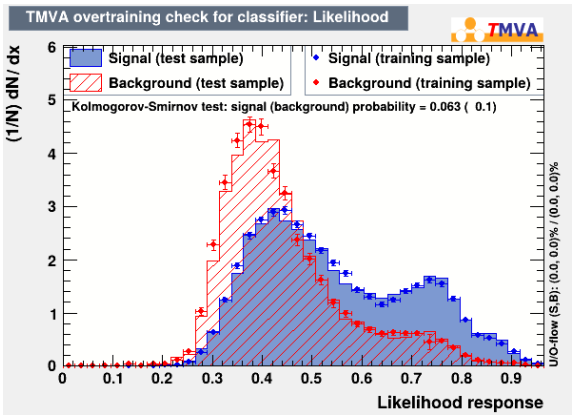
Η μικρή διαφορά μεταξύ των καλύτερων ταξινομητών αποτυπώνεται και στη σημαντική επικάλυψη των αντίστοιχων καμπυλών ROC. Επομένως, παρότι το MLP παρουσιάζει τη μεγαλύτερη συνολική τιμή της AUC, δεν παρατηρείται μεγάλη διαφορά στη συνολική διαχωριστική ικανότητα μεταξύ των καλύτερων μεθόδων.

Αξιοσημείωτο είναι επίσης ότι η χρήση πιο σύνθετων μοντέλων δεν οδηγεί σε μεγάλη βελτίωση της απόδοσης. Οι απλούστερες μέθοδοι εμφανίζουν συγκρίσιμη διαχωριστική ικανότητα με το MLP, το DNN και το BDT . Το γεγονός αυτό μπορεί να συσχετιστεί με το περιορισμένο σύνολο των τριών μεταβλητών εισόδου που χρησιμοποιείται στην παρούσα ανάλυση. Ιδιαίτερα η μεταβλητή $p_T^{\mu\mu}$ εμφανίζει σημαντικά μεγαλύτερη μονοδιάστατη διαχωριστική ικανότητα σε σχέση με τις $Y_{\mu\mu}$ και $\cos\theta^*$, με αποτέλεσμα σημαντικό μέρος της διαθέσιμης πληροφορίας για τον διαχωρισμό να περιέχεται ήδη σε μία από τις μεταβλητές εισόδου.

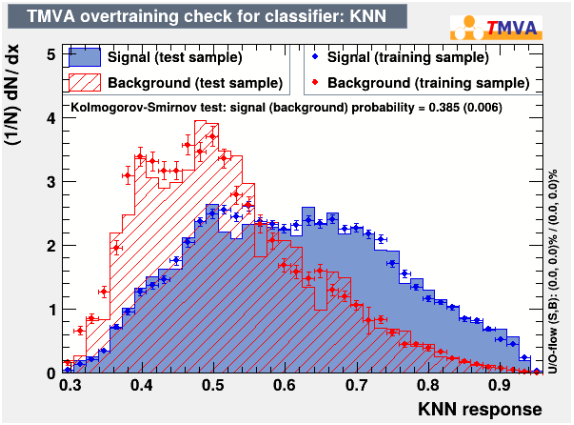
Για τον έλεγχο πιθανής υπερεκπαίδευσης των τελικών μοντέλων, συγκρίθηκαν οι κατανομές εξόδου των ταξινομητών για τα γεγονότα εκπαίδευσης και ελέγχου. Οι αντίστοιχες κατανομές για το σήμα και το υπόβαθρο παρουσιάζονται στο Σχήμα 23, επιτρέποντας την οπτική αξιολόγηση της συμφωνίας μεταξύ των δύο συνόλων δεδομένων.



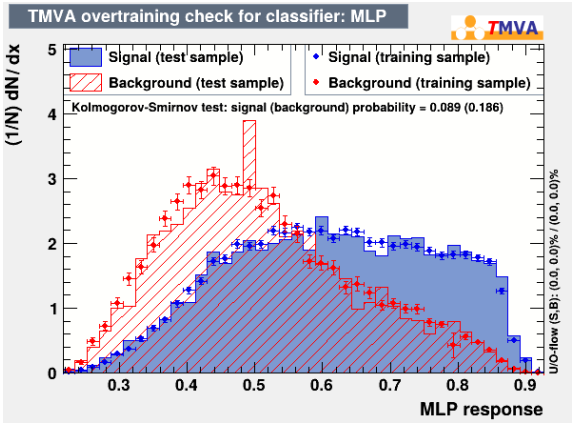
(α′) Fisher



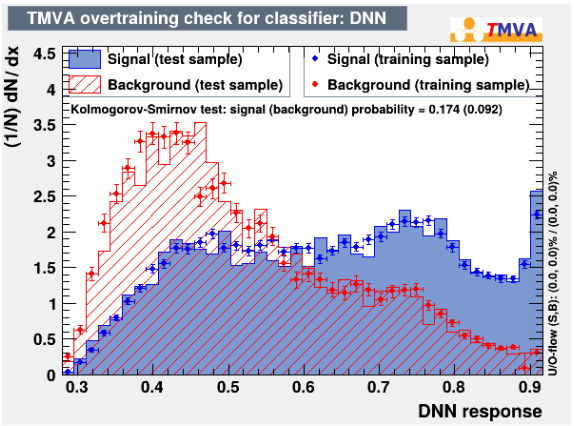
(β′) Likelihood



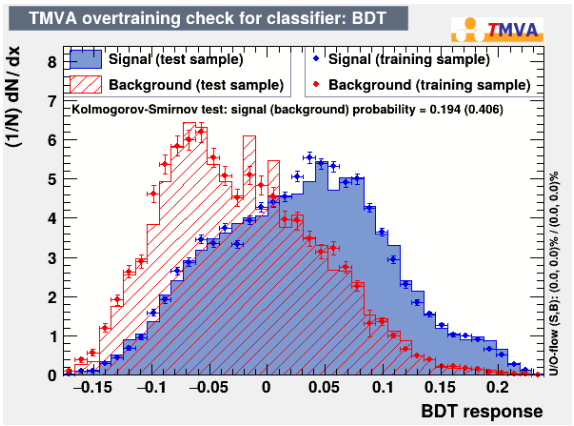
(γ′) kNN



(δ′) MLP



(ε′) DNN



(ϛ′) BDT

Σχήμα 23: Σύγκριση των κατανομών εξόδου των τελικών ταξινομητών για τα σύνολα εκπαίδευσης και ελέγχου στην κατηγορία $n_j = 0$.

Οι κατανομές εξόδου παρουσιάζουν διαφορετική μορφή και εύρος τιμών μεταξύ των ταξινομητών, αντανακλώντας τον διαφορετικό τρόπο με τον οποίο κάθε μέθοδος κατασκευάζει το διακριτικό της μέγεθος. Στην περίπτωση του Fisher, οι κατανομές συγκεντρώνονται κυρίως σε ένα σχετικά περιορισμένο εύρος τιμών και παρουσιάζουν σημαντική επικάλυψη, με τα γεγονότα σήματος να μετατοπίζονται προς μεγαλύτερες τιμές της απόκρισης σε σχέση με το υπόβαθρο. Η συμπεριφορά αυτή είναι συμβατή με τη γραμμική φύση του διακριτικού

μεγέθους της μεθόδου.

Για τις μεθόδους Likelihood και kNN, η έξοδος εκτείνεται σε διαφορετικό εύρος τιμών, με το υπόβαθρο να συγκεντρώνεται περισσότερο προς χαμηλότερες τιμές και το σήμα να εμφανίζει μεγαλύτερη συγκέντρωση προς υψηλότερες. Παρά τη διαφορετική μορφή των κατανομών, παραμένει σημαντική περιοχή επικάλυψης μεταξύ των δύο κλάσεων, σε συμφωνία με την περιορισμένη διαχωριστική ικανότητα που προέκυψε και από τις αντίστοιχες τιμές της AUC.

Αντίστοιχη μετατόπιση του σήματος προς υψηλότερες τιμές της απόκρισης παρατηρείται στις μη γραμμικές μεθόδους MLP και DNN. Ιδιαίτερα στην περίπτωση του DNN, η κατανομή του σήματος εκτείνεται περισσότερο προς το υψηλό άκρο της απόκρισης, ενώ το υπόβαθρο παρουσιάζει μεγαλύτερη συγκέντρωση σε χαμηλότερες τιμές. Στην περίπτωση του BDT, η έξοδος ακολουθεί διαφορετική αριθμητική κλίμακα, με το υπόβαθρο να συγκεντρώνεται περισσότερο προς αρνητικές τιμές και το σήμα προς θετικότερες τιμές της απόκρισης.

Οι διαφορετικές αριθμητικές κλίμακες και μορφές των κατανομών δεν επιτρέπουν την άμεση σύγκριση των τιμών εξόδου μεταξύ διαφορετικών ταξινομητών. Αντίθετα, η διαχωριστική ικανότητα κάθε μεθόδου συνδέεται με τη σχετική θέση και τον βαθμό επικάλυψης των κατανομών σήματος και υποβάθρου και συγκρίνεται ποσοτικά μέσω των καμπυλών ROC και της AUC.

Πέρα από τη μορφή των κατανομών εξόδου, ιδιαίτερη σημασία για τον έλεγχο πιθανής υπερεκπαίδευσης έχει η συμφωνία μεταξύ των αντίστοιχων κατανομών εκπαίδευσης και ελέγχου. Η συμφωνία αυτή αξιολογήθηκε τόσο οπτικά όσο και ποσοτικά μέσω του ελέγχου Kolmogorov–Smirnov[6].

Οι αντίστοιχες πιθανότητες παρουσιάζουν διαφοροποιήσεις μεταξύ των ταξινομητών. Ειδικότερα, για τις μεθόδους Fisher, Likelihood, MLP, DNN και BDT δεν παρατηρούνται ιδιαίτερα χαμηλές τιμές ταυτόχρονα για τα δείγματα σήματος και υποβάθρου. Αντίθετα, στην περίπτωση του kNN προκύπτει τιμή $KS_B = 0.006$ για το υπόβαθρο, ενώ για το σήμα προκύπτει $KS_S = 0.385$. Η χαμηλή τιμή για το υπόβαθρο υποδεικνύει μεγαλύτερη διαφοροποίηση μεταξύ των αντίστοιχων κατανομών εκπαίδευσης και ελέγχου και αποτελεί ένδειξη που πρέπει να λαμβάνεται υπόψη κατά την αξιολόγηση πιθανής υπερεκπαίδευσης.

Ωστόσο, ο έλεγχος Kolmogorov–Smirnov δεν χρησιμοποιείται ως μεμονωμένο κριτήριο για τη διαπίστωση υπερεκπαίδευσης, αλλά εξετάζεται σε συνδυασμό με την οπτική συμφωνία των κατανομών και την απόδοση στο ανεξάρτητο σύνολο ελέγχου. Συνολικά, η γενική συμφωνία των κατανομών εκπαίδευσης και ελέγχου, σε συνδυασμό με τις κοντινές τιμές της $\overline{\mathrm{AUC}}_{\mathrm{CV}}$ και της $\mathrm{AUC}_{\mathrm{test}}$, δεν παρέχει ένδειξη έντονης υπερεκπαίδευσης των τελικών ταξινομητών στην κατηγορία $n_j = 0$, με την παραπάνω απόκλιση του kNN για το υπόβαθρο να αποτελεί τη σημαντικότερη εξαίρεση.

[6] Ο έλεγχος Kolmogorov–Smirnov συγκρίνει δύο κατανομές και ελέγχει κατά πόσο είναι συμβατές με την υπόθεση ότι προέρχονται από την ίδια υποκείμενη κατανομή. Η αντίστοιχη πιθανότητα (p-value) χρησιμοποιείται εδώ ως διαγνωστικό μέτρο της συμφωνίας μεταξύ των κατανομών εκπαίδευσης και ελέγχου. Μικρές τιμές υποδεικνύουν μεγαλύτερη ασυμφωνία μεταξύ των δύο κατανομών.

Κατηγορία $\mathbf{n_j = 1}$

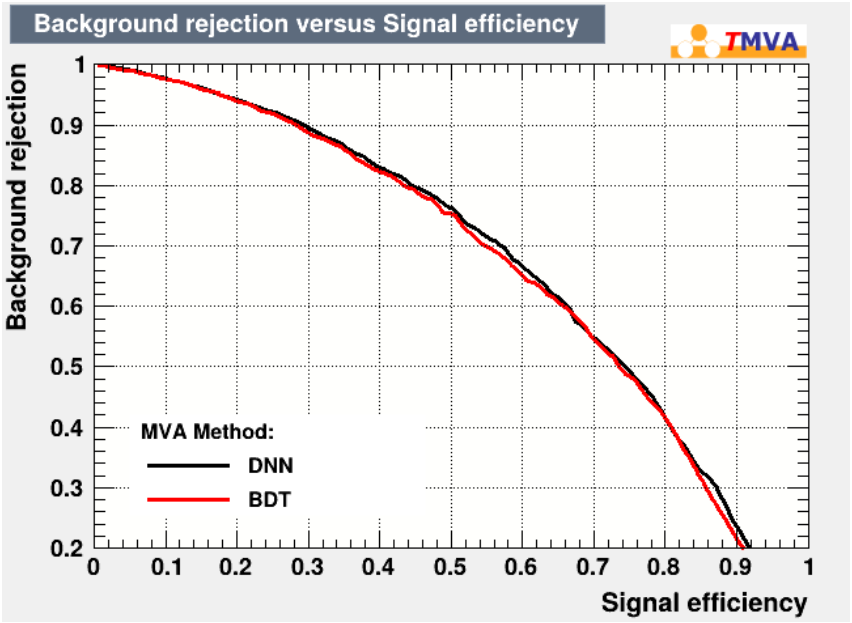
Στον Πίνακα 29 συγκρίνεται η μέση AUC που προέκυψε κατά τη διασταυρούμενη επικύρωση με την τελική τιμή της στο ανεξάρτητο σύνολο ελέγχου για τις μεθόδους DNN και BDT.

Πίνακας 29: Σύγκριση της μέσης AUC της διασταυρούμενης επικύρωσης με την τελική AUC στο ανεξάρτητο σύνολο ελέγχου για την κατηγορία $n_j = 1$.

Μέθοδος	$\overline{\text{AUC}}_{\text{CV}}$	AUC_{test}
DNN	0.673448	**0.686130**
BDT	0.680234	0.673398

Οι τελικές τιμές της AUC_{test} παραμένουν σχετικά κοντά στις αντίστοιχες μέσες τιμές $\overline{\text{AUC}}_{\text{CV}}$. Για το DNN παρατηρείται αύξηση της τελικής AUC σε σχέση με τη μέση τιμή της διασταυρούμενης επικύρωσης, ενώ για το BDT παρατηρείται μικρή μείωση. Συνολικά, η απόδοση των δύο μοντέλων στο ανεξάρτητο σύνολο ελέγχου παραμένει συμβατή με την εικόνα που προέκυψε κατά τη διασταυρούμενη επικύρωση, αποτελώντας ένδειξη ικανοποιητικής γενικευτικής ικανότητας.

Η συνολική διαχωριστική ικανότητα των δύο τελικών ταξινομητών παρουσιάζεται μέσω των καμπυλών ROC, οι οποίες εμφανίζουν και πάλι σημαντική επικάλυψη, στο Σχήμα 24.

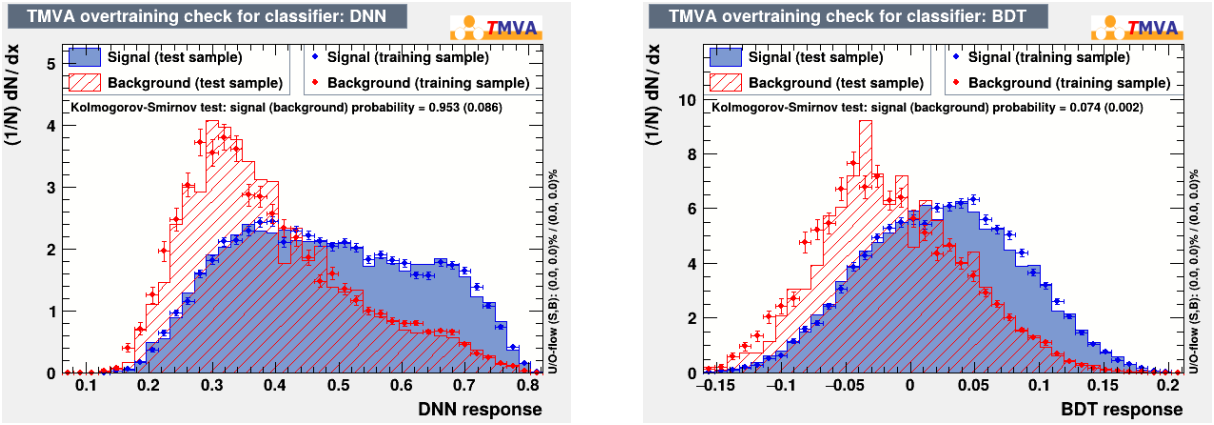


Σχήμα 24: Σύγκριση των καμπυλών ROC των τελικών ταξινομητών στην κατηγορία $n_j = 1$.

Για τον έλεγχο πιθανής υπερεκπαίδευσης συγκρίθηκαν οι κατανομές εξόδου των δύο τελικών ταξινομητών για τα γεγονότα εκπαίδευσης και ελέγχου. Οι αντίστοιχες κατανομές για το σήμα και το υπόβαθρο παρουσιάζονται στο Σχήμα 25.

Και στις δύο μεθόδους παρατηρείται διαχωρισμός των κατανομών σήματος και υποβάθρου, με τα γεγονότα σήματος να τείνουν προς υψηλότερες τιμές της απόκρισης του ταξινομητή. Στην περίπτωση του DNN, το υπόβαθρο συγκεντρώνεται περισσότερο προς τις χαμηλότερες

τιμές της εξόδου, ενώ η κατανομή του σήματος εκτείνεται σε μεγαλύτερες τιμές. Αντίστοιχη συμπεριφορά εμφανίζεται και για το BDT, στη διαφορετική αριθμητική κλίμακα της απόκρισής του, με το υπόβαθρο να συγκεντρώνεται περισσότερο προς αρνητικές τιμές και το σήμα προς θετικότερες.



(α′) DNN (β′) BDT

Σχήμα 25: Σύγκριση των κατανομών εξόδου των DNN και BDT για τα σύνολα εκπαίδευσης και ελέγχου στην κατηγορία $n_j = 1$.

Η σημαντική επικάλυψη που παραμένει μεταξύ των κατανομών σήματος και υποβάθρου είναι συνεπής με τις τιμές της AUC που προέκυψαν για τις δύο μεθόδους και δείχνει ότι ο διαχωρισμός των δύο κλάσεων παραμένει περιορισμένος.

Όσον αφορά τον έλεγχο πιθανής υπερεκπαίδευσης, η συμφωνία μεταξύ των κατανομών εκπαίδευσης και ελέγχου αξιολογήθηκε τόσο οπτικά όσο και μέσω του ελέγχου Kolmogorov–Smirnov. Για το DNN προέκυψαν πιθανότητες $KS_S = 0.953$ για το σήμα και $KS_B = 0.086$ για το υπόβαθρο, με τις αντίστοιχες κατανομές εκπαίδευσης και ελέγχου να παρουσιάζουν συνολικά καλή συμφωνία.

Για το BDT, οι αντίστοιχες πιθανότητες είναι $KS_S = 0.074$ και $KS_B = 0.002$. Ιδιαίτερα η χαμηλή τιμή που προκύπτει για το υπόβαθρο υποδεικνύει διαφοροποίηση μεταξύ των κατανομών εκπαίδευσης και ελέγχου και αποτελεί ένδειξη πιθανής υπερεκπαίδευσης. Η συμπεριφορά αυτή είναι ιδιαίτερα σημαντική για τα BDT, καθώς η συγκεκριμένη κατηγορία ταξινομητών μπορεί να παρουσιάσει αυξημένη ευαισθησία στην υπερεκπαίδευση όταν αυξάνεται υπερβολικά η πολυπλοκότητα των επιμέρους δέντρων. Για τον λόγο αυτό, στο TMVA προτείνεται η χρήση σχετικά μικρού βάθους δέντρων ως μέσο περιορισμού της υπερεκπαίδευσης. Στην παρούσα ανάλυση, η διαδικασία βελτιστοποίησης οδήγησε στην επιλογή `MaxDepth`$= 2$.

Κατηγορία $\mathbf{n_j \geq 2}$

Στον Πίνακα 30 συγκρίνεται η μέση AUC που προέκυψε κατά τη διασταυρούμενη επικύρωση με την τελική τιμή της στο ανεξάρτητο σύνολο ελέγχου για τις μεθόδους DNN και BDT.

Πίνακας 30: Σύγκριση της μέσης AUC της διασταυρούμενης επικύρωσης με την τελική AUC στο ανεξάρτητο σύνολο ελέγχου για την κατηγορία $n_j \geq 2$.

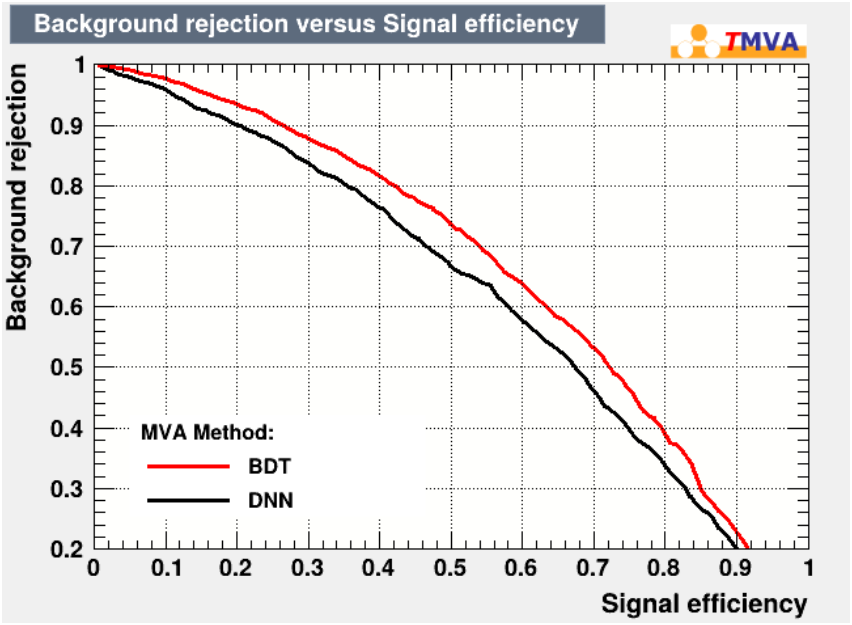
Μέθοδος	$\overline{\text{AUC}}_{\text{CV}}$	AUC_{test}
DNN	0.603158	0.620458
BDT	0.656909	**0.664808**

Οι τελικές τιμές της AUC_{test} είναι υψηλότερες από τις αντίστοιχες μέσες τιμές $\overline{\text{AUC}}_{\text{CV}}$ και για τις δύο μεθόδους. Η διαφορά είναι μεγαλύτερη στην περίπτωση του DNN, ενώ για το BDT η τελική απόδοση παραμένει πιο κοντά στην εκτίμηση της διασταυρούμενης επικύρωσης. Σε κάθε περίπτωση, δεν παρατηρείται υποβάθμιση της απόδοσης κατά την αξιολόγηση στο ανεξάρτητο σύνολο ελέγχου.

Στην κατηγορία $n_j \geq 2$, το BDT παρουσιάζει υψηλότερη συνολική διαχωριστική ικανότητα, επιτυγχάνοντας $\text{AUC}_{\text{test}} = 0.664808$, έναντι $\text{AUC}_{\text{test}} = 0.620458$ για το DNN. Η διαφορά μεταξύ των δύο μεθόδων είναι εντονότερη από εκείνη που παρατηρήθηκε στην κατηγορία $n_j = 1$, όπου οι αντίστοιχες τελικές τιμές της AUC ήταν περισσότερο συγκρίσιμες.

Η υπεροχή του BDT παρατηρείται τόσο κατά τη διασταυρούμενη επικύρωση όσο και κατά την τελική αξιολόγηση στο ανεξάρτητο σύνολο ελέγχου. Επομένως, για τις συγκεκριμένες διαμορφώσεις των δύο μοντέλων και το σύνολο των μεταβλητών που χρησιμοποιείται στην κατηγορία $n_j \geq 2$, το BDT παρουσιάζει καλύτερη διαχωριστική ικανότητα από το DNN.

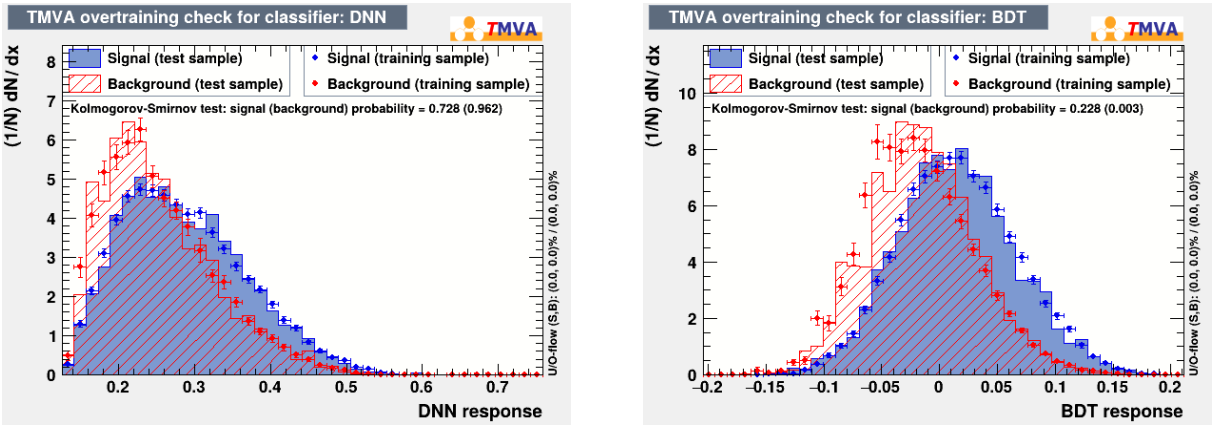
Η συνολική διαχωριστική ικανότητα των δύο τελικών ταξινομητών παρουσιάζεται μέσω των καμπυλών ROC στο Σχήμα 26.



Σχήμα 26: Σύγκριση των καμπυλών ROC των τελικών ταξινομητών στην κατηγορία $n_j \geq 2$.

Η διαφορά αυτή είναι ορατή και στις αντίστοιχες καμπύλες ROC, όπου το BDT παρουσιάζει γενικά υψηλότερη αποδοτικότητα σήματος για δεδομένη αποδοτικότητα υποβάθρου. Η μεγαλύτερη απόσταση μεταξύ των δύο καμπυλών, σε σχέση με την κατηγορία $n_j = 1$, είναι συνεπής με την εντονότερη διαφορά των αντίστοιχων τιμών της AUC.

Για τον έλεγχο πιθανής υπερεκπαίδευσης συγκρίθηκαν οι κατανομές εξόδου των δύο τελικών ταξινομητών για τα γεγονότα εκπαίδευσης και ελέγχου. Οι αντίστοιχες κατανομές για το σήμα και το υπόβαθρο παρουσιάζονται στο Σχήμα 27.



(α′) DNN (β′) BDT

Σχήμα 27: Σύγκριση των κατανομών εξόδου των DNN και BDT για τα σύνολα εκπαίδευσης και ελέγχου στην κατηγορία $n_j \geq 2$.

Και στις δύο μεθόδους παρατηρείται διαχωρισμός μεταξύ των κατανομών σήματος και υποβάθρου, με τα γεγονότα σήματος να τείνουν προς υψηλότερες τιμές της απόκρισης του ταξινομητή. Στην περίπτωση του DNN, οι δύο κατανομές παρουσιάζουν σημαντική επικάλυψη, σε συμφωνία με τη σχετικά χαμηλότερη τιμή της AUC που προέκυψε για τη συγκεκριμένη μέθοδο. Στην περίπτωση του BDT, ο διαχωρισμός των δύο κλάσεων είναι περισσότερο εμφανής, με το υπόβαθρο να συγκεντρώνεται προς χαμηλότερες τιμές της απόκρισης και το σήμα προς υψηλότερες. Η εικόνα αυτή είναι συνεπής με την υψηλότερη τιμή της AUC που επιτυγχάνει το BDT στην κατηγορία $n_j \geq 2$.

Όσον αφορά τον έλεγχο πιθανής υπερεκπαίδευσης, για το DNN προέκυψαν πιθανότητες $KS_S = 0.728$ για το σήμα και $KS_B = 0.962$ για το υπόβαθρο. Οι υψηλές αυτές τιμές, σε συνδυασμό με την οπτική συμφωνία των αντίστοιχων κατανομών, δεν υποδεικνύουν σημαντική διαφοροποίηση μεταξύ των συνόλων εκπαίδευσης και ελέγχου.

Για το BDT, οι αντίστοιχες πιθανότητες είναι $KS_S = 0.228$ και $KS_B = 0.003$. Ενώ για το σήμα παρατηρείται ικανοποιητική συμφωνία, η ιδιαίτερα χαμηλή τιμή για το υπόβαθρο υποδεικνύει διαφοροποίηση μεταξύ των κατανομών εκπαίδευσης και ελέγχου και αποτελεί ένδειξη πιθανής υπερεκπαίδευσης για τη συγκεκριμένη κλάση.

Παρά τη διαφοροποίηση αυτή, η τελική $\mathrm{AUC}_{\mathrm{test}} = 0.664808$ του BDT παραμένει κοντά στη μέση τιμή $\overline{\mathrm{AUC}}_{\mathrm{CV}} = 0.656909$ και είναι υψηλότερη από την αντίστοιχη τιμή του DNN. Επομένως, η χαμηλή τιμή του KS_B λαμβάνεται υπόψη ως διαγνωστική ένδειξη πιθανής υπερεκπαίδευσης, χωρίς όμως από μόνη της να αναιρεί την ικανότητα γενίκευσης που παρατηρείται από την απόδοση του μοντέλου στο ανεξάρτητο σύνολο ελέγχου.

7 Συμπεράσματα

Τα αποτελέσματα της ανάλυσης επιτρέπουν ορισμένες γενικότερες παρατηρήσεις σχετικά με τη φύση του προβλήματος κατηγοριοποίησης. Στην κατηγορία $n_j = 0$, η εφαρμογή ταξινομητών διαφορετικής πολυπλοκότητας δεν οδήγησε σε ουσιαστικά διαφορετική διαχωριστική ικανότητα. Παρότι οι μέθοδοι που εξετάστηκαν εκτείνονται από τον γραμμικό Fisher έως τα νευρωνικά δίκτυα και τα Boosted Decision Trees, οι τελικές τιμές της AUC παρέμειναν σε σχετικά περιορισμένο εύρος. Επομένως, για το συγκεκριμένο σύνολο μεταβλητών, η αύξηση της ευελιξίας του ταξινομητή δεν φαίνεται να αποκαλύπτει σημαντική πρόσθετη πληροφορία για τον διαχωρισμό του σήματος από το υπόβαθρο. Το αποτέλεσμα αυτό υποδεικνύει ότι η διαχωριστική ικανότητα περιορίζεται σε σημαντικό βαθμό από την ίδια την πληροφορία που περιέχεται στις διαθέσιμες κινηματικές μεταβλητές και όχι αποκλειστικά από την επιλογή του αλγορίθμου.

Ιδιαίτερο ενδιαφέρον παρουσιάζει επίσης η μεταβολή της απόδοσης μεταξύ των τριών κατηγοριών γεγονότων. Με τη μετάβαση από την κατηγορία $n_j = 0$ στις κατηγορίες $n_j = 1$ και $n_j \geq 2$, ο αριθμός των διαθέσιμων μεταβλητών αυξάνεται και η κινηματική περιγραφή των γεγονότων γίνεται πλουσιότερη. Θα μπορούσε επομένως αρχικά να αναμένεται ότι η πρόσθετη πληροφορία θα οδηγούσε και σε καλύτερο διαχωρισμό. Τα αποτελέσματα παρουσιάζουν, ωστόσο, την αντίθετη τάση: η υψηλότερη απόδοση επιτυγχάνεται στην κατηγορία $n_j = 0$, ενώ η καλύτερη τιμή της AUC μειώνεται για $n_j = 1$ και περαιτέρω για $n_j \geq 2$.

Η συμπεριφορά αυτή δεν σημαίνει ότι οι μεταβλητές που σχετίζονται με τους πίδακες δεν περιέχουν χρήσιμη διαχωριστική πληροφορία. Οι τρεις κατηγορίες δεν αποτελούν το ίδιο σύνολο γεγονότων στο οποίο προστίθενται διαδοχικά νέες μεταβλητές, αλλά διαφορετικές υποκατηγορίες με διαφορετική κινηματική και τοπολογική δομή. Κατά συνέπεια, η αύξηση της διάστασης του χώρου εισόδου δεν συνεπάγεται από μόνη της ευκολότερο πρόβλημα κατηγοριοποίησης. Η τελική απόδοση καθορίζεται κυρίως από το πόσο διαφορετικές είναι οι κατανομές του σήματος και του υποβάθρου μέσα στη συγκεκριμένη περιοχή του χώρου φάσεων.

Η εξάρτηση της απόδοσης από την επιλογή του ταξινομητή παρουσιάζει επίσης διαφορετική εικόνα μεταξύ των κατηγοριών. Στις κατηγορίες $n_j = 0$ και $n_j = 1$, οι διαφορές μεταξύ των καλύτερων εξεταζόμενων μοντέλων παραμένουν σχετικά περιορισμένες. Αντίθετα, στην κατηγορία $n_j \geq 2$ το BDT παρουσιάζει σαφέστερο πλεονέκτημα έναντι του DNN. Η κατηγορία αυτή χαρακτηρίζεται από μεγαλύτερο αριθμό μεταβλητών και από ισχυρότερες συσχετίσεις μεταξύ ορισμένων από αυτές. Στο συγκεκριμένο πρόβλημα, το BDT φαίνεται επομένως να αξιοποιεί αποτελεσματικότερα τη διαθέσιμη πληροφορία. Το αποτέλεσμα αυτό δείχνει παράλληλα ότι δεν υπάρχει ένας ταξινομητής που να είναι κατ' ανάγκη βέλτιστος για όλες τις κατηγορίες, αλλά ότι η απόδοσή του εξαρτάται από τη δομή των δεδομένων στα οποία εφαρμόζεται.

Η συνολική εικόνα αντανακλά τελικά και τη δυσκολία της αναζήτησης της διάσπασης $H \to \mu^+\mu^-$. Το σήμα και το κυρίαρχο υπόβαθρο $Z/\gamma^* \to \mu^+\mu^-$ οδηγούν στην ίδια βασική τελική κατάσταση δύο μιονίων και παρουσιάζουν σημαντική κινηματική επικάλυψη. Η μηχανική μάθηση μπορεί να συνδυάσει τις μικρές διαφορές που εμφανίζονται ταυτόχρονα σε

πολλές παρατηρήσιμες ποσότητες, δεν μπορεί όμως να δημιουργήσει διαχωριστική πληροφορία που δεν υπάρχει στις μεταβλητές εισόδου. Η σχετικά μέτρια διαχωριστική ικανότητα που παρατηρήθηκε και στις τρεις κατηγορίες είναι επομένως συμβατή με ένα φυσικά απαιτητικό πρόβλημα, στο οποίο η διάκριση σήματος και υποβάθρου βασίζεται στον συνδυασμό πολλών, αλλά όχι απόλυτα διαχωριστικών, κινηματικών χαρακτηριστικών.

Συμπερασματικά, η μελέτη ανέδειξε τη χρησιμότητα των τεχνικών μηχανικής μάθησης στην αξιοποίηση πολλαπλών κινηματικών χαρακτηριστικών για τη διάκριση σπάνιων διεργασιών από κυρίαρχα υπόβαθρα στη φυσική υψηλών ενεργειών. Παράλληλα, έδειξε ότι η αύξηση της πολυπλοκότητας ενός μοντέλου δεν συνεπάγεται κατ΄ ανάγκη καλύτερη απόδοση και ότι η επιλογή του κατάλληλου ταξινομητή και των υπερπαραμέτρων του πρέπει να πραγματοποιείται σε συνάρτηση με τα χαρακτηριστικά της εκάστοτε κατηγορίας γεγονότων.

Η εφαρμογή των διαφορετικών τεχνικών στις τρεις κατηγορίες ανέδειξε επίσης τη σημασία της ανεξάρτητης αξιολόγησης και του ελέγχου της ικανότητας γενίκευσης των μοντέλων. Η διασταυρούμενη επικύρωση, η τελική αξιολόγηση σε ανεξάρτητο σύνολο ελέγχου και η σύγκριση των κατανομών εξόδου αποτέλεσαν συμπληρωματικά εργαλεία για την αξιολόγηση της απόδοσης και της σταθερότητας των ταξινομητών.

Η εργασία παρέχει επομένως ένα ολοκληρωμένο παράδειγμα της διαδρομής από την επιλογή και επεξεργασία φυσικών παρατηρήσιμων μεγεθών έως την εκπαίδευση, βελτιστοποίηση και αξιολόγηση πολυμεταβλητών ταξινομητών σε ένα πραγματικό πρόβλημα Φυσικής Υψηλών Ενεργειών.

Αναφορές

[1] *Machine Learning in Physics: A Short Guide*, Francisco A. Rodrigues: https://iopscience.iop.org/article/10.1209/0295-5075/ad0575/pdf

[2] *Machine Learning in High Energy Physics Community White Paper*, Albertsson, Kim and others: https://arxiv.org/pdf/1807.02876

[3] *TMVA 4 Toolkit for Multivariate Data Analysis with ROOT Users Guide*, A. Hoecker, P. Speckmayer, J. Stelzer, J. Therhaag, E. von Toerne, H. Voss: https://root.cern.ch/download/doc/tmva/TMVAUsersGuide.pdf

[4] *ROOT Software by CERN*: https://root.cern

[5] *ATLAS Open Data*, ATLAS Collaboration: https://atlas.cern/Resources/Opendata

[6] *The ATLAS Experiment at the CERN Large Hadron Collider*, ATLAS Collaboration: https://arxiv.org/pdf/2305.16623

[7] *Machine Learning: An Overview*, Matthew N. O. Sadiku, Samuel A. Ajayi, Janet O. Sadiku: https://www.researchgate.net/publication/397000720_Machine_Learning_An_Overview

[8] *Mathematics for Machine Learning*, Marc Peter Deisenroth, A. Aldo Faisal, and Cheng Soon Ong: https://mml-book.github.io/book/mml-book.pdf

[9] *Deep Learning and Its Application to LHC Physics*, Dan Guest, Kyle Cranmer, and Daniel Whiteson https://arxiv.org/pdf/1806.11484

[10] *The Role of Hyperparameters in Machine Learning Models and How to Tune Them*, Christian Arnold, Luka Biedebach, Andreas Küpfer and Marcel Neunhoeffer: https://doi.org/10.1017/psrm.2023.61

[11] *Deep Learning*, Ian Goodfellow, Yoshua Bengio and Aaron Courville: https://www.deeplearningbook.org/

[12] *Overfitting vs. Underfitting: Finding the Balance*, IBM: https://www.ibm.com/think/topics/overfitting-vs-underfitting

[13] *A Systematic Analysis of Performance Measures for Classification Tasks*, Marina Sokolova, Guy Lapalme: https://doi.org/10.1016/j.ipm.2009.03.002

[14] *An Introduction to ROC Analysis*, Tom Fawcett: https://doi.org/10.1016/j.patrec.2005.10.010

[15] *How to Explain the ROC Curve and ROC AUC Score?*, Evidently AI: https://www.evidentlyai.com/classification-metrics/explain-roc-curve

[16] *The Use of Multiple Measurements in Taxonomic Problems*, R. A. Fisher: https://doi.org/10.1111/j.1469-1809.1936.tb02137.x

[17] *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, Trevor Hastie, Robert Tibshirani and Jerome Friedman: https://doi.org/10.1007/978-0-387-84858-7

[18] *Machine learning with neural networks*, Mehlig Bernhard: https://arxiv.org/pdf/1901.05639

[19] *Activation Functions in Deep Learning: A Comprehensive Survey and Benchmark*, Shiv Ram Dubey, Satish Kumar Singh, Bidyut Baran Chaudhuri: https://doi.org/10.1016/j.neucom.2022.06.111

[20] *Activation Functions in Neural Networks*, GeeksforGeeks: https://www.geeksforgeeks.org/machine-learning/activation-functions-neural-networks/

[21] *Boosted Decision Trees as an Alternative to Artificial Neural Networks for Particle Identification*, Byron P. Roe, Hai-Jun Yang, Ji Zhu, Yong Liu, Ion Stancu and Gordon McGregor: https://arxiv.org/pdf/physics/0408124

[22] *Standard Model of Elementary Particles*, Cush, Wikimedia Commons: https://commons.wikimedia.org/wiki/File:Standard_Model_of_Elementary_Particles.svg

[23] *Review of Particle Physics*, Particle Data Group: https://pdg.lbl.gov/

[24] *The Anatomy of Electro-Weak Symmetry Breaking. I: The Higgs Boson in the Standard Model*, Abdelhak Djouadi: https://arxiv.org/pdf/hep-ph/0503172

[25] *Observation of a New Particle in the Search for the Standard Model Higgs Boson with the ATLAS Detector at the LHC*, ATLAS Collaboration: https://arxiv.org/pdf/1207.7214

[26] *Observation of a New Boson at a Mass of 125 GeV with the CMS Experiment at the LHC*, CMS Collaboration: https://arxiv.org/pdf/1207.7235

[27] *Handbook of LHC Higgs Cross Sections: 4. Deciphering the Nature of the Higgs Sector*, LHC Higgs Cross Section Working Group: https://arxiv.org/pdf/1610.07922

[28] *Status of Higgs Boson Physics*, Particle Data Group: https://pdg.lbl.gov/2023/reviews/rpp2023-rev-higgs-boson.pdf

[29] *Combination of Higgs boson measurements at* $\sqrt{s} = 13$ *TeV and their interpretations by the ATLAS experiment*, ATLAS Collaboration: https://arxiv.org/pdf/2608.07332

[30] *Evidence for the Dimuon Decay of the Higgs Boson in Collisions with the ATLAS Detector*, ATLAS Collaboration: https://arxiv.org/pdf/2507.03595

[31] *ATLAS Coordinate System – ATLAS Cheat Sheet*, ATLAS Collaboration: https://cds.cern.ch/record/2951041

[32] *ATLAS ROOT ntuple format MC simulation, 2015+2016 proton-proton collisions beta release, 2muons skim*, ATLAS Collaboration: https://doi.org/10.7483/OPENDATA.ATLAS.AR66.6RTA

Κώδικες Υλοποίησης

Στο παρόν παράρτημα παρατίθενται οι βασικοί πηγαίοι κώδικες που χρησιμοποιήθηκαν για την ανάλυση των δεδομένων και τη διαμόρφωση των ταξινομητών.

Κώδικας 1: Κατασκευή του συνόλου δεδομένων για $n_j = 0$

```
#include <TFile.h>
#include <TTree.h>
#include <iostream>
#include <vector>
#include <cmath>
#include <map>
#include <algorithm>
const double mMu = 0.105658;
// MC normalization
struct NormInfo { double xsec, kfac, filteff, sumw; };
std::map<unsigned int, NormInfo> normInfo = {
    {345097, {28.3,   1.717, 0.000218, 4247621.5}},
    {345098, {0.1227, 1.0,   0.000275, 2239.9658}},
    {345106, {3.782,  1.0,   0.000275, 448400.06}},
    {700323, {2221.3, 1.0,   0.024394, 1.2741689475072e13}},
    {700324, {2221.3, 1.0,   0.13003,  1.46058376118272e14}},
    {700325, {2221.3, 1.0,   0.846393, 1.19632371174605e15}} };
double GetNormalization(unsigned int channel)
{ auto it = normInfo.find(channel);
    if (it == normInfo.end()) { std::cerr << "ERROR: unknown channelNumber = " << channel << std::endl;
     return 0.0; }
    const NormInfo& n = it->second;
    return n.xsec * n.kfac * n.filteff / n.sumw;
}
// Output variables
struct OutputVariables { float pT_mumu, Y_mumu, cosTheta, m_mumu; ULong64_t eventNumber; int jet_n;
     unsigned int channelNumber; double eventWeight;};
void CreateBranches(TTree* tree, OutputVariables& v)
{
    tree->Branch("pT_mumu", &v.pT_mumu, "pT_mumu/F");
    tree->Branch("Y_mumu", &v.Y_mumu, "Y_mumu/F");
    tree->Branch("cosTheta", &v.cosTheta, "cosTheta/F");
    tree->Branch("m_mumu", &v.m_mumu, "m_mumu/F");
    tree->Branch("eventNumber", &v.eventNumber, "eventNumber/l");
    tree->Branch("jet_n", &v.jet_n, "jet_n/I");
    tree->Branch("channelNumber", &v.channelNumber, "channelNumber/i");
    tree->Branch("eventWeight", &v.eventWeight, "eventWeight/D");
}
// Process one input sample
void ProcessSample(const char* filename, TTree* outputTree,  OutputVariables& out)
{
    TFile* file = TFile::Open(filename);
    if (!file || file->IsZombie()) { std::cerr << "ERROR opening " << filename << std::endl; return; }
    TTree* tree = (TTree*)file->Get("analysis");
    if (!tree) { std::cerr << "ERROR: tree 'analysis' not found in " << filename << std::endl;
        file->Close();  return;}

    // Input branches
    std::vector<float>* lep_pt = nullptr;
    std::vector<float>* lep_eta = nullptr;
    std::vector<float>* lep_phi = nullptr;
    std::vector<int>* lep_type = nullptr;
```

```
std::vector<int>* lep_charge = nullptr;
std::vector<char>* lep_isLooseID = nullptr;
std::vector<char>* lep_isLooseIso = nullptr;
std::vector<char>* lep_isTrigMatched = nullptr;
std::vector<float>* lep_z0 = nullptr;
std::vector<float>* lep_d0sig = nullptr;
int jet_n = 0;
unsigned int channelNumber = 0;
ULong64_t eventNumber = 0;
float mcWeight = 0.0, ScaleFactor_MUON = 0.0;
float ScaleFactor_LepTRIGGER = 0.0, ScaleFactor_PILEUP = 0.0;
tree->SetBranchAddress("lep_pt", &lep_pt);
tree->SetBranchAddress("lep_eta", &lep_eta);
tree->SetBranchAddress("lep_phi", &lep_phi);
tree->SetBranchAddress("lep_type", &lep_type);
tree->SetBranchAddress("lep_charge", &lep_charge);
tree->SetBranchAddress("lep_isLooseID", &lep_isLooseID);
tree->SetBranchAddress("lep_isLooseIso", &lep_isLooseIso);
tree->SetBranchAddress("lep_isTrigMatched", &lep_isTrigMatched);
tree->SetBranchAddress("lep_z0", &lep_z0);
tree->SetBranchAddress("lep_d0sig", &lep_d0sig);
tree->SetBranchAddress("jet_n", &jet_n);
tree->SetBranchAddress("channelNumber", &channelNumber);
tree->SetBranchAddress("eventNumber", &eventNumber);
tree->SetBranchAddress("mcWeight", &mcWeight);
tree->SetBranchAddress("ScaleFactor_MUON", &ScaleFactor_MUON);
tree->SetBranchAddress("ScaleFactor_LepTRIGGER", &ScaleFactor_LepTRIGGER);
tree->SetBranchAddress("ScaleFactor_PILEUP", &ScaleFactor_PILEUP);
Long64_t nEntries = tree->GetEntries();
// Event loop
for (Long64_t entry = 0; entry < nEntries; ++entry) {
    tree->GetEntry(entry);
    // nj = 0
    if (jet_n != 0) continue;
    // Qualifying muons: |type| = 13, pT > 10 GeV, Loose ID
    std::vector<int> goodMuons;
    for (size_t i = 0; i < lep_pt->size(); ++i) {
        if (std::abs(lep_type->at(i)) != 13) continue;
        if (lep_pt->at(i) <= 10.0) continue;
        if (!lep_isLooseID->at(i)) continue;
        goodMuons.push_back(static_cast<int>(i)); }
    if (goodMuons.size() < 2) continue;
    // Highest-pT opposite-sign pair
    int idx1 = -1, idx2 = -1; double bestLeadPt = -1.0, bestSubPt = -1.0;
    for (size_t a = 0; a < goodMuons.size(); ++a) {
        for (size_t b = a + 1; b < goodMuons.size(); ++b) {
            int i = goodMuons[a], j = goodMuons[b];
            if (lep_charge->at(i) * lep_charge->at(j) >= 0) continue;

            double pti = lep_pt->at(i), ptj = lep_pt->at(j);
            double leadPt = std::max(pti, ptj); double subPt = std::min(pti, ptj);

            if (leadPt > bestLeadPt ||
                (leadPt == bestLeadPt && subPt > bestSubPt)) {
                bestLeadPt = leadPt; bestSubPt = subPt;
                idx1 = i; idx2 = j; }}}
    if (idx1 < 0 || idx2 < 0) continue;

    // Final muon selection
    int lead = (lep_pt->at(idx1) >= lep_pt->at(idx2)) ? idx1 : idx2;
    int sub = (lead == idx1) ? idx2 : idx1;
```

```
if (lep_pt->at(lead) <= 27.0) continue;
if (lep_pt->at(sub) <= 15.0) continue;
if (std::abs(lep_eta->at(idx1)) >= 2.5) continue;
if (std::abs(lep_eta->at(idx2)) >= 2.5) continue;

double z01 = lep_z0->at(idx1) / std::cosh(lep_eta->at(idx1));
double z02 = lep_z0->at(idx2) / std::cosh(lep_eta->at(idx2));

if (std::abs(z01) >= 0.5 || std::abs(z02) >= 0.5) continue;
if (std::abs(lep_d0sig->at(idx1)) >= 3.0 || std::abs(lep_d0sig->at(idx2)) >= 3.0) continue;
if (!lep_isLooseIso->at(idx1) || !lep_isLooseIso->at(idx2)) continue;
if (!lep_isTrigMatched->at(idx1) && !lep_isTrigMatched->at(idx2)) continue;

// Additional-lepton veto
bool extraLepton = false;
for (size_t i = 0; i < lep_pt->size(); ++i) {
    if (static_cast<int>(i) == idx1 || static_cast<int>(i) == idx2) continue;
    if (std::abs(lep_type->at(i)) != 11 && std::abs(lep_type->at(i)) != 13) continue;
    if (lep_pt->at(i) <= 7.0) continue;
    if (!lep_isLooseID->at(i)) continue;
    extraLepton = true; break; }
if (extraLepton) continue;

// Muon four-vectors
double pt1 = lep_pt->at(idx1), eta1 = lep_eta->at(idx1);
double phi1 = lep_phi->at(idx1);
double pt2 = lep_pt->at(idx2), eta2 = lep_eta->at(idx2);
double phi2 = lep_phi->at(idx2);
double px1 = pt1 * std::cos(phi1);
double py1 = pt1 * std::sin(phi1);
double pz1 = pt1 * std::sinh(eta1);
double px2 = pt2 * std::cos(phi2);
double py2 = pt2 * std::sin(phi2);
double pz2 = pt2 * std::sinh(eta2);
double E1 = std::sqrt(pt1 * pt1 * std::cosh(eta1) * std::cosh(eta1) + mMu * mMu);
double E2 = std::sqrt(pt2 * pt2 * std::cosh(eta2) *std::cosh(eta2) + mMu * mMu);

// Dimuon four-vector and invariant mass
double px = px1 + px2, py = py1 + py2;
double pz = pz1 + pz2, E = E1 + E2;
double mass2 = E * E - px * px - py * py - pz * pz;
if (mass2 <= 0.0) continue;

double m_mumu = std::sqrt(mass2);
if (m_mumu <= 120.0 || m_mumu >= 130.0) continue;

// Dimuon transverse momentum and rapidity
double pT_mumu = std::sqrt(px * px + py * py);
double rapidityNumerator = E + pz;
double rapidityDenominator = E - pz;
if (rapidityNumerator <= 0.0 || rapidityDenominator <= 0.0) continue;
double Y_mumu =  0.5 * std::log(rapidityNumerator / rapidityDenominator);

// Collins-Soper cos(theta*)
int iMinus = (lep_charge->at(idx1) < 0) ? idx1 : idx2;
int iPlus = (lep_charge->at(idx1) < 0) ? idx2 : idx1;
double ptMinus = lep_pt->at(iMinus);
double etaMinus = lep_eta->at(iMinus);
double ptPlus = lep_pt->at(iPlus);
double etaPlus = lep_eta->at(iPlus);
```

```
        double pzMinus = ptMinus * std::sinh(etaMinus);
        double pzPlus = ptPlus * std::sinh(etaPlus);
        double EMinus = std::sqrt(ptMinus * ptMinus * std::cosh(etaMinus) * std::cosh(etaMinus) + mMu * mMu
    );
        double EPlus = std::sqrt(ptPlus * ptPlus * std::cosh(etaPlus) * std::cosh(etaPlus) + mMu * mMu);
        double P1plus = (EMinus + pzMinus) / std::sqrt(2.0);
        double P1minus = (EMinus - pzMinus) / std::sqrt(2.0);
        double P2plus = (EPlus + pzPlus) / std::sqrt(2.0);
        double P2minus = (EPlus - pzPlus) / std::sqrt(2.0);
        double denominatorCS = std::sqrt( m_mumu * m_mumu *(m_mumu * m_mumu + pT_mumu * pT_mumu));
        if (denominatorCS <= 0.0 || pz == 0.0) continue;

        double signPz = (pz > 0.0) ? 1.0 : -1.0;
        double cosTheta = 2.0 * (P1plus * P2minus -P1minus * P2plus) / denominatorCS * signPz;

        // Numerical sanity check
        if (!std::isfinite(pT_mumu) || !std::isfinite(Y_mumu) ||
            !std::isfinite(cosTheta) || !std::isfinite(m_mumu)) continue;

        // Physical normalized event weight
        double eventWeight =
            static_cast<double>(mcWeight) *
            static_cast<double>(ScaleFactor_MUON) *
            static_cast<double>(ScaleFactor_LepTRIGGER) *
            static_cast<double>(ScaleFactor_PILEUP) *
            GetNormalization(channelNumber);
        if (!std::isfinite(eventWeight)) continue;

        // Fill output tree
        out.pT_mumu = static_cast<float>(pT_mumu);
        out.Y_mumu = static_cast<float>(Y_mumu);
        out.cosTheta = static_cast<float>(cosTheta);
        out.m_mumu = static_cast<float>(m_mumu);
        out.eventNumber = eventNumber;
        out.jet_n = jet_n;
        out.channelNumber = channelNumber;
        out.eventWeight = eventWeight;
        outputTree->Fill(); }
    file->Close(); }

// Main function
void build_nj0_dataset()
{
    // Signal dataset
    TFile* signalFile = new TFile("nj0_signal.root", "RECREATE");
    TTree* signalTree = new TTree("signal", "Final nj=0 signal dataset");
    OutputVariables signalOut;
    CreateBranches(signalTree, signalOut);
    ProcessSample("ggH.root", signalTree, signalOut);
    ProcessSample("ggZH.root", signalTree, signalOut);
    ProcessSample("VBF.root", signalTree, signalOut);
    signalFile->cd();
    signalTree->Write();
    signalFile->Close();
    // Background dataset
    TFile* backgroundFile = new TFile("nj0_background.root", "RECREATE");
    TTree* backgroundTree = new TTree("background", "Final nj=0 background dataset");
    OutputVariables backgroundOut;
    CreateBranches(backgroundTree, backgroundOut);
    ProcessSample("Zskimmed_nj0.root", backgroundTree, backgroundOut);
    backgroundFile->cd();
```

```
    backgroundTree->Write();
    backgroundFile->Close();
}
```

Κώδικας 2: Κατασκευή του συνόλου δεδομένων για $n_j = 1$

```
#include <TFile.h>
#include <TTree.h>
#include <iostream>
#include <vector>
#include <cmath>
#include <map>
#include <algorithm>

const double mMu = 0.105658;

// MC normalization
struct NormInfo { double xsec, kfac, filteff, sumw; };
std::map<unsigned int, NormInfo> normInfo = {
    {345097, {28.3,   1.717, 0.000218, 4247621.5}},
    {345098, {0.1227, 1.0,   0.000275, 2239.9658}},
    {345106, {3.782,  1.0,   0.000275, 448400.06}},
    {700323, {2221.3, 1.0,   0.024394, 1.2741689475072e13}},
    {700324, {2221.3, 1.0,   0.13003,  1.46058376118272e14}},
    {700325, {2221.3, 1.0,   0.846393, 1.19632371174605e15}} };

double GetNormalization(unsigned int channel)
{
    auto it = normInfo.find(channel);
    if (it == normInfo.end()) {std::cerr << "ERROR: unknown channelNumber = " << channel << std::endl;
     return 0.0;}
    const NormInfo& n = it->second;
    return n.xsec * n.kfac * n.filteff / n.sumw;
}
// Output variables
struct OutputVariables { float pT_mumu, Y_mumu, cosTheta; float jet1_pt, jet1_eta, dPhi_mumu_j1, m_mumu;
     ULong64_t eventNumber; int jet_n; unsigned int channelNumber; double eventWeight; };

void CreateBranches(TTree* tree, OutputVariables& v)
{
    tree->Branch("pT_mumu", &v.pT_mumu, "pT_mumu/F");
    tree->Branch("Y_mumu", &v.Y_mumu, "Y_mumu/F");
    tree->Branch("cosTheta", &v.cosTheta, "cosTheta/F");
    tree->Branch("jet1_pt", &v.jet1_pt, "jet1_pt/F");
    tree->Branch("jet1_eta", &v.jet1_eta, "jet1_eta/F");
    tree->Branch("dPhi_mumu_j1", &v.dPhi_mumu_j1, "dPhi_mumu_j1/F");
    tree->Branch("m_mumu", &v.m_mumu, "m_mumu/F");
    tree->Branch("eventNumber", &v.eventNumber, "eventNumber/l");
    tree->Branch("jet_n", &v.jet_n, "jet_n/I");
    tree->Branch("channelNumber", &v.channelNumber, "channelNumber/i");
    tree->Branch("eventWeight", &v.eventWeight, "eventWeight/D");
}
// Process one input sample
void ProcessSample(const char* filename, TTree* outputTree, OutputVariables& out)
{
    TFile* file = TFile::Open(filename);
    if (!file || file->IsZombie()) { std::cerr << "ERROR opening " << filename << std::endl; return; }

    TTree* tree = (TTree*)file->Get("analysis");
```

```
if (!tree) { std::cerr << "ERROR: tree 'analysis' not found in " << filename << std::endl;   file->
 Close(); return; }

// Input branches
std::vector<float>* lep_pt = nullptr;
std::vector<float>* lep_eta = nullptr;
std::vector<float>* lep_phi = nullptr;
std::vector<int>* lep_type = nullptr;
std::vector<int>* lep_charge = nullptr;
std::vector<char>* lep_isLooseID = nullptr;
std::vector<char>* lep_isLooseIso = nullptr;
std::vector<char>* lep_isTrigMatched = nullptr;
std::vector<float>* lep_z0 = nullptr;
std::vector<float>* lep_d0sig = nullptr;
std::vector<float>* jet_pt = nullptr;
std::vector<float>* jet_eta = nullptr;
std::vector<float>* jet_phi = nullptr;

int jet_n = 0; unsigned int channelNumber = 0;
ULong64_t eventNumber = 0;
float mcWeight = 0.0, ScaleFactor_MUON = 0.0;
float ScaleFactor_LepTRIGGER = 0.0, ScaleFactor_PILEUP = 0.0;

tree->SetBranchAddress("lep_pt", &lep_pt);
tree->SetBranchAddress("lep_eta", &lep_eta);
tree->SetBranchAddress("lep_phi", &lep_phi);
tree->SetBranchAddress("lep_type", &lep_type);
tree->SetBranchAddress("lep_charge", &lep_charge);
tree->SetBranchAddress("lep_isLooseID", &lep_isLooseID);
tree->SetBranchAddress("lep_isLooseIso", &lep_isLooseIso);
tree->SetBranchAddress("lep_isTrigMatched", &lep_isTrigMatched);
tree->SetBranchAddress("lep_z0", &lep_z0);
tree->SetBranchAddress("lep_d0sig", &lep_d0sig);
tree->SetBranchAddress("jet_n", &jet_n);
tree->SetBranchAddress("jet_pt", &jet_pt);
tree->SetBranchAddress("jet_eta", &jet_eta);
tree->SetBranchAddress("jet_phi", &jet_phi);
tree->SetBranchAddress("channelNumber", &channelNumber);
tree->SetBranchAddress("eventNumber", &eventNumber);
tree->SetBranchAddress("mcWeight", &mcWeight);
tree->SetBranchAddress("ScaleFactor_MUON", &ScaleFactor_MUON);
tree->SetBranchAddress("ScaleFactor_LepTRIGGER", &ScaleFactor_LepTRIGGER);
tree->SetBranchAddress("ScaleFactor_PILEUP", &ScaleFactor_PILEUP);
Long64_t nEntries = tree->GetEntries();

// Event loop
for (Long64_t entry = 0; entry < nEntries; ++entry) { tree->GetEntry(entry);

    // nj = 1 and jet-vector safety
    if (jet_n != 1) continue;
    if (!jet_pt || !jet_eta || !jet_phi) continue;
    if (jet_pt->size() < 1 || jet_eta->size() < 1 || jet_phi->size() < 1) continue;
    if (jet_pt->size() != jet_eta->size() || jet_pt->size() != jet_phi->size()) continue;

    // Qualifying muons: |type| = 13, pT > 10 GeV, Loose ID
    std::vector<int> goodMuons;
    for (size_t i = 0; i < lep_pt->size(); ++i) {
        if (std::abs(lep_type->at(i)) != 13) continue;
        if (lep_pt->at(i) <= 10.0) continue;
        if (!lep_isLooseID->at(i)) continue;
        goodMuons.push_back(static_cast<int>(i)); }
```

```
if (goodMuons.size() < 2) continue;

// Highest-pT opposite-sign pair
int idx1 = -1, idx2 = -1;
double bestLeadPt = -1.0, bestSubPt = -1.0;

for (size_t a = 0; a < goodMuons.size(); ++a) {
    for (size_t b = a + 1; b < goodMuons.size(); ++b) {
        int i = goodMuons[a], j = goodMuons[b];
        if (lep_charge->at(i) * lep_charge->at(j) >= 0) continue;

        double pti = lep_pt->at(i), ptj = lep_pt->at(j);
        double leadPt = std::max(pti, ptj), subPt = std::min(pti, ptj);

        if (leadPt > bestLeadPt ||
            (leadPt == bestLeadPt && subPt > bestSubPt)) {
            bestLeadPt = leadPt; bestSubPt = subPt;
            idx1 = i; idx2 = j; }}}
if (idx1 < 0 || idx2 < 0) continue;

// Final muon selection
int lead = (lep_pt->at(idx1) >= lep_pt->at(idx2)) ? idx1 : idx2;
int sub = (lead == idx1) ? idx2 : idx1;

if (lep_pt->at(lead) <= 27.0) continue;
if (lep_pt->at(sub) <= 15.0) continue;
if (std::abs(lep_eta->at(idx1)) >= 2.5) continue;
if (std::abs(lep_eta->at(idx2)) >= 2.5) continue;

double z01 = lep_z0->at(idx1) / std::cosh(lep_eta->at(idx1));
double z02 = lep_z0->at(idx2) / std::cosh(lep_eta->at(idx2));

if (std::abs(z01) >= 0.5 || std::abs(z02) >= 0.5) continue;
if (std::abs(lep_d0sig->at(idx1)) >= 3.0 || std::abs(lep_d0sig->at(idx2)) >= 3.0) continue;
if (!lep_isLooseIso->at(idx1) || !lep_isLooseIso->at(idx2)) continue;
if (!lep_isTrigMatched->at(idx1) && !lep_isTrigMatched->at(idx2)) continue;

// Additional-lepton veto
bool extraLepton = false;
for (size_t i = 0; i < lep_pt->size(); ++i) {
    if (static_cast<int>(i) == idx1 || static_cast<int>(i) == idx2) continue;
    if (std::abs(lep_type->at(i)) != 11 && std::abs(lep_type->at(i)) != 13) continue;
    if (lep_pt->at(i) <= 7.0) continue;
    if (!lep_isLooseID->at(i)) continue;
    extraLepton = true; break; }
if (extraLepton) continue;

// Muon four-vectors
double pt1 = lep_pt->at(idx1), eta1 = lep_eta->at(idx1), phi1 = lep_phi->at(idx1);
double pt2 = lep_pt->at(idx2), eta2 = lep_eta->at(idx2), phi2 = lep_phi->at(idx2);
double px1 = pt1 * std::cos(phi1), py1 = pt1 * std::sin(phi1), pz1 = pt1 * std::sinh(eta1);
double px2 = pt2 * std::cos(phi2), py2 = pt2 * std::sin(phi2), pz2 = pt2 * std::sinh(eta2);
double E1 = std::sqrt(pt1 * pt1 * std::cosh(eta1) * std::cosh(eta1) + mMu * mMu);
double E2 = std::sqrt(pt2 * pt2 * std::cosh(eta2) * std::cosh(eta2) + mMu * mMu);

// Dimuon four-vector and invariant mass
double px = px1 + px2, py = py1 + py2, pz = pz1 + pz2, E = E1 + E2;
double mass2 = E * E - px * px - py * py - pz * pz;
if (mass2 <= 0.0) continue;

double m_mumu = std::sqrt(mass2);
```

```
    if (m_mumu <= 120.0 || m_mumu >= 130.0) continue;

    // Dimuon transverse momentum and rapidity
    double pT_mumu = std::sqrt(px * px + py * py);
    double rapidityNumerator = E + pz, rapidityDenominator = E - pz;
    if (rapidityNumerator <= 0.0 || rapidityDenominator <= 0.0) continue;
    double Y_mumu = 0.5 * std::log(rapidityNumerator / rapidityDenominator);

    // Collins-Soper cos(theta*)
    int iMinus = (lep_charge->at(idx1) < 0) ? idx1 : idx2;
    int iPlus = (lep_charge->at(idx1) < 0) ? idx2 : idx1;
    double ptMinus = lep_pt->at(iMinus), etaMinus = lep_eta->at(iMinus);
    double ptPlus = lep_pt->at(iPlus), etaPlus = lep_eta->at(iPlus);
    double pzMinus = ptMinus * std::sinh(etaMinus), pzPlus = ptPlus * std::sinh(etaPlus);
    double EMinus = std::sqrt(ptMinus * ptMinus * std::cosh(etaMinus) * std::cosh(etaMinus) + mMu * mMu
);
    double EPlus = std::sqrt(ptPlus * ptPlus * std::cosh(etaPlus) * std::cosh(etaPlus) + mMu * mMu);
    double P1plus = (EMinus + pzMinus) / std::sqrt(2.0);
    double P1minus = (EMinus - pzMinus) / std::sqrt(2.0);
    double P2plus = (EPlus + pzPlus) / std::sqrt(2.0);
    double P2minus = (EPlus - pzPlus) / std::sqrt(2.0);
    double denominatorCS = std::sqrt(m_mumu * m_mumu *(m_mumu * m_mumu + pT_mumu * pT_mumu));
    if (denominatorCS <= 0.0 || pz == 0.0) continue;

    double signPz = (pz > 0.0) ? 1.0 : -1.0;
    double cosTheta = 2.0 * (P1plus * P2minus - P1minus * P2plus) / denominatorCS * signPz;

    // Jet variables
    double jet1_pt = jet_pt->at(0), jet1_eta = jet_eta->at(0);
    double jet1_phi = jet_phi->at(0);
    double phi_mumu = std::atan2(py, px);
    double rawDPhi = phi_mumu - jet1_phi;
    double wrappedDPhi = std::atan2(std::sin(rawDPhi), std::cos(rawDPhi));
    double dPhi_mumu_j1 = std::abs(wrappedDPhi);

    // Numerical sanity check
    if (!std::isfinite(pT_mumu) || !std::isfinite(Y_mumu) ||
        !std::isfinite(cosTheta) || !std::isfinite(jet1_pt) ||
        !std::isfinite(jet1_eta) || !std::isfinite(dPhi_mumu_j1) ||
        !std::isfinite(m_mumu)) continue;

    // Physical normalized event weight
    double eventWeight =
        static_cast<double>(mcWeight) *
        static_cast<double>(ScaleFactor_MUON) *
        static_cast<double>(ScaleFactor_LepTRIGGER) *
        static_cast<double>(ScaleFactor_PILEUP) *
        GetNormalization(channelNumber);

    if (!std::isfinite(eventWeight)) continue;

    // Fill output tree
    out.pT_mumu = static_cast<float>(pT_mumu);
    out.Y_mumu = static_cast<float>(Y_mumu);
    out.cosTheta = static_cast<float>(cosTheta);
    out.jet1_pt = static_cast<float>(jet1_pt);
    out.jet1_eta = static_cast<float>(jet1_eta);
    out.dPhi_mumu_j1 = static_cast<float>(dPhi_mumu_j1);
    out.m_mumu = static_cast<float>(m_mumu);
    out.eventNumber = eventNumber; out.jet_n = jet_n;
    out.channelNumber = channelNumber; out.eventWeight = eventWeight;
```

```
        outputTree->Fill();
    }

    file->Close();
}

// Main function
void build_nj1_dataset()
{
    // Signal dataset
    TFile* signalFile = new TFile("nj1_signal.root", "RECREATE");
    TTree* signalTree = new TTree("signal", "Final nj=1 signal dataset");
    OutputVariables signalOut;
    CreateBranches(signalTree, signalOut);
    ProcessSample("ggH.root", signalTree, signalOut);
    ProcessSample("ggZH.root", signalTree, signalOut);
    ProcessSample("VBF.root", signalTree, signalOut);
    signalFile->cd(); signalTree->Write(); signalFile->Close();
    // Background dataset
    TFile* backgroundFile = new TFile("nj1_background.root", "RECREATE");
    TTree* backgroundTree = new TTree("background", "Final nj=1 background dataset");
    OutputVariables backgroundOut;
    CreateBranches(backgroundTree, backgroundOut);
    ProcessSample("Zskimmed_nj1.root", backgroundTree, backgroundOut);
    backgroundFile->cd(); backgroundTree->Write(); backgroundFile->Close();
}
```

Κώδικας 3: Κατασκευή του συνόλου δεδομένων για $n_j \geq 2$

```
#include <TFile.h>
#include <TTree.h>
#include <iostream>
#include <vector>
#include <cmath>
#include <algorithm>

const double mMu = 0.105658;

// MC normalization
double GetNormalization(unsigned int channelNumber)
{
    double xsec = 0.0, kfac = 1.0, filteff = 0.0, sumw = 0.0;

    switch (channelNumber) {
        case 345097: xsec=28.3;   kfac=1.717; filteff=0.000218; sumw=4247621.5; break;
        case 345098: xsec=0.1227; kfac=1.0;   filteff=0.000275; sumw=2239.9658; break;
        case 345106: xsec=3.782;  kfac=1.0;   filteff=0.000275; sumw=448400.06; break;
        case 700323: xsec=2221.3; kfac=1.0; filteff=0.024394; sumw=1.2741689475072e13; break;
        case 700324: xsec=2221.3; kfac=1.0; filteff=0.13003; sumw=1.46058376118272e14; break;
        case 700325: xsec=2221.3; kfac=1.0; filteff=0.846393; sumw=1.19632371174605e15; break;
        default:
            std::cerr << "Unknown channelNumber: " << channelNumber << std::endl;
            return 0.0;
    }
    return xsec * kfac * filteff / sumw;
}

double DeltaPhi(double phi1, double phi2)
{
```

```
    double dphi = phi1 - phi2;
    return std::abs(std::atan2(std::sin(dphi), std::cos(dphi)));
}

// Output variables
struct OutputVariables {
    float pT_mumu, Y_mumu, cosTheta;
    float jet1_pt, jet1_eta, dPhi_j1_mumu;
    float jet2_pt, jet2_eta, dPhi_j2_mumu;
    float pT_jj, Y_jj, dPhi_jj_mumu;
    float m_jj, met, HT, m_mumu;
    ULong64_t eventNumber; int jet_n;
    unsigned int channelNumber; double eventWeight;
};

void CreateBranches(TTree* tree, OutputVariables& v)
{
    tree->Branch("pT_mumu", &v.pT_mumu, "pT_mumu/F");
    tree->Branch("Y_mumu", &v.Y_mumu, "Y_mumu/F");
    tree->Branch("cosTheta", &v.cosTheta, "cosTheta/F");
    tree->Branch("jet1_pt", &v.jet1_pt, "jet1_pt/F");
    tree->Branch("jet1_eta", &v.jet1_eta, "jet1_eta/F");
    tree->Branch("dPhi_j1_mumu", &v.dPhi_j1_mumu, "dPhi_j1_mumu/F");
    tree->Branch("jet2_pt", &v.jet2_pt, "jet2_pt/F");
    tree->Branch("jet2_eta", &v.jet2_eta, "jet2_eta/F");
    tree->Branch("dPhi_j2_mumu", &v.dPhi_j2_mumu, "dPhi_j2_mumu/F");
    tree->Branch("pT_jj", &v.pT_jj, "pT_jj/F");
    tree->Branch("Y_jj", &v.Y_jj, "Y_jj/F");
    tree->Branch("dPhi_jj_mumu", &v.dPhi_jj_mumu, "dPhi_jj_mumu/F");
    tree->Branch("m_jj", &v.m_jj, "m_jj/F");
    tree->Branch("met", &v.met, "met/F");
    tree->Branch("HT", &v.HT, "HT/F");
    tree->Branch("m_mumu", &v.m_mumu, "m_mumu/F");
    tree->Branch("eventNumber", &v.eventNumber, "eventNumber/l");
    tree->Branch("jet_n", &v.jet_n, "jet_n/I");
    tree->Branch("channelNumber", &v.channelNumber, "channelNumber/i");
    tree->Branch("eventWeight", &v.eventWeight, "eventWeight/D");
}

// Process one input sample
void ProcessFile(const char* inputFile, TTree* outputTree, OutputVariables& out)
{
    TFile* file = TFile::Open(inputFile, "READ");
    if (!file || file->IsZombie()) {
        std::cerr << "Cannot open " << inputFile << std::endl; return; }

    TTree* tree = (TTree*)file->Get("analysis");
    if (!tree) { std::cerr << "Tree 'analysis' not found in " << inputFile << std::endl;
        file->Close(); return; }

    // Input branches
    std::vector<float>* lep_pt = nullptr;
    std::vector<float>* lep_eta = nullptr;
    std::vector<float>* lep_phi = nullptr;
    std::vector<int>* lep_type = nullptr;
    std::vector<int>* lep_charge = nullptr;
    std::vector<char>* lep_isLooseID = nullptr;
    std::vector<char>* lep_isLooseIso = nullptr;
    std::vector<char>* lep_isTrigMatched = nullptr;
    std::vector<float>* lep_z0 = nullptr;
    std::vector<float>* lep_d0sig = nullptr;
```

```
std::vector<float>* jet_pt = nullptr;
std::vector<float>* jet_eta = nullptr;
std::vector<float>* jet_phi = nullptr;
std::vector<float>* jet_e = nullptr;

int jet_n = 0; float met = 0.0;
ULong64_t eventNumber = 0; unsigned int channelNumber = 0;
float mcWeight = 0.0, ScaleFactor_MUON = 0.0;
float ScaleFactor_LepTRIGGER = 0.0, ScaleFactor_PILEUP = 0.0;

tree->SetBranchAddress("lep_pt", &lep_pt);
tree->SetBranchAddress("lep_eta", &lep_eta);
tree->SetBranchAddress("lep_phi", &lep_phi);
tree->SetBranchAddress("lep_type", &lep_type);
tree->SetBranchAddress("lep_charge", &lep_charge);
tree->SetBranchAddress("lep_isLooseID", &lep_isLooseID);
tree->SetBranchAddress("lep_isLooseIso", &lep_isLooseIso);
tree->SetBranchAddress("lep_isTrigMatched", &lep_isTrigMatched);
tree->SetBranchAddress("lep_z0", &lep_z0);
tree->SetBranchAddress("lep_d0sig", &lep_d0sig);
tree->SetBranchAddress("jet_n", &jet_n);
tree->SetBranchAddress("jet_pt", &jet_pt);
tree->SetBranchAddress("jet_eta", &jet_eta);
tree->SetBranchAddress("jet_phi", &jet_phi);
tree->SetBranchAddress("jet_e", &jet_e);
tree->SetBranchAddress("met", &met);
tree->SetBranchAddress("eventNumber", &eventNumber);
tree->SetBranchAddress("channelNumber", &channelNumber);
tree->SetBranchAddress("mcWeight", &mcWeight);
tree->SetBranchAddress("ScaleFactor_MUON", &ScaleFactor_MUON);
tree->SetBranchAddress("ScaleFactor_LepTRIGGER", &ScaleFactor_LepTRIGGER);
tree->SetBranchAddress("ScaleFactor_PILEUP", &ScaleFactor_PILEUP);

Long64_t nEntries = tree->GetEntries();

// Event loop
for (Long64_t entry = 0; entry < nEntries; ++entry) { tree->GetEntry(entry);

    // nj >= 2 and jet-vector safety
    if (jet_n < 2) continue;
    if (!jet_pt || !jet_eta || !jet_phi || !jet_e) continue;
    if (jet_pt->size() < 2 ||
        jet_pt->size() != jet_eta->size() ||
        jet_pt->size() != jet_phi->size() ||
        jet_pt->size() != jet_e->size()) continue;

    // Two highest-pT jets
    int j1 = -1, j2 = -1;
    for (size_t j = 0; j < jet_pt->size(); ++j) {
        if (j1 < 0 || jet_pt->at(j) > jet_pt->at(j1)) {
            j2 = j1; j1 = static_cast<int>(j);
        } else if (j2 < 0 || jet_pt->at(j) > jet_pt->at(j2))
            j2 = static_cast<int>(j);
    }
    if (j1 < 0 || j2 < 0) continue;

    // Qualifying muons
    std::vector<int> goodMuons;
    for (size_t i = 0; i < lep_pt->size(); ++i) {
        if (std::abs(lep_type->at(i)) != 13) continue;
        if (lep_pt->at(i) <= 10.0) continue;
```

```
    if (!lep_isLooseID->at(i)) continue;
    goodMuons.push_back(static_cast<int>(i)); }
if (goodMuons.size() < 2) continue;

// Highest-pT opposite-sign dimuon candidate
int idx1 = -1, idx2 = -1;
double bestLeadPt = -1.0, bestSubPt = -1.0;

for (size_t a = 0; a < goodMuons.size(); ++a) {
    for (size_t b = a + 1; b < goodMuons.size(); ++b) {
        int i = goodMuons[a], j = goodMuons[b];
        if (lep_charge->at(i) * lep_charge->at(j) >= 0) continue;

        double pti = lep_pt->at(i), ptj = lep_pt->at(j);
        double leadPt = std::max(pti, ptj), subPt = std::min(pti, ptj);

        if (leadPt > bestLeadPt ||
            (leadPt == bestLeadPt && subPt > bestSubPt)) {
            bestLeadPt = leadPt; bestSubPt = subPt;
            idx1 = i; idx2 = j; }}}
if (idx1 < 0 || idx2 < 0) continue;

// Final muon selection
int lead = (lep_pt->at(idx1) >= lep_pt->at(idx2)) ? idx1 : idx2;
int sub = (lead == idx1) ? idx2 : idx1;

if (lep_pt->at(lead) <= 27.0 || lep_pt->at(sub) <= 15.0) continue;
if (std::abs(lep_eta->at(idx1)) >= 2.5 ||
    std::abs(lep_eta->at(idx2)) >= 2.5) continue;

double z01 = lep_z0->at(idx1) / std::cosh(lep_eta->at(idx1));
double z02 = lep_z0->at(idx2) / std::cosh(lep_eta->at(idx2));

if (std::abs(z01) >= 0.5 || std::abs(z02) >= 0.5) continue;
if (std::abs(lep_d0sig->at(idx1)) >= 3.0 || std::abs(lep_d0sig->at(idx2)) >= 3.0) continue;
if (!lep_isLooseIso->at(idx1) || !lep_isLooseIso->at(idx2)) continue;
if (!lep_isTrigMatched->at(idx1) && !lep_isTrigMatched->at(idx2)) continue;

// Additional-lepton veto
bool extraLepton = false;
for (size_t i = 0; i < lep_pt->size(); ++i) {
    if (static_cast<int>(i) == idx1 || static_cast<int>(i) == idx2) continue;
    int absType = std::abs(lep_type->at(i));
    if (absType != 11 && absType != 13) continue;
    if (lep_pt->at(i) <= 7.0 || !lep_isLooseID->at(i)) continue;
    extraLepton = true; break; }
if (extraLepton) continue;

// Dimuon four-vector
double pt1=lep_pt->at(idx1), eta1=lep_eta->at(idx1), phi1=lep_phi->at(idx1);
double pt2=lep_pt->at(idx2), eta2=lep_eta->at(idx2), phi2=lep_phi->at(idx2);
double px1=pt1*std::cos(phi1), py1=pt1*std::sin(phi1), pz1=pt1*std::sinh(eta1);
double px2=pt2*std::cos(phi2), py2=pt2*std::sin(phi2), pz2=pt2*std::sinh(eta2);
double E1=std::sqrt(pt1*pt1*std::cosh(eta1)*std::cosh(eta1)+mMu*mMu);
double E2=std::sqrt(pt2*pt2*std::cosh(eta2)*std::cosh(eta2)+mMu*mMu);
double px_mumu=px1+px2, py_mumu=py1+py2, pz_mumu=pz1+pz2, E_mumu=E1+E2;
double mass2=E_mumu*E_mumu-px_mumu*px_mumu-py_mumu*py_mumu-pz_mumu*pz_mumu;
if (mass2 <= 0.0) continue;

double m_mumu = std::sqrt(mass2);
if (m_mumu <= 120.0 || m_mumu >= 130.0) continue;
```

```
// Dimuon variables
double pT_mumu = std::sqrt(px_mumu*px_mumu + py_mumu*py_mumu);
double rapidityPlus=E_mumu+pz_mumu, rapidityMinus=E_mumu-pz_mumu;
if (rapidityPlus <= 0.0 || rapidityMinus <= 0.0) continue;
double Y_mumu = 0.5 * std::log(rapidityPlus / rapidityMinus);
double phi_mumu = std::atan2(py_mumu, px_mumu);

// Collins-Soper cos(theta*)
int iMinus = (lep_charge->at(idx1) < 0) ? idx1 : idx2;
int iPlus = (lep_charge->at(idx1) < 0) ? idx2 : idx1;
double ptMinus=lep_pt->at(iMinus), etaMinus=lep_eta->at(iMinus);
double ptPlus=lep_pt->at(iPlus), etaPlus=lep_eta->at(iPlus);
double pzMinus=ptMinus*std::sinh(etaMinus), pzPlus=ptPlus*std::sinh(etaPlus);
double EMinus=std::sqrt(ptMinus*ptMinus*std::cosh(etaMinus)*std::cosh(etaMinus)+mMu*mMu);
double EPlus=std::sqrt(ptPlus*ptPlus*std::cosh(etaPlus)*std::cosh(etaPlus)+mMu*mMu);
double P1plus=(EMinus+pzMinus)/std::sqrt(2.0), P1minus=(EMinus-pzMinus)/std::sqrt(2.0);
double P2plus=(EPlus+pzPlus)/std::sqrt(2.0), P2minus=(EPlus-pzPlus)/std::sqrt(2.0);
double denominatorCS=std::sqrt(m_mumu*m_mumu*(m_mumu*m_mumu+pT_mumu*pT_mumu));
if (denominatorCS <= 0.0 || !std::isfinite(denominatorCS)) continue;

double signPz = 0.0;
if (pz_mumu > 0.0) signPz = 1.0;
else if (pz_mumu < 0.0) signPz = -1.0;

double cosTheta = 2.0*(P1plus*P2minus-P1minus*P2plus) / denominatorCS * signPz;

// Leading jets
double jet1_pt=jet_pt->at(j1), jet1_eta=jet_eta->at(j1);
double jet1_phi=jet_phi->at(j1), jet1_E=jet_e->at(j1);
double jet2_pt=jet_pt->at(j2), jet2_eta=jet_eta->at(j2);
double jet2_phi=jet_phi->at(j2), jet2_E=jet_e->at(j2);
double dPhi_j1_mumu=DeltaPhi(jet1_phi,phi_mumu);
double dPhi_j2_mumu=DeltaPhi(jet2_phi,phi_mumu);

// Dijet variables
double j1_px=jet1_pt*std::cos(jet1_phi), j1_py=jet1_pt*std::sin(jet1_phi);
double j1_pz=jet1_pt*std::sinh(jet1_eta);
double j2_px=jet2_pt*std::cos(jet2_phi), j2_py=jet2_pt*std::sin(jet2_phi);
double j2_pz=jet2_pt*std::sinh(jet2_eta);
double jj_px=j1_px+j2_px, jj_py=j1_py+j2_py;
double jj_pz=j1_pz+j2_pz, jj_E=jet1_E+jet2_E;
double pT_jj=std::sqrt(jj_px*jj_px+jj_py*jj_py);
double jjRapidityPlus=jj_E+jj_pz, jjRapidityMinus=jj_E-jj_pz;
if (jjRapidityPlus <= 0.0 || jjRapidityMinus <= 0.0) continue;

double Y_jj=0.5*std::log(jjRapidityPlus/jjRapidityMinus);
double phi_jj=std::atan2(jj_py,jj_px);
double dPhi_jj_mumu=DeltaPhi(phi_jj,phi_mumu);
double m2_jj=jj_E*jj_E-jj_px*jj_px-jj_py*jj_py-jj_pz*jj_pz;

if (m2_jj < 0.0 && m2_jj > -1e-6) m2_jj=0.0;
if (m2_jj < 0.0) continue;
double m_jj=std::sqrt(m2_jj);

// HT = scalar sum of pT of all stored jets
double HT=0.0;
for (size_t j=0; j<jet_pt->size(); ++j) HT += jet_pt->at(j);

// Event weight
double eventWeight =
```

```
            static_cast<double>(mcWeight) *
            static_cast<double>(ScaleFactor_MUON) *
            static_cast<double>(ScaleFactor_LepTRIGGER) *
            static_cast<double>(ScaleFactor_PILEUP) *
            GetNormalization(channelNumber);

        // Final numerical safety check
        bool finite =
            std::isfinite(pT_mumu) && std::isfinite(Y_mumu) &&
            std::isfinite(cosTheta) && std::isfinite(jet1_pt) &&
            std::isfinite(jet1_eta) && std::isfinite(dPhi_j1_mumu) &&
            std::isfinite(jet2_pt) && std::isfinite(jet2_eta) &&
            std::isfinite(dPhi_j2_mumu) && std::isfinite(pT_jj) &&
            std::isfinite(Y_jj) && std::isfinite(dPhi_jj_mumu) &&
            std::isfinite(m_jj) && std::isfinite(met) &&
            std::isfinite(HT) && std::isfinite(m_mumu) &&
            std::isfinite(eventWeight);
        if (!finite) continue;

        // Fill output tree
        out.pT_mumu=static_cast<float>(pT_mumu);
        out.Y_mumu=static_cast<float>(Y_mumu);
        out.cosTheta=static_cast<float>(cosTheta);
        out.jet1_pt=static_cast<float>(jet1_pt);
        out.jet1_eta=static_cast<float>(jet1_eta);
        out.dPhi_j1_mumu=static_cast<float>(dPhi_j1_mumu);
        out.jet2_pt=static_cast<float>(jet2_pt);
        out.jet2_eta=static_cast<float>(jet2_eta);
        out.dPhi_j2_mumu=static_cast<float>(dPhi_j2_mumu);
        out.pT_jj=static_cast<float>(pT_jj);
        out.Y_jj=static_cast<float>(Y_jj);
        out.dPhi_jj_mumu=static_cast<float>(dPhi_jj_mumu);
        out.m_jj=static_cast<float>(m_jj); out.met=met;
        out.HT=static_cast<float>(HT); out.m_mumu=static_cast<float>(m_mumu);
        out.eventNumber=eventNumber; out.jet_n=jet_n;
        out.channelNumber=channelNumber; out.eventWeight=eventWeight;
        outputTree->Fill();
    }

    file->Close();
    delete file;
}

// Main function
void build_nj2_dataset()
{
    // Signal dataset
    TFile* signalFile=TFile::Open("nj2_signal.root","RECREATE");
    TTree* signalTree=new TTree("signal","nj >= 2 signal");
    OutputVariables signalOut;
    CreateBranches(signalTree,signalOut);
    ProcessFile("ggH.root",signalTree,signalOut);
    ProcessFile("ggZH.root",signalTree,signalOut);
    ProcessFile("VBF.root",signalTree,signalOut);
    signalFile->cd(); signalTree->Write(); signalFile->Close();

    // Background dataset
    TFile* backgroundFile=TFile::Open("nj2_background.root","RECREATE");
    TTree* backgroundTree=new TTree("background","nj >= 2 background");
    OutputVariables backgroundOut;
    CreateBranches(backgroundTree,backgroundOut);
```

```
    ProcessFile("Zskimmed_nj2.root",backgroundTree,backgroundOut);
    backgroundFile->cd(); backgroundTree->Write(); backgroundFile->Close();
}
```

Κώδικας 4: Εκπαίδευση του ταξινομητή Fisher

```
#include "TFile.h"
#include "TTree.h"
#include "TMVA/Tools.h"
#include "TMVA/Factory.h"
#include "TMVA/DataLoader.h"
#include "TMVA/Types.h"

void Fisher()
{
    TMVA::Tools::Instance();

    TFile* sigFile = TFile::Open("nj0_signal.root");
    TFile* bkgFile = TFile::Open("nj0_background.root");
    TTree* sigTree = (TTree*)sigFile->Get("signal");
    TTree* bkgTree = (TTree*)bkgFile->Get("background");
    TFile* outputFile = TFile::Open("Fisher_results.root", "RECREATE");

    TMVA::Factory* factory = new TMVA::Factory(
        "Fisher", outputFile,
        "!V:!Silent:Color:DrawProgressBar:"
        "Transformations=I:AnalysisType=Classification");

    TMVA::DataLoader* loader = new TMVA::DataLoader("dataset_Fisher");

    // Input variables
    loader->AddVariable("pT_mumu", 'F');
    loader->AddVariable("Y_mumu", 'F');
    loader->AddVariable("cosTheta", 'F');

    // Spectator variables
    loader->AddSpectator("m_mumu", 'F');
    loader->AddSpectator("eventNumber", 'l');
    loader->AddSpectator("jet_n", 'I');

    // Signal, background and event weights
    loader->AddSignalTree(sigTree, 1.0);
    loader->AddBackgroundTree(bkgTree, 1.0);
    loader->SetSignalWeightExpression("eventWeight");
    loader->SetBackgroundWeightExpression("eventWeight");

    // Fixed 80/20 training-test split
    loader->PrepareTrainingAndTestTree("", "",
        "nTrain_Signal=46273:nTest_Signal=11569:"
        "nTrain_Background=31770:nTest_Background=7943:"
        "SplitMode=Random:SplitSeed=12345:"
        "NormMode=NumEvents:!V");

    // Fisher classifier
    factory->BookMethod(
        loader, TMVA::Types::kFisher, "Fisher",
        "!H:!V:Fisher:VarTransform=None:" "IgnoreNegWeightsInTraining");

    factory->TrainAllMethods();
```

```
    factory->TestAllMethods();
    factory->EvaluateAllMethods();

    outputFile->Write();
    outputFile->Close();

    delete factory;
    delete loader;
    sigFile->Close();
    bkgFile->Close();
}
```

Κώδικας 5: Βελτιστοποίηση και εκπαίδευση του ταξινομητή Likelihood

```
#include <iostream>
#include <vector>
#include "TFile.h"
#include "TTree.h"
#include "TString.h"
#include "TSystem.h"
#include "TMVA/Tools.h"
#include "TMVA/Factory.h"
#include "TMVA/DataLoader.h"
#include "TMVA/CrossValidation.h"
#include "TMVA/Types.h"

void Likelihood()
{
    TMVA::Tools::Instance();
    std::vector<int> nBinsValues = {50, 100, 150, 200, 250, 300};

    TFile* sigFile = TFile::Open("nj0_signal.root");
    TFile* bkgFile = TFile::Open("nj0_background.root");
    if (!sigFile || sigFile->IsZombie() || !bkgFile || bkgFile->IsZombie()) {
        std::cerr << "ERROR opening input files." << std::endl; return;}

    TTree* sigTree = (TTree*)sigFile->Get("signal");
    TTree* bkgTree = (TTree*)bkgFile->Get("background");
    if (!sigTree || !bkgTree) {
        std::cerr << "ERROR loading input trees." << std::endl;
        sigFile->Close(); bkgFile->Close(); return;}

    // 5-fold cross-validation
    TFile* cvFile = TFile::Open("Likelihood_CV_results.root", "RECREATE");
    if (!cvFile || cvFile->IsZombie()) {
        std::cerr << "ERROR creating CV output file." << std::endl;
        sigFile->Close(); bkgFile->Close(); return;}

    TMVA::DataLoader* cvLoader = new TMVA::DataLoader("cv_Likelihood");
    cvLoader->AddVariable("pT_mumu", 'F');
    cvLoader->AddVariable("Y_mumu", 'F');
    cvLoader->AddVariable("cosTheta", 'F');
    cvLoader->AddSpectator("m_mumu", 'F');
    cvLoader->AddSpectator("eventNumber", 'l');
    cvLoader->AddSpectator("jet_n", 'I');
    cvLoader->AddSignalTree(sigTree, 1.0);
    cvLoader->AddBackgroundTree(bkgTree, 1.0);
    cvLoader->SetSignalWeightExpression("eventWeight");
    cvLoader->SetBackgroundWeightExpression("eventWeight");
```

```
cvLoader->PrepareTrainingAndTestTree("", "",
    "nTrain_Signal=46273:nTest_Signal=11569:"
    "nTrain_Background=31770:nTest_Background=7943:"
    "SplitMode=Random:SplitSeed=12345:"
    "NormMode=NumEvents:!V");

TString cvOptions =
    "!V:!Silent:!ModelPersistence:AnalysisType=Classification:"
    "SplitType=RandomStratified:NumFolds=5";

TMVA::CrossValidation cv( "Likelihood_CV", cvLoader, cvFile, cvOptions);

for (int nBins : nBinsValues) {
    TString methodName = Form("Likelihood_Nbins%d", nBins);
    TString methodOptions = Form(
        "!H:!V:TransformOutput=false:PDFInterpol=Spline2:"
        "NSmooth=1:NAvEvtPerBin=0:Nbins=%d:"
        "VarTransform=None:IgnoreNegWeightsInTraining", nBins);

    cv.BookMethod( TMVA::Types::kLikelihood, methodName, methodOptions);}

cv.Evaluate();
const auto& results = cv.GetResults();

if (results.size() != nBinsValues.size()) {
    std::cerr << "ERROR: unexpected number of CV results." << std::endl;
    delete cvLoader; cvFile->Close();
    sigFile->Close(); bkgFile->Close(); return;}

// Select Nbins with the highest mean CV AUC
double bestAUC = -1.0;
int bestNBins = -1;

for (size_t i = 0; i < results.size(); ++i) {
    double meanAUC = results[i].GetROCAverage();
    double stdAUC = results[i].GetROCStandardDeviation();

    std::cout << "Nbins=" << nBinsValues[i]
              << "  Mean AUC=" << meanAUC
              << "  Std=" << stdAUC << std::endl;

    if (meanAUC > bestAUC) {
        bestAUC = meanAUC;
        bestNBins = nBinsValues[i];}}

cvFile->Write();
cvFile->Close();

// Final training with the selected Nbins
if (gSystem->AccessPathName("dataset_Likelihood"))
    gSystem->mkdir("dataset_Likelihood");

if (gSystem->AccessPathName("dataset_Likelihood")) {
    std::cerr << "ERROR creating dataset_Likelihood directory." << std::endl;
    delete cvLoader; sigFile->Close(); bkgFile->Close(); return;}

TFile* outputFile = TFile::Open( "dataset_Likelihood/Likelihood_results.root", "RECREATE");

if (!outputFile || outputFile->IsZombie()) {
    std::cerr << "ERROR creating final output file." << std::endl;
```

```
        delete cvLoader; sigFile->Close(); bkgFile->Close(); return;}

    TMVA::Factory* factory = new TMVA::Factory(
        "Likelihood", outputFile,
        "!V:!Silent:Color:DrawProgressBar:"
        "Transformations=I:AnalysisType=Classification");

    TMVA::DataLoader* finalLoader = new TMVA::DataLoader("dataset_Likelihood");

    finalLoader->AddVariable("pT_mumu", 'F');
    finalLoader->AddVariable("Y_mumu", 'F');
    finalLoader->AddVariable("cosTheta", 'F');
    finalLoader->AddSpectator("m_mumu", 'F');
    finalLoader->AddSpectator("eventNumber", 'l');
    finalLoader->AddSpectator("jet_n", 'I');
    finalLoader->AddSignalTree(sigTree, 1.0);
    finalLoader->AddBackgroundTree(bkgTree, 1.0);
    finalLoader->SetSignalWeightExpression("eventWeight");
    finalLoader->SetBackgroundWeightExpression("eventWeight");

    finalLoader->PrepareTrainingAndTestTree("", "",
        "nTrain_Signal=46273:nTest_Signal=11569:"
        "nTrain_Background=31770:nTest_Background=7943:"
        "SplitMode=Random:SplitSeed=12345:"
        "NormMode=NumEvents:!V");

    TString finalOptions = Form(
        "!H:!V:TransformOutput=false:PDFInterpol=Spline2:"
        "NSmooth=1:NAvEvtPerBin=0:Nbins=%d:"
        "VarTransform=None:IgnoreNegWeightsInTraining", bestNBins);

    factory->BookMethod( finalLoader, TMVA::Types::kLikelihood, "Likelihood", finalOptions);

    factory->TrainAllMethods();
    factory->TestAllMethods();
    factory->EvaluateAllMethods();

    double testAUC = factory->GetROCIntegral(
        finalLoader, "Likelihood", 0, TMVA::Types::kTesting);
    std::cout << "Final test AUC = " << testAUC << std::endl;

    outputFile->Write();
    outputFile->Close();

    delete factory;
    delete finalLoader;
    delete cvLoader;
    sigFile->Close();
    bkgFile->Close();
    delete sigFile;
    delete bkgFile;
}
```

Κώδικας 6: Βελτιστοποίηση και εκπαίδευση του ταξινομητή kNN

```
#include <iostream>
#include <vector>
#include "TFile.h"
#include "TTree.h"
```

```
#include "TString.h"
#include "TSystem.h"
#include "TMVA/Tools.h"
#include "TMVA/Factory.h"
#include "TMVA/DataLoader.h"
#include "TMVA/CrossValidation.h"
#include "TMVA/Types.h"

void KNN()
{
    TMVA::Tools::Instance();
    std::vector<int> kValues = {50, 100, 150, 200, 250, 300};

    TFile* sigFile = TFile::Open("nj0_signal.root");
    TFile* bkgFile = TFile::Open("nj0_background.root");
    if (!sigFile || sigFile->IsZombie() || !bkgFile || bkgFile->IsZombie()) {
        std::cerr << "ERROR opening input files." << std::endl; return; }

    TTree* sigTree = (TTree*)sigFile->Get("signal");
    TTree* bkgTree = (TTree*)bkgFile->Get("background");
    if (!sigTree || !bkgTree) {
        std::cerr << "ERROR loading input trees." << std::endl;
        sigFile->Close(); bkgFile->Close(); return; }

    // Create output directory
    if (gSystem->AccessPathName("dataset_KNN"))
        gSystem->mkdir("dataset_KNN");

    if (gSystem->AccessPathName("dataset_KNN")) {
        std::cerr << "ERROR creating dataset_KNN directory." << std::endl;
        sigFile->Close(); bkgFile->Close(); return;}

    // 5-fold cross-validation
    TFile* cvFile = TFile::Open("dataset_KNN/KNN_CV_results.root", "RECREATE");
    if (!cvFile || cvFile->IsZombie()) {
        std::cerr << "ERROR creating CV output file." << std::endl;
        sigFile->Close(); bkgFile->Close(); return; }

    TMVA::DataLoader* cvLoader = new TMVA::DataLoader("cv_KNN");
    cvLoader->AddVariable("pT_mumu", 'F');
    cvLoader->AddVariable("Y_mumu", 'F');
    cvLoader->AddVariable("cosTheta", 'F');
    cvLoader->AddSpectator("m_mumu", 'F');
    cvLoader->AddSpectator("eventNumber", 'l');
    cvLoader->AddSpectator("jet_n", 'I');
    cvLoader->AddSignalTree(sigTree, 1.0);
    cvLoader->AddBackgroundTree(bkgTree, 1.0);
    cvLoader->SetSignalWeightExpression("eventWeight");
    cvLoader->SetBackgroundWeightExpression("eventWeight");

    cvLoader->PrepareTrainingAndTestTree("", "",
        "nTrain_Signal=46273:nTest_Signal=11569:"
        "nTrain_Background=31770:nTest_Background=7943:"
        "SplitMode=Random:SplitSeed=12345:"
        "NormMode=NumEvents:!V");

    TString cvOptions =
        "!V:!Silent:!ModelPersistence:AnalysisType=Classification:"
        "SplitType=RandomStratified:NumFolds=5";

    TMVA::CrossValidation cv("KNN_CV", cvLoader, cvFile, cvOptions);
```

```
for (int k : kValues) {
    TString methodName = Form("KNN_k%d", k);
    TString methodOptions = Form(
        "!H:!V:nkNN=%d:ScaleFrac=0.8:"
        "SigmaFact=1.0:Kernel=Gaus:UseKernel=F:"
        "UseWeight=T:Trim=F:IgnoreNegWeightsInTraining", k);

    cv.BookMethod(TMVA::Types::kKNN, methodName, methodOptions);}

cv.Evaluate();
const auto& results = cv.GetResults();

if (results.size() != kValues.size()) {
    std::cerr << "ERROR: unexpected number of CV results." << std::endl;
    delete cvLoader; cvFile->Close();
    sigFile->Close(); bkgFile->Close(); return;}

// Select k with the highest mean CV AUC
double bestAUC = -1.0;
int bestK = -1;

for (size_t i = 0; i < results.size(); ++i) {
    double meanAUC = results[i].GetROCAverage();
    if (meanAUC > bestAUC) {
        bestAUC = meanAUC;
        bestK = kValues[i];}}

cvFile->Write();
cvFile->Close();

// Final training with the selected k
TFile* outputFile = TFile::Open("dataset_KNN/KNN_results.root", "RECREATE");

if (!outputFile || outputFile->IsZombie()) {
    std::cerr << "ERROR creating final output file." << std::endl;
    delete cvLoader; sigFile->Close(); bkgFile->Close(); return;}

TMVA::Factory* factory = new TMVA::Factory(
    "KNN", outputFile,
    "!V:!Silent:Color:DrawProgressBar:"
    "Transformations=I:AnalysisType=Classification");

TMVA::DataLoader* finalLoader = new TMVA::DataLoader("dataset_KNN");

finalLoader->AddVariable("pT_mumu", 'F');
finalLoader->AddVariable("Y_mumu", 'F');
finalLoader->AddVariable("cosTheta", 'F');
finalLoader->AddSpectator("m_mumu", 'F');
finalLoader->AddSpectator("eventNumber", 'l');
finalLoader->AddSpectator("jet_n", 'I');
finalLoader->AddSignalTree(sigTree, 1.0);
finalLoader->AddBackgroundTree(bkgTree, 1.0);
finalLoader->SetSignalWeightExpression("eventWeight");
finalLoader->SetBackgroundWeightExpression("eventWeight");

finalLoader->PrepareTrainingAndTestTree("", "",
    "nTrain_Signal=46273:nTest_Signal=11569:"
    "nTrain_Background=31770:nTest_Background=7943:"
    "SplitMode=Random:SplitSeed=12345:"
    "NormMode=NumEvents:!V");
```

```
    TString finalOptions = Form(
        "!H:!V:nkNN=%d:ScaleFrac=0.8:"
        "SigmaFact=1.0:Kernel=Gaus:UseKernel=F:"
        "UseWeight=T:Trim=F:IgnoreNegWeightsInTraining", bestK);

    factory->BookMethod(finalLoader, TMVA::Types::kKNN, "KNN", finalOptions);

    factory->TrainAllMethods();
    factory->TestAllMethods();
    factory->EvaluateAllMethods();

    outputFile->Write();
    outputFile->Close();

    delete factory;
    delete finalLoader;
    delete cvLoader;
    sigFile->Close();
    bkgFile->Close();
    delete sigFile;
    delete bkgFile;
}
```

Κώδικας 7: Βελτιστοποίηση και εκπαίδευση του ταξινομητή MLP

```
#include <iostream>
#include <vector>
#include "TFile.h"
#include "TTree.h"
#include "TString.h"
#include "TSystem.h"
#include "TMVA/Tools.h"
#include "TMVA/Factory.h"
#include "TMVA/DataLoader.h"
#include "TMVA/CrossValidation.h"
#include "TMVA/Types.h"

void MLP()
{
    TMVA::Tools::Instance();
    std::vector<int> neuronValues = {4, 6, 8, 10, 12, 14, 16, 18, 20};

    TFile* sigFile = TFile::Open("nj0_signal.root");
    TFile* bkgFile = TFile::Open("nj0_background.root");
    if (!sigFile || sigFile->IsZombie() || !bkgFile || bkgFile->IsZombie()) {
        std::cerr << "ERROR opening input files." << std::endl; return; }

    TTree* sigTree = (TTree*)sigFile->Get("signal");
    TTree* bkgTree = (TTree*)bkgFile->Get("background");
    if (!sigTree || !bkgTree) {
        std::cerr << "ERROR loading input trees." << std::endl;
        sigFile->Close(); bkgFile->Close(); return; }

    // 5-fold cross-validation
    TFile* cvFile = TFile::Open("MLP_CV_results.root", "RECREATE");
    if (!cvFile || cvFile->IsZombie()) {
        std::cerr << "ERROR creating CV output file." << std::endl;
        sigFile->Close(); bkgFile->Close(); return; }
```

```
TMVA::DataLoader* cvLoader = new TMVA::DataLoader("cv_MLP");
cvLoader->AddVariable("pT_mumu", 'F');
cvLoader->AddVariable("Y_mumu", 'F');
cvLoader->AddVariable("cosTheta", 'F');
cvLoader->AddSpectator("m_mumu", 'F');
cvLoader->AddSpectator("eventNumber", 'l');
cvLoader->AddSpectator("jet_n", 'I');
cvLoader->AddSignalTree(sigTree, 1.0);
cvLoader->AddBackgroundTree(bkgTree, 1.0);
cvLoader->SetSignalWeightExpression("eventWeight");
cvLoader->SetBackgroundWeightExpression("eventWeight");

cvLoader->PrepareTrainingAndTestTree("", "",
    "nTrain_Signal=46273:nTest_Signal=11569:"
    "nTrain_Background=31770:nTest_Background=7943:"
    "SplitMode=Random:SplitSeed=12345:"
    "NormMode=NumEvents:!V");

TString cvOptions =
    "!V:!Silent:!ModelPersistence:AnalysisType=Classification:"
    "SplitType=RandomStratified:NumFolds=5";

TMVA::CrossValidation cv("MLP_CV", cvLoader, cvFile, cvOptions);

for (int nNeurons : neuronValues) {
    TString methodName = Form("MLP_N%d", nNeurons);
    TString methodOptions = Form(
        "H:!V:NeuronType=tanh:VarTransform=N:"
        "NCycles=600:HiddenLayers=%d:TestRate=5:"
        "TrainingMethod=BP:!UseRegulator:"
        "IgnoreNegWeightsInTraining", nNeurons);

    cv.BookMethod(TMVA::Types::kMLP, methodName, methodOptions);}

cv.Evaluate();
const auto& results = cv.GetResults();

if (results.size() != neuronValues.size()) {
    std::cerr << "ERROR: unexpected number of CV results." << std::endl;
    delete cvLoader; cvFile->Close();
    sigFile->Close(); bkgFile->Close(); return; }

// Select number of neurons with the highest mean CV AUC
double bestAUC = -1.0;
int bestNeurons = -1;

for (size_t i = 0; i < results.size(); ++i) {
    double meanAUC = results[i].GetROCAverage();
    if (meanAUC > bestAUC) {
        bestAUC = meanAUC;
        bestNeurons = neuronValues[i];}}

cvFile->Write();
cvFile->Close();

// Final training with the selected number of neurons
if (gSystem->AccessPathName("dataset_MLP"))
    gSystem->mkdir("dataset_MLP");

if (gSystem->AccessPathName("dataset_MLP")) {
```

```
        std::cerr << "ERROR creating dataset_MLP directory." << std::endl;
        delete cvLoader; sigFile->Close(); bkgFile->Close(); return;}

    TFile* outputFile = TFile::Open("dataset_MLP/MLP_results.root", "RECREATE");

    if (!outputFile || outputFile->IsZombie()) {
        std::cerr << "ERROR creating final output file." << std::endl;
        delete cvLoader; sigFile->Close(); bkgFile->Close(); return;}

    TMVA::Factory* factory = new TMVA::Factory(
        "MLP", outputFile,
        "!V:!Silent:Color:DrawProgressBar:"
        "Transformations=I:AnalysisType=Classification");

    TMVA::DataLoader* finalLoader = new TMVA::DataLoader("dataset_MLP");

    finalLoader->AddVariable("pT_mumu", 'F');
    finalLoader->AddVariable("Y_mumu", 'F');
    finalLoader->AddVariable("cosTheta", 'F');
    finalLoader->AddSpectator("m_mumu", 'F');
    finalLoader->AddSpectator("eventNumber", 'l');
    finalLoader->AddSpectator("jet_n", 'I');
    finalLoader->AddSignalTree(sigTree, 1.0);
    finalLoader->AddBackgroundTree(bkgTree, 1.0);
    finalLoader->SetSignalWeightExpression("eventWeight");
    finalLoader->SetBackgroundWeightExpression("eventWeight");

    finalLoader->PrepareTrainingAndTestTree("", "",
        "nTrain_Signal=46273:nTest_Signal=11569:"
        "nTrain_Background=31770:nTest_Background=7943:"
        "SplitMode=Random:SplitSeed=12345:"
        "NormMode=NumEvents:!V");

    TString finalOptions = Form(
        "H:!V:NeuronType=tanh:VarTransform=N:"
        "NCycles=600:HiddenLayers=%d:TestRate=5:"
        "TrainingMethod=BP:!UseRegulator:"
        "IgnoreNegWeightsInTraining", bestNeurons);

    factory->BookMethod( finalLoader, TMVA::Types::kMLP, "MLP", finalOptions);

    factory->TrainAllMethods();
    factory->TestAllMethods();
    factory->EvaluateAllMethods();

    outputFile->Write();
    outputFile->Close();

    delete factory;
    delete finalLoader;
    delete cvLoader;
    sigFile->Close();
    bkgFile->Close();
    delete sigFile;
    delete bkgFile;
}
```

Κώδικας 8: Βελτιστοποίηση και εκπαίδευση του ταξινομητή DNN για $n_j = 0$

```
#include <iostream>
#include <vector>
#include "TFile.h"
#include "TTree.h"
#include "TString.h"
#include "TSystem.h"
#include "TMVA/Tools.h"
#include "TMVA/Factory.h"
#include "TMVA/DataLoader.h"
#include "TMVA/CrossValidation.h"
#include "TMVA/Types.h"

void DNN()
{
    TMVA::Tools::Instance();
    std::vector<int> depthValues = {1, 2, 3, 4, 5, 6};

    TFile* sigFile = TFile::Open("nj0_signal.root");
    TFile* bkgFile = TFile::Open("nj0_background.root");
    if (!sigFile || sigFile->IsZombie() || !bkgFile || bkgFile->IsZombie()) {
        std::cerr << "ERROR opening input files." << std::endl; return; }

    TTree* sigTree = (TTree*)sigFile->Get("signal");
    TTree* bkgTree = (TTree*)bkgFile->Get("background");
    if (!sigTree || !bkgTree) {
        std::cerr << "ERROR loading input trees." << std::endl;
        sigFile->Close(); bkgFile->Close(); return; }

    // 5-fold cross-validation
    TFile* cvFile = TFile::Open("DNN_CV_results.root", "RECREATE");
    if (!cvFile || cvFile->IsZombie()) {
        std::cerr << "ERROR creating CV output file." << std::endl;
        sigFile->Close(); bkgFile->Close(); return; }

    TMVA::DataLoader* cvLoader = new TMVA::DataLoader("cv_DNN");
    cvLoader->AddVariable("pT_mumu", 'F');
    cvLoader->AddVariable("Y_mumu", 'F');
    cvLoader->AddVariable("cosTheta", 'F');
    cvLoader->AddSpectator("m_mumu", 'F');
    cvLoader->AddSpectator("eventNumber", 'l');
    cvLoader->AddSpectator("jet_n", 'I');
    cvLoader->AddSignalTree(sigTree, 1.0);
    cvLoader->AddBackgroundTree(bkgTree, 1.0);
    cvLoader->SetSignalWeightExpression("eventWeight");
    cvLoader->SetBackgroundWeightExpression("eventWeight");

    cvLoader->PrepareTrainingAndTestTree("", "",
        "nTrain_Signal=46273:nTest_Signal=11569:"
        "nTrain_Background=31770:nTest_Background=7943:"
        "SplitMode=Random:SplitSeed=12345:"
        "NormMode=NumEvents:!V");

    TString cvOptions =
        "!V:!Silent:!ModelPersistence:AnalysisType=Classification:"
        "SplitType=RandomStratified:NumFolds=5";

    TString trainingStrategy =
```

```
    "TrainingStrategy=LearningRate=1e-3,Momentum=0.9,"
    "ConvergenceSteps=50,BatchSize=100,TestRepetitions=1,"
    "MaxEpochs=1000,WeightDecay=0.0,Regularization=None,"
    "Optimizer=ADAM";

TString commonOptions =
    "!H:!V:ErrorStrategy=CROSSENTROPY:VarTransform=N:"
    "WeightInitialization=XAVIERUNIFORM:Architecture=CPU";

TMVA::CrossValidation cv("DNN_CV", cvLoader, cvFile, cvOptions);

for (int depth : depthValues) {
    TString methodName = Form("DNN_L%d", depth);
    TString layout = "Layout=";

    for (int l = 0; l < depth; ++l) {
        if (l > 0) layout += ",";
        layout += "TANH|6";}
    layout += ",LINEAR";

    TString methodOptions = commonOptions + ":" + layout + ":" + trainingStrategy;

    cv.BookMethod(TMVA::Types::kDL, methodName, methodOptions);}

cv.Evaluate();
const auto& results = cv.GetResults();

if (results.size() != depthValues.size()) {
    std::cerr << "ERROR: unexpected number of CV results." << std::endl;
    delete cvLoader; cvFile->Close();
    sigFile->Close(); bkgFile->Close(); return;}

// Select valid depth with the highest mean CV AUC
double bestAUC = -1.0;
int bestDepth = -1;

for (size_t i = 0; i < results.size(); ++i) {
    bool validConfiguration = true;

    for (const auto& entry : results[i].GetROCValues()) {
        if (entry.second < 0.5) {
            validConfiguration = false;
            break;}}

    double meanAUC = results[i].GetROCAverage();
    if (validConfiguration && meanAUC > bestAUC) {
        bestAUC = meanAUC;
        bestDepth = depthValues[i];} }

if (bestDepth < 0) {
    std::cerr << "ERROR: no valid DNN configuration." << std::endl;
    cvFile->Write(); cvFile->Close();
    delete cvLoader;
    sigFile->Close(); bkgFile->Close();
    delete sigFile; delete bkgFile;
    return;}

cvFile->Write();
cvFile->Close();

// Final training with the selected depth
```

```
    if (gSystem->AccessPathName("dataset_DNN"))
        gSystem->mkdir("dataset_DNN");

    if (gSystem->AccessPathName("dataset_DNN")) {
        std::cerr << "ERROR creating dataset_DNN directory." << std::endl;
        delete cvLoader; sigFile->Close(); bkgFile->Close(); return;}

    TFile* outputFile = TFile::Open("dataset_DNN/DNN_results.root", "RECREATE");

    if (!outputFile || outputFile->IsZombie()) {
        std::cerr << "ERROR creating final output file." << std::endl;
        delete cvLoader; sigFile->Close(); bkgFile->Close(); return;}

    TMVA::Factory* factory = new TMVA::Factory(
        "DNN", outputFile,
        "!V:!Silent:Color:DrawProgressBar:"
        "Transformations=I:AnalysisType=Classification");

    TMVA::DataLoader* finalLoader = new TMVA::DataLoader("dataset_DNN");
    finalLoader->AddVariable("pT_mumu", 'F');
    finalLoader->AddVariable("Y_mumu", 'F');
    finalLoader->AddVariable("cosTheta", 'F');
    finalLoader->AddSpectator("m_mumu", 'F');
    finalLoader->AddSpectator("eventNumber", 'l');
    finalLoader->AddSpectator("jet_n", 'I');
    finalLoader->AddSignalTree(sigTree, 1.0);
    finalLoader->AddBackgroundTree(bkgTree, 1.0);
    finalLoader->SetSignalWeightExpression("eventWeight");
    finalLoader->SetBackgroundWeightExpression("eventWeight");

    finalLoader->PrepareTrainingAndTestTree("", "",
        "nTrain_Signal=46273:nTest_Signal=11569:"
        "nTrain_Background=31770:nTest_Background=7943:"
        "SplitMode=Random:SplitSeed=12345:"
        "NormMode=NumEvents:!V");

    TString bestLayout = "Layout=";
    for (int l = 0; l < bestDepth; ++l) {
        if (l > 0) bestLayout += ",";
        bestLayout += "TANH|6";}
    bestLayout += ",LINEAR";

    TString finalOptions = commonOptions + ":" + bestLayout + ":" + trainingStrategy;

    factory->BookMethod( finalLoader, TMVA::Types::kDL, "DNN", finalOptions);

    factory->TrainAllMethods();
    factory->TestAllMethods();
    factory->EvaluateAllMethods();

    outputFile->Write();
    outputFile->Close();

    delete factory;
    delete finalLoader;
    delete cvLoader;
    sigFile->Close();
    bkgFile->Close();
    delete sigFile;
    delete bkgFile;
}
```

Κώδικας 9: Βελτιστοποίηση και εκπαίδευση του ταξινομητή BDT για $n_j = 0$

```
#include <iostream>
#include <vector>
#include "TFile.h"
#include "TTree.h"
#include "TString.h"
#include "TSystem.h"
#include "TMVA/Tools.h"
#include "TMVA/Factory.h"
#include "TMVA/DataLoader.h"
#include "TMVA/CrossValidation.h"
#include "TMVA/Types.h"

void BDT()
{
    TMVA::Tools::Instance();
    std::vector<int> depthValues = {1, 2, 3, 4, 5, 6};

    TFile* sigFile = TFile::Open("nj0_signal.root");
    TFile* bkgFile = TFile::Open("nj0_background.root");
    if (!sigFile || sigFile->IsZombie() || !bkgFile || bkgFile->IsZombie()) {
        std::cerr << "ERROR opening input files." << std::endl; return; }

    TTree* sigTree = (TTree*)sigFile->Get("signal");
    TTree* bkgTree = (TTree*)bkgFile->Get("background");
    if (!sigTree || !bkgTree) {
        std::cerr << "ERROR loading input trees." << std::endl;
        sigFile->Close(); bkgFile->Close(); return; }

    // 5-fold cross-validation
    TFile* cvFile = TFile::Open("BDT_CV_results.root", "RECREATE");
    if (!cvFile || cvFile->IsZombie()) {
        std::cerr << "ERROR creating CV output file." << std::endl;
        sigFile->Close(); bkgFile->Close(); return; }

    TMVA::DataLoader* cvLoader = new TMVA::DataLoader("cv_BDT");
    cvLoader->AddVariable("pT_mumu", 'F');
    cvLoader->AddVariable("Y_mumu", 'F');
    cvLoader->AddVariable("cosTheta", 'F');
    cvLoader->AddSpectator("m_mumu", 'F');
    cvLoader->AddSpectator("eventNumber", 'l');
    cvLoader->AddSpectator("jet_n", 'I');
    cvLoader->AddSignalTree(sigTree, 1.0);
    cvLoader->AddBackgroundTree(bkgTree, 1.0);
    cvLoader->SetSignalWeightExpression("eventWeight");
    cvLoader->SetBackgroundWeightExpression("eventWeight");

    cvLoader->PrepareTrainingAndTestTree("", "",
        "nTrain_Signal=46273:nTest_Signal=11569:"
        "nTrain_Background=31770:nTest_Background=7943:"
        "SplitMode=Random:SplitSeed=12345:"
        "NormMode=NumEvents:!V");

    TString cvOptions =
        "!V:!Silent:!ModelPersistence:AnalysisType=Classification:"
        "SplitType=RandomStratified:NumFolds=5";

    TString commonOptions =
```

```
    "!H:!V:NTrees=800:MinNodeSize=2.5%:"
    "BoostType=AdaBoost:AdaBoostBeta=0.5:UseBaggedBoost:"
    "BaggedSampleFraction=0.5:SeparationType=GiniIndex:"
    "nCuts=20:NegWeightTreatment=IgnoreNegWeightsInTraining";

TMVA::CrossValidation cv("BDT_CV", cvLoader, cvFile, cvOptions);

for (int depth : depthValues) {
    TString methodName = Form("BDT_D%d", depth);
    TString methodOptions = commonOptions + Form(":MaxDepth=%d", depth);

    cv.BookMethod(TMVA::Types::kBDT, methodName, methodOptions); }

cv.Evaluate();
const auto& results = cv.GetResults();

if (results.size() != depthValues.size()) {
    std::cerr << "ERROR: unexpected number of CV results." << std::endl;
    delete cvLoader; cvFile->Close();
    sigFile->Close(); bkgFile->Close(); return;}

// Select MaxDepth with the highest mean CV AUC
double bestAUC = -1.0;
int bestDepth = -1;

for (size_t i = 0; i < results.size(); ++i) {
    double meanAUC = results[i].GetROCAverage();
    if (meanAUC > bestAUC) {
        bestAUC = meanAUC;
        bestDepth = depthValues[i]; }}

cvFile->Write();
cvFile->Close();

// Final training with the selected MaxDepth
if (gSystem->AccessPathName("dataset_BDT"))
    gSystem->mkdir("dataset_BDT");

if (gSystem->AccessPathName("dataset_BDT")) {
    std::cerr << "ERROR creating dataset_BDT directory." << std::endl;
    delete cvLoader; sigFile->Close(); bkgFile->Close(); return; }

TFile* outputFile = TFile::Open("dataset_BDT/BDT_results.root", "RECREATE");

if (!outputFile || outputFile->IsZombie()) {
    std::cerr << "ERROR creating final output file." << std::endl;
    delete cvLoader; sigFile->Close(); bkgFile->Close(); return; }

TMVA::Factory* factory = new TMVA::Factory(
    "BDT", outputFile,
    "!V:!Silent:Color:DrawProgressBar:"
    "Transformations=I:AnalysisType=Classification");

TMVA::DataLoader* finalLoader = new TMVA::DataLoader("dataset_BDT");
finalLoader->AddVariable("pT_mumu", 'F');
finalLoader->AddVariable("Y_mumu", 'F');
finalLoader->AddVariable("cosTheta", 'F');
finalLoader->AddSpectator("m_mumu", 'F');
finalLoader->AddSpectator("eventNumber", 'l');
finalLoader->AddSpectator("jet_n", 'I');
finalLoader->AddSignalTree(sigTree, 1.0);
```

```
    finalLoader->AddBackgroundTree(bkgTree, 1.0);
    finalLoader->SetSignalWeightExpression("eventWeight");
    finalLoader->SetBackgroundWeightExpression("eventWeight");

    finalLoader->PrepareTrainingAndTestTree("", "",
        "nTrain_Signal=46273:nTest_Signal=11569:"
        "nTrain_Background=31770:nTest_Background=7943:"
        "SplitMode=Random:SplitSeed=12345:"
        "NormMode=NumEvents:!V");

    TString finalOptions = commonOptions + Form(":MaxDepth=%d", bestDepth);

    factory->BookMethod( finalLoader, TMVA::Types::kBDT, "BDT", finalOptions);

    factory->TrainAllMethods();
    factory->TestAllMethods();
    factory->EvaluateAllMethods();

    outputFile->Write();
    outputFile->Close();

    delete factory;
    delete finalLoader;
    delete cvLoader;
    sigFile->Close();
    bkgFile->Close();
    delete sigFile;
    delete bkgFile;
}
```

Κώδικας 10: Βελτιστοποίηση και εκπαίδευση του ταξινομητή DNN για $n_j = 1$

```
#include <iostream>
#include <vector>
#include "TFile.h"
#include "TTree.h"
#include "TString.h"
#include "TSystem.h"
#include "TMVA/Tools.h"
#include "TMVA/Factory.h"
#include "TMVA/DataLoader.h"
#include "TMVA/CrossValidation.h"
#include "TMVA/Types.h"

void DNN()
{
    TMVA::Tools::Instance();

    std::vector<TString> configNames = {
        "6_6", "12_12", "24_12", "6_6_6", "12_12_12",
        "24_24_24", "24_12_6", "12_12_12_12", "36_24_12_6" };

    std::vector<TString> layouts = {
        "Layout=TANH|6,TANH|6,LINEAR",
        "Layout=TANH|12,TANH|12,LINEAR",
        "Layout=TANH|24,TANH|12,LINEAR",
        "Layout=TANH|6,TANH|6,TANH|6,LINEAR",
        "Layout=TANH|12,TANH|12,TANH|12,LINEAR",
```

```
    "Layout=TANH|24,TANH|24,TANH|24,LINEAR",
    "Layout=TANH|24,TANH|12,TANH|6,LINEAR",
    "Layout=TANH|12,TANH|12,TANH|12,TANH|12,LINEAR",
    "Layout=TANH|36,TANH|24,TANH|12,TANH|6,LINEAR" };

TFile* sigFile = TFile::Open("nj1_signal.root");
TFile* bkgFile = TFile::Open("nj1_background.root");
if (!sigFile || sigFile->IsZombie() || !bkgFile || bkgFile->IsZombie()) {
    std::cerr << "ERROR opening input files." << std::endl; return; }

TTree* sigTree = (TTree*)sigFile->Get("signal");
TTree* bkgTree = (TTree*)bkgFile->Get("background");
if (!sigTree || !bkgTree) {
    std::cerr << "ERROR loading input trees." << std::endl;
    sigFile->Close(); bkgFile->Close(); return; }

// 5-fold cross-validation
TFile* cvFile = TFile::Open("DNN_nj1_CV_results.root", "RECREATE");
if (!cvFile || cvFile->IsZombie()) {
    std::cerr << "ERROR creating CV output file." << std::endl;
    sigFile->Close(); bkgFile->Close(); return; }

TMVA::DataLoader* cvLoader = new TMVA::DataLoader("cv_DNN_nj1");
cvLoader->AddVariable("pT_mumu", 'F');
cvLoader->AddVariable("Y_mumu", 'F');
cvLoader->AddVariable("cosTheta", 'F');
cvLoader->AddVariable("jet1_pt", 'F');
cvLoader->AddVariable("jet1_eta", 'F');
cvLoader->AddVariable("dPhi_mumu_j1", 'F');
cvLoader->AddSpectator("m_mumu", 'F');
cvLoader->AddSpectator("eventNumber", 'l');
cvLoader->AddSpectator("jet_n", 'I');
cvLoader->AddSpectator("channelNumber", 'i');
cvLoader->AddSignalTree(sigTree, 1.0);
cvLoader->AddBackgroundTree(bkgTree, 1.0);
cvLoader->SetSignalWeightExpression("eventWeight");
cvLoader->SetBackgroundWeightExpression("eventWeight");

cvLoader->PrepareTrainingAndTestTree("", "",
    "nTrain_Signal=42378:nTest_Signal=10595:"
    "nTrain_Background=57632:nTest_Background=14408:"
    "SplitMode=Random:SplitSeed=12345:"
    "NormMode=NumEvents:!V");

TString cvOptions =
    "!V:!Silent:!ModelPersistence:AnalysisType=Classification:"
    "SplitType=RandomStratified:NumFolds=5";

TString trainingStrategy =
    "TrainingStrategy=LearningRate=1e-3,Momentum=0.9,"
    "ConvergenceSteps=50,BatchSize=100,TestRepetitions=1,"
    "MaxEpochs=1000,WeightDecay=0.0,Regularization=None,"
    "Optimizer=ADAM";

TString commonOptions =
    "!H:!V:ErrorStrategy=CROSSENTROPY:VarTransform=N:"
    "WeightInitialization=XAVIERUNIFORM:Architecture=CPU";

TMVA::CrossValidation cv("DNN_CV_nj1", cvLoader, cvFile, cvOptions);

for (size_t i = 0; i < layouts.size(); ++i) {
```

```
    TString methodName = "DNN_" + configNames[i]; TString methodOptions =  commonOptions + ":" +
 layouts[i] + ":" + trainingStrategy;

    cv.BookMethod(TMVA::Types::kDL, methodName, methodOptions);}

cv.Evaluate();
const auto& results = cv.GetResults();

if (results.size() != layouts.size()) {
    std::cerr << "ERROR: unexpected number of CV results." << std::endl;
    delete cvLoader; cvFile->Close();
    sigFile->Close(); bkgFile->Close(); return; }

// Select valid architecture with the highest mean CV AUC
double bestAUC = -1.0;
int bestConfig = -1;

for (size_t i = 0; i < results.size(); ++i) {
    bool validConfiguration = true;

    for (const auto& entry : results[i].GetROCValues()) {
        if (entry.second < 0.5) {
            validConfiguration = false;
            break; }}

    double meanAUC = results[i].GetROCAverage();
    if (validConfiguration && meanAUC > bestAUC) {
        bestAUC = meanAUC;
        bestConfig = static_cast<int>(i); } }

if (bestConfig < 0) {
    std::cerr << "ERROR: no valid DNN configuration." << std::endl;
    cvFile->Write(); cvFile->Close();
    delete cvLoader;
    sigFile->Close(); bkgFile->Close();
    delete sigFile; delete bkgFile;
    return; }

cvFile->Write();
cvFile->Close();

// Final training with the selected architecture
if (gSystem->AccessPathName("dataset_DNN_nj1"))
    gSystem->mkdir("dataset_DNN_nj1");

if (gSystem->AccessPathName("dataset_DNN_nj1")) {
    std::cerr << "ERROR creating dataset_DNN_nj1 directory." << std::endl;
    delete cvLoader; sigFile->Close(); bkgFile->Close(); return; }

TFile* outputFile = TFile::Open("dataset_DNN_nj1/DNN_results.root", "RECREATE");

if (!outputFile || outputFile->IsZombie()) {
    std::cerr << "ERROR creating final output file." << std::endl;
    delete cvLoader; sigFile->Close(); bkgFile->Close(); return; }

TMVA::Factory* factory = new TMVA::Factory(
    "DNN", outputFile,
    "!V:!Silent:Color:DrawProgressBar:"
    "Transformations=I:AnalysisType=Classification");

TMVA::DataLoader* finalLoader = new TMVA::DataLoader("dataset_DNN_nj1");
```

```
    finalLoader->AddVariable("pT_mumu", 'F');
    finalLoader->AddVariable("Y_mumu", 'F');
    finalLoader->AddVariable("cosTheta", 'F');
    finalLoader->AddVariable("jet1_pt", 'F');
    finalLoader->AddVariable("jet1_eta", 'F');
    finalLoader->AddVariable("dPhi_mumu_j1", 'F');
    finalLoader->AddSpectator("m_mumu", 'F');
    finalLoader->AddSpectator("eventNumber", 'l');
    finalLoader->AddSpectator("jet_n", 'I');
    finalLoader->AddSpectator("channelNumber", 'i');
    finalLoader->AddSignalTree(sigTree, 1.0);
    finalLoader->AddBackgroundTree(bkgTree, 1.0);
    finalLoader->SetSignalWeightExpression("eventWeight");
    finalLoader->SetBackgroundWeightExpression("eventWeight");

    finalLoader->PrepareTrainingAndTestTree("", "",
        "nTrain_Signal=42378:nTest_Signal=10595:"
        "nTrain_Background=57632:nTest_Background=14408:"
        "SplitMode=Random:SplitSeed=12345:"
        "NormMode=NumEvents:!V");

    TString finalOptions = commonOptions + ":" + layouts[bestConfig] + ":" + trainingStrategy;

    factory->BookMethod( finalLoader, TMVA::Types::kDL, "DNN", finalOptions);

    factory->TrainAllMethods();
    factory->TestAllMethods();
    factory->EvaluateAllMethods();

    outputFile->Write();
    outputFile->Close();

    delete factory;
    delete finalLoader;
    delete cvLoader;
    sigFile->Close();
    bkgFile->Close();
    delete sigFile;
    delete bkgFile;
}
```

Κώδικας 11: Βελτιστοποίηση και εκπαίδευση του ταξινομητή BDT για $n_j = 1$

```
#include <iostream>
#include <vector>
#include "TFile.h"
#include "TTree.h"
#include "TString.h"
#include "TSystem.h"
#include "TMVA/Tools.h"
#include "TMVA/Factory.h"
#include "TMVA/DataLoader.h"
#include "TMVA/CrossValidation.h"
#include "TMVA/Types.h"

void BDT()
{
```

```
TMVA::Tools::Instance();
std::vector<int> depthValues = {1, 2, 3, 4, 5, 6, 7, 8};

TFile* sigFile = TFile::Open("nj1_signal.root");
TFile* bkgFile = TFile::Open("nj1_background.root");
if (!sigFile || sigFile->IsZombie() || !bkgFile || bkgFile->IsZombie()) {
    std::cerr << "ERROR opening input files." << std::endl; return; }

TTree* sigTree = (TTree*)sigFile->Get("signal");
TTree* bkgTree = (TTree*)bkgFile->Get("background");
if (!sigTree || !bkgTree) {
    std::cerr << "ERROR loading input trees." << std::endl;
    sigFile->Close(); bkgFile->Close(); return; }

// 5-fold cross-validation
TFile* cvFile = TFile::Open("BDT_nj1_CV_results.root", "RECREATE");
if (!cvFile || cvFile->IsZombie()) {
    std::cerr << "ERROR creating CV output file." << std::endl;
    sigFile->Close(); bkgFile->Close(); return; }

TMVA::DataLoader* cvLoader = new TMVA::DataLoader("cv_BDT_nj1");
cvLoader->AddVariable("pT_mumu", 'F');
cvLoader->AddVariable("Y_mumu", 'F');
cvLoader->AddVariable("cosTheta", 'F');
cvLoader->AddVariable("jet1_pt", 'F');
cvLoader->AddVariable("jet1_eta", 'F');
cvLoader->AddVariable("dPhi_mumu_j1", 'F');
cvLoader->AddSpectator("m_mumu", 'F');
cvLoader->AddSpectator("eventNumber", 'l');
cvLoader->AddSpectator("jet_n", 'I');
cvLoader->AddSpectator("channelNumber", 'i');
cvLoader->AddSignalTree(sigTree, 1.0);
cvLoader->AddBackgroundTree(bkgTree, 1.0);
cvLoader->SetSignalWeightExpression("eventWeight");
cvLoader->SetBackgroundWeightExpression("eventWeight");

cvLoader->PrepareTrainingAndTestTree("", "",
    "nTrain_Signal=42378:nTest_Signal=10595:"
    "nTrain_Background=57632:nTest_Background=14408:"
    "SplitMode=Random:SplitSeed=12345:"
    "NormMode=NumEvents:!V");

TString cvOptions = "!V:!Silent:!ModelPersistence:AnalysisType=Classification:"  "SplitType=
 RandomStratified:NumFolds=5";

TString commonOptions =
    "!H:!V:NTrees=800:MinNodeSize=2.5%:"
    "BoostType=AdaBoost:AdaBoostBeta=0.5:UseBaggedBoost:"
    "BaggedSampleFraction=0.5:SeparationType=GiniIndex:"
    "nCuts=20:NegWeightTreatment=IgnoreNegWeightsInTraining";

TMVA::CrossValidation cv("BDT_CV_nj1", cvLoader, cvFile, cvOptions);

for (int depth : depthValues) {
    TString methodName = Form("BDT_D%d", depth);
    TString methodOptions = commonOptions + Form(":MaxDepth=%d", depth);

    cv.BookMethod(TMVA::Types::kBDT, methodName, methodOptions); }

cv.Evaluate();
const auto& results = cv.GetResults();
```

```
if (results.size() != depthValues.size()) {
    std::cerr << "ERROR: unexpected number of CV results." << std::endl;
    delete cvLoader; cvFile->Close();
    sigFile->Close(); bkgFile->Close(); return; }

// Select MaxDepth with the highest mean CV AUC
double bestAUC = -1.0;
int bestDepth = -1;

for (size_t i = 0; i < results.size(); ++i) {
    double meanAUC = results[i].GetROCAverage();
    if (meanAUC > bestAUC) {
        bestAUC = meanAUC;
        bestDepth = depthValues[i]; } }

cvFile->Write();
cvFile->Close();

// Final training with the selected MaxDepth
if (gSystem->AccessPathName("dataset_BDT_nj1"))
    gSystem->mkdir("dataset_BDT_nj1");

if (gSystem->AccessPathName("dataset_BDT_nj1")) {
    std::cerr << "ERROR creating dataset_BDT_nj1 directory." << std::endl;
    delete cvLoader; sigFile->Close(); bkgFile->Close(); return; }

TFile* outputFile =
    TFile::Open("dataset_BDT_nj1/BDT_results.root", "RECREATE");

if (!outputFile || outputFile->IsZombie()) {
    std::cerr << "ERROR creating final output file." << std::endl;
    delete cvLoader; sigFile->Close(); bkgFile->Close(); return; }

TMVA::Factory* factory = new TMVA::Factory(
    "BDT", outputFile,
    "!V:!Silent:Color:DrawProgressBar:"
    "Transformations=I:AnalysisType=Classification");

TMVA::DataLoader* finalLoader = new TMVA::DataLoader("dataset_BDT_nj1");

finalLoader->AddVariable("pT_mumu", 'F');
finalLoader->AddVariable("Y_mumu", 'F');
finalLoader->AddVariable("cosTheta", 'F');
finalLoader->AddVariable("jet1_pt", 'F');
finalLoader->AddVariable("jet1_eta", 'F');
finalLoader->AddVariable("dPhi_mumu_j1", 'F');
finalLoader->AddSpectator("m_mumu", 'F');
finalLoader->AddSpectator("eventNumber", 'l');
finalLoader->AddSpectator("jet_n", 'I');
finalLoader->AddSpectator("channelNumber", 'i');
finalLoader->AddSignalTree(sigTree, 1.0);
finalLoader->AddBackgroundTree(bkgTree, 1.0);
finalLoader->SetSignalWeightExpression("eventWeight");
finalLoader->SetBackgroundWeightExpression("eventWeight");

finalLoader->PrepareTrainingAndTestTree("", "",
    "nTrain_Signal=42378:nTest_Signal=10595:"
    "nTrain_Background=57632:nTest_Background=14408:"
    "SplitMode=Random:SplitSeed=12345:"
    "NormMode=NumEvents:!V");
```

```
    TString finalOptions = commonOptions + Form(":MaxDepth=%d", bestDepth);

    factory->BookMethod( finalLoader, TMVA::Types::kBDT, "BDT", finalOptions);

    factory->TrainAllMethods();
    factory->TestAllMethods();
    factory->EvaluateAllMethods();

    outputFile->Write();
    outputFile->Close();

    delete factory;
    delete finalLoader;
    delete cvLoader;
    sigFile->Close();
    bkgFile->Close();
    delete sigFile;
    delete bkgFile;
}
```

Κώδικας 12: Βελτιστοποίηση και εκπαίδευση του ταξινομητή DNN για $n_j \geq 2$

```
#include <iostream>
#include <vector>
#include "TFile.h"
#include "TTree.h"
#include "TString.h"
#include "TSystem.h"
#include "TMVA/Tools.h"
#include "TMVA/Factory.h"
#include "TMVA/DataLoader.h"
#include "TMVA/CrossValidation.h"
#include "TMVA/Types.h"

void DNN_nj2()
{
    TMVA::Tools::Instance();

    std::vector<TString> configNames = {
        "15_15", "30_15", "30_30", "15_15_15", "30_30_30",
        "60_30_15", "15_15_15_15", "30_30_30_30", "75_60_30_15"};

    std::vector<TString> layouts = {
        "Layout=TANH|15,TANH|15,LINEAR",
        "Layout=TANH|30,TANH|15,LINEAR",
        "Layout=TANH|30,TANH|30,LINEAR",
        "Layout=TANH|15,TANH|15,TANH|15,LINEAR",
        "Layout=TANH|30,TANH|30,TANH|30,LINEAR",
        "Layout=TANH|60,TANH|30,TANH|15,LINEAR",
        "Layout=TANH|15,TANH|15,TANH|15,TANH|15,LINEAR",
        "Layout=TANH|30,TANH|30,TANH|30,TANH|30,LINEAR",
        "Layout=TANH|75,TANH|60,TANH|30,TANH|15,LINEAR"};

    TFile* sigFile = TFile::Open("nj2_signal.root");
    TFile* bkgFile = TFile::Open("nj2_background.root");
    if (!sigFile || sigFile->IsZombie() || !bkgFile || bkgFile->IsZombie()) {
        std::cerr << "ERROR opening input files." << std::endl; return; }
```

```
TTree* sigTree = (TTree*)sigFile->Get("signal");
TTree* bkgTree = (TTree*)bkgFile->Get("background");
if (!sigTree || !bkgTree) {
    std::cerr << "ERROR loading input trees." << std::endl;
    sigFile->Close(); bkgFile->Close(); return; }

// 5-fold cross-validation
TFile* cvFile = TFile::Open("DNN_nj2_CV_results.root", "RECREATE");
if (!cvFile || cvFile->IsZombie()) {
    std::cerr << "ERROR creating CV output file." << std::endl;
    sigFile->Close(); bkgFile->Close(); return; }

TMVA::DataLoader* cvLoader = new TMVA::DataLoader("cv_DNN_nj2");

cvLoader->AddVariable("pT_mumu", 'F');
cvLoader->AddVariable("Y_mumu", 'F');
cvLoader->AddVariable("cosTheta", 'F');
cvLoader->AddVariable("jet1_pt", 'F');
cvLoader->AddVariable("jet1_eta", 'F');
cvLoader->AddVariable("dPhi_j1_mumu", 'F');
cvLoader->AddVariable("jet2_pt", 'F');
cvLoader->AddVariable("jet2_eta", 'F');
cvLoader->AddVariable("dPhi_j2_mumu", 'F');
cvLoader->AddVariable("pT_jj", 'F');
cvLoader->AddVariable("Y_jj", 'F');
cvLoader->AddVariable("dPhi_jj_mumu", 'F');
cvLoader->AddVariable("m_jj", 'F');
cvLoader->AddVariable("met", 'F');
cvLoader->AddVariable("HT", 'F');

cvLoader->AddSpectator("m_mumu", 'F');
cvLoader->AddSpectator("eventNumber", 'l');
cvLoader->AddSpectator("jet_n", 'I');
cvLoader->AddSpectator("channelNumber", 'i');

cvLoader->AddSignalTree(sigTree, 1.0);
cvLoader->AddBackgroundTree(bkgTree, 1.0);
cvLoader->SetSignalWeightExpression("eventWeight");
cvLoader->SetBackgroundWeightExpression("eventWeight");

cvLoader->PrepareTrainingAndTestTree("", "",
    "nTrain_Signal=34550:nTest_Signal=8638:"
    "nTrain_Background=100521:nTest_Background=25131:"
    "SplitMode=Random:SplitSeed=12345:"
    "NormMode=NumEvents:!V");

TString cvOptions =
    "!V:!Silent:!ModelPersistence:AnalysisType=Classification:"
    "SplitType=RandomStratified:NumFolds=5";

TString trainingStrategy =
    "TrainingStrategy=LearningRate=1e-3,Momentum=0.9,"
    "ConvergenceSteps=50,BatchSize=100,TestRepetitions=1,"
    "MaxEpochs=1000,WeightDecay=0.0,Regularization=None,"
    "Optimizer=ADAM";

TString commonOptions =
    "!H:!V:ErrorStrategy=CROSSENTROPY:VarTransform=N:"
    "WeightInitialization=XAVIERUNIFORM:Architecture=CPU";
```

```
TMVA::CrossValidation cv("DNN_CV_nj2", cvLoader, cvFile, cvOptions);

for (size_t i = 0; i < layouts.size(); ++i) {
    TString methodName = "DNN_" + configNames[i];
    TString methodOptions = commonOptions + ":" + layouts[i] + ":" + trainingStrategy;

    cv.BookMethod(TMVA::Types::kDL, methodName, methodOptions);}

cv.Evaluate();
const auto& results = cv.GetResults();

if (results.size() != layouts.size()) {
    std::cerr << "ERROR: unexpected number of CV results." << std::endl;
    delete cvLoader; cvFile->Close();
    sigFile->Close(); bkgFile->Close(); return;}

// Select valid architecture with the highest mean CV AUC
double bestAUC = -1.0;
int bestConfig = -1;

for (size_t i = 0; i < results.size(); ++i) {
    bool validConfiguration = true;

    for (const auto& entry : results[i].GetROCValues()) {
        if (entry.second < 0.5) {
            validConfiguration = false;
            break;}}

    double meanAUC = results[i].GetROCAverage();
    if (validConfiguration && meanAUC > bestAUC) {
        bestAUC = meanAUC;
        bestConfig = static_cast<int>(i);}}

if (bestConfig < 0) {
    std::cerr << "ERROR: no valid DNN configuration." << std::endl;
    cvFile->Write(); cvFile->Close();
    delete cvLoader;
    sigFile->Close(); bkgFile->Close();
    delete sigFile; delete bkgFile;
    return; }

cvFile->Write();
cvFile->Close();

// Final training with the selected architecture
if (gSystem->AccessPathName("dataset_DNN_nj2"))
    gSystem->mkdir("dataset_DNN_nj2");

if (gSystem->AccessPathName("dataset_DNN_nj2")) {
    std::cerr << "ERROR creating dataset_DNN_nj2 directory." << std::endl;
    delete cvLoader; sigFile->Close(); bkgFile->Close(); return;}

TFile* outputFile = TFile::Open("dataset_DNN_nj2/DNN_results.root", "RECREATE");

if (!outputFile || outputFile->IsZombie()) {
    std::cerr << "ERROR creating final output file." << std::endl;
    delete cvLoader; sigFile->Close(); bkgFile->Close(); return;}

TMVA::Factory* factory = new TMVA::Factory(
    "DNN_nj2", outputFile,
    "!V:!Silent:Color:DrawProgressBar:"
```

```
        "Transformations=I:AnalysisType=Classification");

    TMVA::DataLoader* finalLoader = new TMVA::DataLoader("dataset_DNN_nj2");

    finalLoader->AddVariable("pT_mumu", 'F');
    finalLoader->AddVariable("Y_mumu", 'F');
    finalLoader->AddVariable("cosTheta", 'F');
    finalLoader->AddVariable("jet1_pt", 'F');
    finalLoader->AddVariable("jet1_eta", 'F');
    finalLoader->AddVariable("dPhi_j1_mumu", 'F');
    finalLoader->AddVariable("jet2_pt", 'F');
    finalLoader->AddVariable("jet2_eta", 'F');
    finalLoader->AddVariable("dPhi_j2_mumu", 'F');
    finalLoader->AddVariable("pT_jj", 'F');
    finalLoader->AddVariable("Y_jj", 'F');
    finalLoader->AddVariable("dPhi_jj_mumu", 'F');
    finalLoader->AddVariable("m_jj", 'F');
    finalLoader->AddVariable("met", 'F');
    finalLoader->AddVariable("HT", 'F');

    finalLoader->AddSpectator("m_mumu", 'F');
    finalLoader->AddSpectator("eventNumber", 'l');
    finalLoader->AddSpectator("jet_n", 'I');
    finalLoader->AddSpectator("channelNumber", 'i');

    finalLoader->AddSignalTree(sigTree, 1.0);
    finalLoader->AddBackgroundTree(bkgTree, 1.0);
    finalLoader->SetSignalWeightExpression("eventWeight");
    finalLoader->SetBackgroundWeightExpression("eventWeight");

    finalLoader->PrepareTrainingAndTestTree("", "",
        "nTrain_Signal=34550:nTest_Signal=8638:"
        "nTrain_Background=100521:nTest_Background=25131:"
        "SplitMode=Random:SplitSeed=12345:"
        "NormMode=NumEvents:!V");

    TString finalOptions = commonOptions + ":" + layouts[bestConfig] + ":" + trainingStrategy;

    factory->BookMethod( finalLoader, TMVA::Types::kDL, "DNN", finalOptions);

    factory->TrainAllMethods();
    factory->TestAllMethods();
    factory->EvaluateAllMethods();

    outputFile->Write();
    outputFile->Close();

    delete factory;
    delete finalLoader;
    delete cvLoader;
    sigFile->Close();
    bkgFile->Close();
    delete sigFile;
    delete bkgFile;
}
```

Κώδικας 13: Βελτιστοποίηση και εκπαίδευση του ταξινομητή BDT για $n_j \geq 2$

```
#include <iostream>
#include <vector>
#include "TFile.h"
#include "TTree.h"
#include "TString.h"
#include "TSystem.h"
#include "TMVA/Tools.h"
#include "TMVA/Factory.h"
#include "TMVA/DataLoader.h"
#include "TMVA/CrossValidation.h"
#include "TMVA/Types.h"

void BDT_nj2()
{
    TMVA::Tools::Instance();
    std::vector<int> depthValues = {1, 2, 3, 4, 5, 6, 7, 8};

    TFile* sigFile = TFile::Open("nj2_signal.root");
    TFile* bkgFile = TFile::Open("nj2_background.root");
    if (!sigFile || sigFile->IsZombie() || !bkgFile || bkgFile->IsZombie()) {
        std::cerr << "ERROR opening input files." << std::endl; return; }

    TTree* sigTree = (TTree*)sigFile->Get("signal");
    TTree* bkgTree = (TTree*)bkgFile->Get("background");
    if (!sigTree || !bkgTree) {
        std::cerr << "ERROR loading input trees." << std::endl;
        sigFile->Close(); bkgFile->Close(); return; }

    // 5-fold cross-validation
    TFile* cvFile = TFile::Open("BDT_nj2_CV_results.root", "RECREATE");
    if (!cvFile || cvFile->IsZombie()) {
        std::cerr << "ERROR creating CV output file." << std::endl;
        sigFile->Close(); bkgFile->Close(); return; }

    TMVA::DataLoader* cvLoader = new TMVA::DataLoader("cv_BDT_nj2");

    cvLoader->AddVariable("pT_mumu", 'F');
    cvLoader->AddVariable("Y_mumu", 'F');
    cvLoader->AddVariable("cosTheta", 'F');
    cvLoader->AddVariable("jet1_pt", 'F');
    cvLoader->AddVariable("jet1_eta", 'F');
    cvLoader->AddVariable("dPhi_j1_mumu", 'F');
    cvLoader->AddVariable("jet2_pt", 'F');
    cvLoader->AddVariable("jet2_eta", 'F');
    cvLoader->AddVariable("dPhi_j2_mumu", 'F');
    cvLoader->AddVariable("pT_jj", 'F');
    cvLoader->AddVariable("Y_jj", 'F');
    cvLoader->AddVariable("dPhi_jj_mumu", 'F');
    cvLoader->AddVariable("m_jj", 'F');
    cvLoader->AddVariable("met", 'F');
    cvLoader->AddVariable("HT", 'F');

    cvLoader->AddSpectator("m_mumu", 'F');
    cvLoader->AddSpectator("eventNumber", 'l');
    cvLoader->AddSpectator("jet_n", 'I');
    cvLoader->AddSpectator("channelNumber", 'i');

    cvLoader->AddSignalTree(sigTree, 1.0);
    cvLoader->AddBackgroundTree(bkgTree, 1.0);
    cvLoader->SetSignalWeightExpression("eventWeight");
    cvLoader->SetBackgroundWeightExpression("eventWeight");
```

```
cvLoader->PrepareTrainingAndTestTree("", "",
    "nTrain_Signal=34550:nTest_Signal=8638:"
    "nTrain_Background=100521:nTest_Background=25131:"
    "SplitMode=Random:SplitSeed=12345:"
    "NormMode=NumEvents:!V");

TString cvOptions =
    "!V:!Silent:!ModelPersistence:AnalysisType=Classification:"
    "SplitType=RandomStratified:NumFolds=5";

TString commonOptions =
    "!H:!V:NTrees=800:MinNodeSize=2.5%:"
    "BoostType=AdaBoost:AdaBoostBeta=0.5:UseBaggedBoost:"
    "BaggedSampleFraction=0.5:SeparationType=GiniIndex:"
    "nCuts=20:NegWeightTreatment=IgnoreNegWeightsInTraining";

TMVA::CrossValidation cv("BDT_CV_nj2", cvLoader, cvFile, cvOptions);

for (int depth : depthValues) {
    TString methodName = Form("BDT_D%d", depth);
    TString methodOptions =
        commonOptions + Form(":MaxDepth=%d", depth);

    cv.BookMethod(TMVA::Types::kBDT, methodName, methodOptions);}

cv.Evaluate();
const auto& results = cv.GetResults();

if (results.size() != depthValues.size()) {
    std::cerr << "ERROR: unexpected number of CV results." << std::endl;
    delete cvLoader; cvFile->Close();
    sigFile->Close(); bkgFile->Close(); return;}

// Select MaxDepth with the highest mean CV AUC
double bestAUC = -1.0;
int bestDepth = -1;

for (size_t i = 0; i < results.size(); ++i) {
    double meanAUC = results[i].GetROCAverage();
    if (meanAUC > bestAUC) {
        bestAUC = meanAUC;
        bestDepth = depthValues[i];}}

cvFile->Write();
cvFile->Close();

// Final training with the selected MaxDepth
if (gSystem->AccessPathName("dataset_BDT_nj2"))
    gSystem->mkdir("dataset_BDT_nj2");

if (gSystem->AccessPathName("dataset_BDT_nj2")) {
    std::cerr << "ERROR creating dataset_BDT_nj2 directory." << std::endl;
    delete cvLoader; sigFile->Close(); bkgFile->Close(); return;}

TFile* outputFile = TFile::Open("dataset_BDT_nj2/BDT_results.root", "RECREATE");

if (!outputFile || outputFile->IsZombie()) {
    std::cerr << "ERROR creating final output file." << std::endl;
    delete cvLoader; sigFile->Close(); bkgFile->Close(); return;}
```

```
    TMVA::Factory* factory = new TMVA::Factory(
        "BDT_nj2", outputFile,
        "!V:!Silent:Color:DrawProgressBar:"
        "Transformations=I:AnalysisType=Classification");

    TMVA::DataLoader* finalLoader = new TMVA::DataLoader("dataset_BDT_nj2");

    finalLoader->AddVariable("pT_mumu", 'F');
    finalLoader->AddVariable("Y_mumu", 'F');
    finalLoader->AddVariable("cosTheta", 'F');
    finalLoader->AddVariable("jet1_pt", 'F');
    finalLoader->AddVariable("jet1_eta", 'F');
    finalLoader->AddVariable("dPhi_j1_mumu", 'F');
    finalLoader->AddVariable("jet2_pt", 'F');
    finalLoader->AddVariable("jet2_eta", 'F');
    finalLoader->AddVariable("dPhi_j2_mumu", 'F');
    finalLoader->AddVariable("pT_jj", 'F');
    finalLoader->AddVariable("Y_jj", 'F');
    finalLoader->AddVariable("dPhi_jj_mumu", 'F');
    finalLoader->AddVariable("m_jj", 'F');
    finalLoader->AddVariable("met", 'F');
    finalLoader->AddVariable("HT", 'F');

    finalLoader->AddSpectator("m_mumu", 'F');
    finalLoader->AddSpectator("eventNumber", 'l');
    finalLoader->AddSpectator("jet_n", 'I');
    finalLoader->AddSpectator("channelNumber", 'i');

    finalLoader->AddSignalTree(sigTree, 1.0);
    finalLoader->AddBackgroundTree(bkgTree, 1.0);
    finalLoader->SetSignalWeightExpression("eventWeight");
    finalLoader->SetBackgroundWeightExpression("eventWeight");

    finalLoader->PrepareTrainingAndTestTree("", "",
        "nTrain_Signal=34550:nTest_Signal=8638:"
        "nTrain_Background=100521:nTest_Background=25131:"
        "SplitMode=Random:SplitSeed=12345:"
        "NormMode=NumEvents:!V");

    TString finalOptions = commonOptions + Form(":MaxDepth=%d", bestDepth);

    factory->BookMethod( finalLoader, TMVA::Types::kBDT, "BDT", finalOptions);

    factory->TrainAllMethods();
    factory->TestAllMethods();
    factory->EvaluateAllMethods();

    outputFile->Write();
    outputFile->Close();

    delete factory;
    delete finalLoader;
    delete cvLoader;
    sigFile->Close();
    bkgFile->Close();
    delete sigFile;
    delete bkgFile;
}
```